R Guide

Jaewon Kim

2025-12-13

Table of contents

Welcome to "Introduction to R" **6**

1 Structure of R Studio **7**
1.1 Creating a Project . . . 7
1.2 UI of R Studio . . . 8
1.3 Using Console . . . 8
1.4 Creating a script . . . 9

2 Data Types and Variables **11**
2.1 Data Types . . . 11
2.2 Variables . . . 11
2.3 Simple Calculations . . . 12
2.4 Logical Calculations . . . 13

3 Vectors **14**
3.1 Vectors with Numbers . . . 14
3.1.1 Ways to create vectors . . . 14
3.1.2 Calculations with Vectors . . . 16
3.1.3 Functions helpful for numeric vectors . . . 17
3.2 Vectors with Boolean . . . 19
3.2.1 Functions you should know for Boolean vectors . . . 19
3.3 Vectors with Text . . . 20
3.3.1 Functions helpful for text vectors . . . 21
3.4 Other vectors with special data tpyes . . . 23
3.4.1 Vectors with NA . . . 23
3.4.2 Vectors with NaN . . . 24
3.4.3 Index . . . 24

4 Matrix **25**
4.1 Creating a matrix . . . 25
4.2 Index of matrix . . . 27
4.3 Calculation of matrix . . . 31
4.4 Functions regarding matrix . . . 34
4.4.1 rownames(), colnames() . . . 34
4.4.2 t() . . . 35
4.4.3 Apply() . . . 36

4.5 Array . . . 36
4.5.1 Creating an array . . . 37
4.5.2 Calculation of Array . . . 39
4.5.3 Advanced caluclation . . . 39
4.6 Summary of Vector, Matrix, and Array. . . . 40

5 List **41**
5.1 Object and Class . . . 41
5.1.1 What is Object? . . . 41
5.1.2 Class . . . 43
5.1.3 Attributes . . . 43
5.2 Summary & List . . . 44
5.3 Creating a List . . . 45
5.4 Indexing List . . . 46
5.5 Editing List . . . 49
5.5.1 Adding Component . . . 49
5.5.2 Editing Component . . . 50
5.5.3 Removing Component . . . 52
5.5.4 Vector to list, list to Vector . . . 52
5.6 Functions with List . . . 54
5.6.1 lapply() . . . 54
5.6.2 sapply() . . . 55
5.6.3 mapply() . . . 56
5.7 Summary . . . 57

6 Data Frame **58**
6.1 Categorial Variables and Factors . . . 59
6.1.1 Factors and Levels . . . 59
6.1.2 Ordered Levels . . . 61
6.1.3 Editing Level orders . . . 62
6.2 Creating a data frame . . . 63
6.3 Indexing Data Frame . . . 65
6.4 Functions in data frame . . . 69
6.5 Reading, Editing, Saving Data Frame . . . 69
6.5.1 Working with Text Files . . . 69
6.5.2 Working with CSV data . . . 72
6.5.3 Saving Multiple Objects . . . 74
6.6 Summary . . . 74

7 Editing Data Frame **75**
7.1 tidy data (Organized Data) . . . 75
7.2 tidyverse package . . . 78
7.3 What does dplyr do? . . . 80

7.4 Loading Example Dataset . . . 82
7.5 Selecting Rows . . . 82
7.5.1 filter() . . . 82
7.5.2 slice() . . . 85
7.6 Selecting Columns . . . 88
7.6.1 select() . . . 88
7.7 Adding Columns . . . 92
7.7.1 mutate() . . . 92
7.8 Summarizing Data . . . 93
7.8.1 summarize() . . . 93
7.8.2 group_by() . . . 94
7.9 Pipline Operator . . . 96
7.10 Summary . . . 98

8 Vizualizing Data with ggplot2 99
8.1 Components in ggplot2() . . . 99
8.1.1 Three Mandatory Components . . . 101
8.2 mapping Argument . . . 103
8.2.1 Global versus Local mapping . . . 105
8.3 aes() . . . 108
8.3.1 color . . . 108
8.3.2 shape . . . 109
8.3.3 size and alpha . . . 110
8.3.4 Manually Setting Arguments . . . 112
8.3.5 group . . . 114
8.4 facet . . . 118
8.4.1 1D facet . . . 119
8.4.2 2D facet . . . 122
8.5 geom() . . . 124
8.5.1 Simple statistics to ggplot() . . . 124
8.5.2 More geom() types . . . 143
8.6 Theme . . . 162
8.7 Coordinates() . . . 164
8.7.1 name() . . . 166
8.7.2 breaks(), minor_breaks() . . . 167
8.7.3 trans . . . 168
8.7.4 position . . . 168
8.7.5 coord() . . . 169
8.8 Colors . . . 171
8.9 Summary . . . 174

9 Dimension Reduction 175
9.1 Why do we need it? . . . 175

9.2 PCA (Principal Components Analysis) 178
9.2.1 Concept of PCA 178
9.2.2 Running PCA 181
9.2.3 Scree Plot 185
9.2.4 Loading Plot 197
9.3 Conclusion 202

Welcome to “Introduction to R”

Here, you will learn how to use R for data analysis. Try practice questions, and feel free to contact me anything you have questions.

1 Structure of R Studio

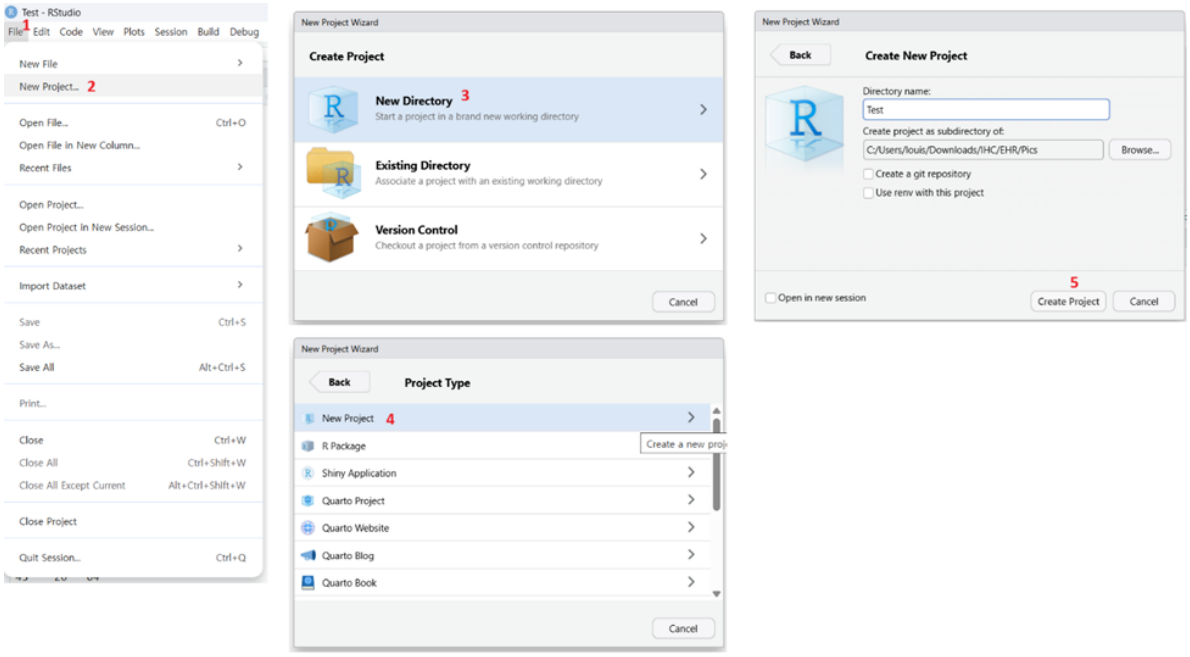
R is a programming language. R Studio is a UI that helps you write, run, and organize your R code. Let's explore how R Studio looks and what each section does.

1.1 Creating a Project

When we write codes and save data, it's important to keep everything organized so that R can remember where things were saved. To tell R Studio where to save and find your files, we use project. It's like a home for your codes and data.

Let's make a new project.

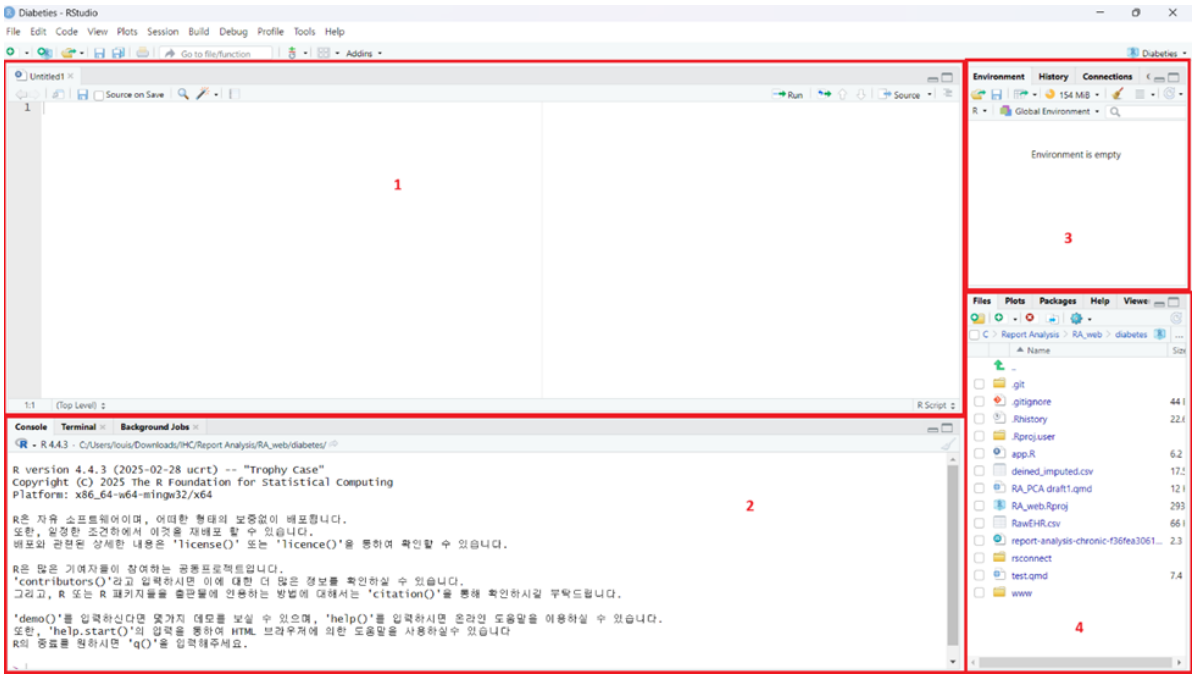
1. Click "File -> New Project -> New Directory -> New Project"
2. Choose location of folder you want to be, a good name, and click "Create Project"



Congrats! You just made your first project in R. Now let's start exploring what features are there in R Studio.

1.2 UI of R Studio

On the main page of R Studio, you'll see 4 main sections.



1 – Script Editor: This is where you write and save your code.

2 – Console: It's a communication window between you and the computer. You can give him commands to execute, and console will show you what it's doing.

3 – Environment: You'll work with countless variables. This section shows what variables you've saved as you run your code.

4 – Directories: Sometimes you need to load data or files for your code. This section shows which files are in your working folder. Default is folder where your project is.

1.3 Using Console

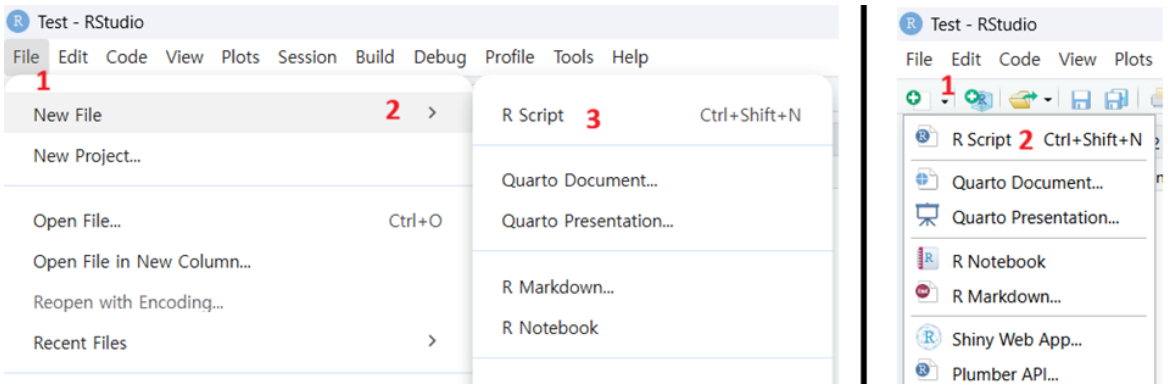
So Console is where actual works are done. Let's try using it. Input 1 + 1 and press enter. You'll see it's printing 2, the result of what you asked R to calculate.

```
> 1 + 1
[1] 2
>
```

However, Console is one-time-communicator. It can't remember anything you say or R executes, making you to retype everything to do same job. This gets annoying fast, especially if your code gets long. How can we make this easy? We'll figure it out in next the next section.

1.4 Creating a script

Script is a file that saves your code so that you don't have to rewrite same thing every time. Let's make a script. There are two ways to create a script.



Script and Quarto Document are most common files used to save the codes. Difference between to are what the default text is.

Type	Description
Script (Talking to computer)	Plain texts are considered as code, and you need to use special characters to insert "comments." It's for purely performing code it's written.
Quarto Document (Talking to human)	Plain texts are considered as text, and you need to insert "Chunks" to tell R Studio 'These are codes I want to run.' You can export Quarto Doc as beautiful report, which Script can't.

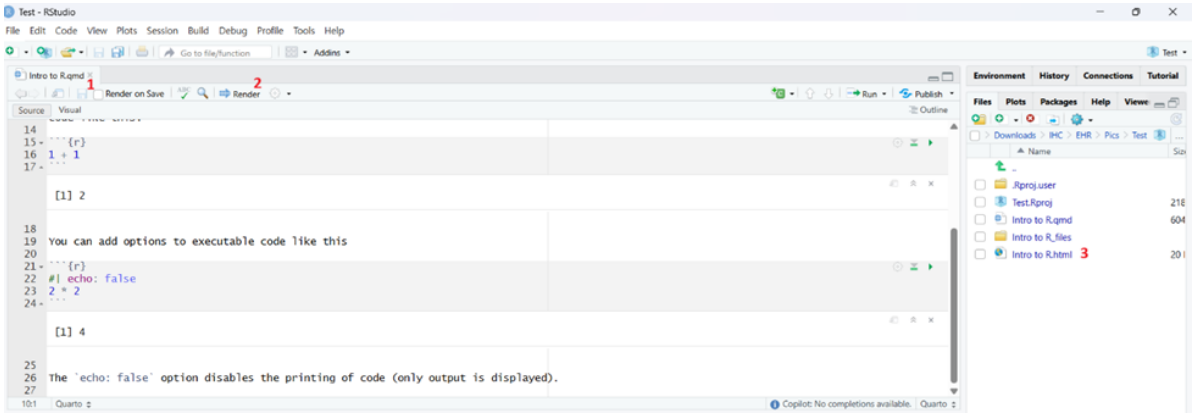
This year, we'll use Quarto Doc as we need to create reports and share codes to others. Create Quarto Doc. You can use whatever title and Author name you want. Leave other options as it is.

1 – # in the doc indicates the title. The more # you put, the smaller the text size gets. # (biggest) -> ####(Smallest)

2 – Area that wrapped with `{r} and` is called "Chunk," where codes you want to run goes. # in the chunks indicates comment, which will not get executed.

3 – When you click Run, you'll see which part of doc you want to execute. If you click specific chunk and "Run Current Chunk," only clicked chunk will be executed. This is one advantage of Quarto doc too.

Now, save your quarto doc and Render. You'll see that (Title).html file is created in your directory (1->2->3 marked in the picture). Congrats! You just created your first report using R. When you open the file, you'll see how quarto doc became visually pretty and clear.



2 Data Types and Variables

2.1 Data Types

Now, let's talk about how data are saved in R. R classifies data into four categories: Number, Text, Boolean, Special.

Type	Description
Number	Any numbers. They're classified into two categories. Integer – Type of number WITHOUT decimals. E.g.) 1, 10, 100 Float – Type of number WITH decimals. E.g) 1.24, 2.00 (Not int because of ".00"!)
Text	Anything you put into " ", regardless of what's inside. E.g) "Hi", "121"
Boolean	A logic data. It's either "True" or "False"
Special	Values that cannot be specified Null – Empty value without determined data type (Number/Text/Boolean) NA – Missing value. Known data type but nothing inside. NaN – Unable to mathematically define. E.g.) Saving "Hi" as int/float inf – Yes, infinite number. They're under special characters!

2.2 Variables

Now we know that there are multiple types of data. But how do we tell R to remember them? That's where variables come in! Variables are names that store data values.

You can declare a variable by using the assignment operator '<-' or '='.

```
Test1 <- 3
Test2 = 4
```

Once you declare the variable, you'll see them in the Environment tab. Also, you'll be able to call them by simply typing their names.

```
Test1
```

```
[1] 3
```

```
Test2
```

```
[1] 4
```

If it's not working, it's probably due to variable names. There're specific rules in naming - First character must be dot(.) or letter. It cannot be number or special character. Second character and later can be anything - You cannot use other function's name as variable name (sum, mean, etc. We'll cover functions as we go on) - Variable names are case-sensitive. "test" and "Test" are different variables. - Avoid using spaces in variable names. Use dot(.) or underscore(_) instead. E.g.) my.variable, my_variable

2.3 Simple Calculations

Here, we'll review basic operators in program language.

Type	Operation
Addition	+
Subtraction	-
Multiplication	*
Remainder	%
Exponents	/

Try some on the script you made! Also, you can use variables in calculations.

```
Test1 + Test2
```

```
[1] 7
```

There are multiple functions that will make calculations easy

Operation	**Description**
ceiling(x)	Round x up as "int" E.g.) 3.4 (float) becomes 3 (int)

floor(x)	Round x down as “int”
trunc(x)	Drop decimals as “float” E.g.) 3.4 (float) becomes 3.0 (float)
round(x, digits = n)	Round x as “float” at nth decimal place. default (If you omit “digits =n”) is to round at ones place.
sqrt(x)	$\sqrt{x}$
exp(x)	e^x
log(x, base = n)	$\log_n(x)$. Default is base with n = e
sin(x), cos(x), etc.	Trig functions!
Factorial(n)	n! = 1 x 2 x 3 x … x n
Others	There are many other function!

2.4 Logical Calculations

Sometimes you need more than simple calculations, such as “which number is bigger?” Return values of logical calculations are Boolean (True/False).

Type	Operation
Comparison	<, >, <=, >=
Equal	==
Not Equal	!=
Or	|
And	&

3 Vectors

Last chapter, we reviewed what variables are. Imagine we're storing heights of 50 students. Declaring 50 variables like "Height1", "Height2", ..., "Height50" takes forever. Instead, we can use "vector" to store multiple values in a single variable.

Vector is a 1-dimensional sequence of "same type" of data. For example, vector with int can only store int values. Length of Vector is equal to number of values stored in the vector.

3.1 Vectors with Numbers

3.1.1 Ways to create vectors

```
y <- c(2, 4, 6, 8, 10)
y
```

```
[1]  2  4  6  8 10
```

Again, vector must be same data type. For example, if you create one with mix of int and float, all values will be float. Why? because all int can be float by adding ".0" but float cannot be int without losing data. Let's see an example.

```
x <- c(1,3,5,7,9.2)
x
```

```
[1] 1.0 3.0 5.0 7.0 9.2
```

Recall c() meant to "connect." This means you can connect multiple vectors into one as long as they match data type.

```
z <- c(x, y)
z
```

```
 [1]  1.0  3.0  5.0  7.0  9.2  2.0  4.0  6.0  8.0 10.0
```

Question: What will happen if we flip the order of element? As you can see below, resulting order of elements in the vector also changes.

```
z <- c(y, x)
z
```

```
 [1]  2.0  4.0  6.0  8.0 10.0  1.0  3.0  5.0  7.0  9.2
```

Now we learned how to create vectors with number. But what if we want to label something and need 1 to 100? Typing them manually will take some time. Instead, there's a short cut to create continuous numbers with ":" operator. There there two points to remember when using :.

1. It has fixed increment of 1.
2. It follows the order from the left to the right. Check the example below.

```
1:10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
10:1
```

```
 [1] 10  9  8  7  6  5  4  3  2  1
```

```
0.2:3
```

```
[1] 0.2 1.2 2.2
```

What if you want to create vector with different increment? You can use seq() function. It has three main arguments - from, to, by. "from" is where the sequence starts, "to" is where it ends, and "by" is the increment.

```
seq(1, 10)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```r
seq(1, 10, by=2)
```

```
[1] 1 3 5 7 9
```

```r
seq(length=5, from=1, by=2)
```

```
[1] 1 3 5 7 9
```

If you want to create vector with repeated values, you can use rep() function. It has two main arguments - "x" and "times". "x" is the value you want to repeat, and "times" is how many times you want to repeat it.

```r
rep(5, times=4)
```

```
[1] 5 5 5 5
```

3.1.2 Calculations with Vectors

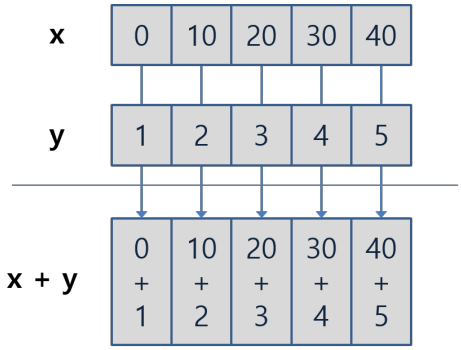
- Element wise operation Just like how we perform calculations with single numbers, we can do the same with vectors. One strong advantage of R is that it's one of rare programming languages that can perform "vectorized" calculations. This means you can perform calculations using same position of different vectors without complex logics. Let's perform example calculation using vectors we created.

```r
x <- seq(0, 40, by = 10)
y <- c(1:5)

x + y
```

```
[1]  1 12 23 34 45
```

```r
x * y
```

```
[1]   0  20  60 120 200
```



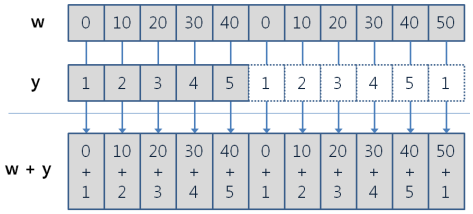
- Recycling rule What if the length of vectors are different? In this case, R will recycle the shorter vector until it matches the length of longer vector. Check the example below.

```
z <- c(rep(x, times=2), 50)
z + y
```

```
Warning in z + y:
```

```
 [1]  1 12 23 34 45  1 12 23 34 45 51
```

As you can see, addition is performed even though length of two vectors didn't match. The warning sign is indicating that length's are not multiple of each other, so it shows what we mean by "recycle until it matches the length." If it's difficult to grasp, there's a visualization below.



3.1.3 Functions helpful for numeric vectors

There are many functions that can be used with numeric vectors. Here are some of the most commonly used ones. While you might not end up not using them all, try them out and see which ones are useful for you!

Operation	Discription
length(x)	Return length of the vector
sum(x)	Return sum of all elements in the vector
mean(x)	Return average of all elements
median(x)	Return median of all elements
var(x)	Return variance of all elements
sd(x)	Return standard deviation of all elements
max(x)	Return highest number in the vector
min(x)	Return lowest number in the vector
range(x)	Return the range (highest and lowest) number
rank(x)	Return the "ranking" of each element from smallest to biggest
sort(x)	Reorganize vector from smallest to biggest elements
order(x)	Return index of smallest to biggest element's location
which.max(x)	Return the highest number's index
which.min(x)	Return the lowest number's index
which(x)	Return index of whatever element that satisfies the condition

rank/sort/order can be confusing, so here's the example.

```
x <- c(11,14,12,13,15)
x
```

```
[1] 11 14 12 13 15
```

```
# Rank returns the ranking of each element in entire vector
rank(x)
```

```
[1] 1 4 2 3 5
```

```
# Order returns Where the element is located in the original vector if we sorted it
order(x)
```

```
[1] 1 3 4 2 5
```

```
# Sort returns the vector sorted from smallest to biggest
sort(x)
```

```
[1] 11 12 13 14 15
```

3.2 Vectors with Boolean

Just like numeric vectors, you can create vectors with Boolean values (TRUE/FALSE). You can use c() function to create them. You can use T/F instead too.

```
booleanvec <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
booleanvec
```

```
[1]  TRUE FALSE  TRUE  TRUE FALSE
```

Special point about the Boolean vectors is that you can use logical operators to create vectors. Let's say you want to know which students scored higher than 60 on their final.

```
scores <- c(55, 70, 65, 40, 90)
passed <- scores > 60
passed
```

```
[1] FALSE  TRUE  TRUE FALSE  TRUE
```

3.2.1 Functions you should know for Boolean vectors

While it's not literally a boolean logic, there are some functions that can be useful when working with Boolean vectors: any/all/if/else

any() statement returns TRUE if at least one element in the vector is TRUE.

```
any(passed)
```

```
[1] TRUE
```

all() statement returns TRUE only if all elements in the vector are TRUE.

```
all(passed)
```

```
[1] FALSE
```

if and else statements are fundemental for all programming languages. They allow you to control the flow of your program based on conditions. However, we'll discuss about it briefly here since other contents are heavy as well. If you want to learn more about it, let me know anytime.

if statement returns true when the condition is met. It's like a gateway before executing next code. "Do if condition is met? Then move to next line of code. If not, then skip to else statement (or end the if statement)."

else statement is executed when the if condition is not met. It's like the alternative path when the if condition fails.

Both if and else statements are wrapped around {}.

Here's the example:

```
Iftest <- c(1,3,5,7,9)

if (any(Iftest > 5))
  {
  print("At least one element is greater than 5")
} else
  {
  print("No elements are greater than 5")
}
```

```
[1] "At least one element is greater than 5"
```

For readability, R allows you to combine ifelse into a single line. It looks like this: ifelse(Condition, Value if TRUE, Value if FALSE)

```
ifelse(any(Iftest > 5), "At least one element is greater than 5", "No elements are greater th
```

```
[1] "At least one element is greater than 5"
```

3.3 Vectors with Text

Just like other two types, this is just vector with a sequence of text values. You can create them using c() function.

```r
names <- c("Banana", "Apple", "Carrot", "Grape", "Whatelse")
names
```

```
[1] "Banana"   "Apple"    "Carrot"   "Grape"    "Whatelse"
```

Unlike numbers, you can't add or subtract texts. So there's other ways to handle manipulation of text vectors.

3.3.1 Functions helpful for text vectors

First, we should check if all elements are text, as vector require everything to be same type. But in huge data set, it might be tough to manually check everything. So we have ".ascharacter()" function to convert everything into text.

```r
mixedvec <- c("Apple", 2, "Banana", 4.5, TRUE)
mixedvec <- as.character(mixedvec)
mixedvec
```

```
[1] "Apple"  "2"      "Banana" "4.5"    "TRUE"
```

Try code by yourself. Did you notice that all inputs are as text when they were originally text, int, float, bool?

Now, let's look at how we can combine and split text vectors like addition/substraction. Both cases, they follow same element wise operation and recycle rule like numeric vectors.

For combining text vectors, we can use paste() function. If you want to add space between combined texts, use sep=" " argument. One point is that you can paste how many vectors you want, but you can call sep =" " one once per line. So if you want to have different seperators between multiple vectors, you'll need to call paste() multiple times. So it'll look like this:

```r
names <- c("Banana", "Apple", "Carrot", "Grape", "Whatelse")
price <- c("1.00", "2.50", "0.75")
combined <- paste(names, price, sep=": $") #Combine name and price with ": $" in between
combined #Do you notice recycling rule here?
```

```
[1] "Banana: $1.00"   "Apple: $2.50"    "Carrot: $0.75"   "Grape: $1.00"
[5] "Whatelse: $2.50"
```

For spliting text vectors, we can use strsplit() function. When you split, you use split=" " argument to specify where to split.

```
splitnames <- strsplit(combined, split=": ")
splitnames #Name and price are seperated using ":" as the split point
```

```
[[1]]
[1] "Banana" "$1.00"

[[2]]
[1] "Apple" "$2.50"

[[3]]
[1] "Carrot" "$0.75"

[[4]]
[1] "Grape" "$1.00"

[[5]]
[1] "Whatelse" "$2.50"
```

There are many other functions that's helpful with handling vectors. However, due to it's amount, we'll mention only few that's commonly used.

Opertiona	Description
nchar(x)	Return character counts of each elements in the vector x. *Literally all characters, including space, are counted!
substr(x, start, stop)	Return character counts of each element in the vector x, starting from 'start'th character to the 'stop'th character.
grep(target, x, ignore.case=T, fixed=T)	Return the index of element in vector x that contains "target". ignore.case asks if you want grep() to be case sensitive (T as yes). fixed asks if you want to use Regex (We don't be discussing about this here. Keep it true)
sub(target, replacement, x, ignore.case=T, fixed=T)	If any element in vector x contains "target," replace it to "replacement."
toupper(x) tolower(x)	change all characters in the vector x into upper/lower case

3.4 Other vectors with special data tpyes

Earlier, we said there's a special data types. During data analysis, handling these becomes essential as we don't know how data that doesn't align with your dataset will influence the general trend (hence analysis result). So let's briefly look at how to create vectors with these special data types, especially Na and NaN.

3.4.1 Vectors with NA

NA is used to represent missing values with undetermined data type. For example, if you're collecting final exam scores but completely missed one student's score, you can use NA to represent it.

How we do process NA without manually checking all data? We can use is.na() and na.omit() functions. is.na() checks if there is NA values, and na.omit() removes all NA values from the vector.

```
scores <- c(90, 85, NA, 70, 95, NA, 80)
is.na(scores)  #It'll return TRUE if there's NA in the vector
```

```
[1] FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE
```

```
scores <- na.omit(scores)  #It'll remove all NA
scores
```

```
[1] 90 85 70 95 80
attr(,"na.action")
[1] 3 6
attr(,"class")
[1] "omit"
```

Look at the output of na.omit(). Second/Fourth line - attr() shows what type of work is by the code. Ignore them First line - Vector after removing NA values Third line - Attribute showing which index values were removed due to NA Fifth line - What's done by the code (removing NA values)

However, using is.na() and na.omit() everytime can be time-consuming in big dataset. How can we "ignore" them without removing them? Many functions in R have "na.rm" argument. Setting na.rm=T will ignore NA values during calculation.

```r
scores <- c(90, 85, NA, 70, 95, NA, 80)
mean(scores, na.rm=T)  #Calculate mean while ignoring NA values
```

```
[1] 84
```

3.4.2 Vectors with NaN

NaN is used to represent mathematically undefined values. For example, if you try to divide 0 by 0, the result is undefined even thought we know that data type is numbers. In this case, R will return NaN. Just like NA, we can use is.nan() function to check if there's NaN values in the vector. To know if vector actually contains NaN, you must use is.nan() because is.na() will return TRUE for both NA and NaN.

3.4.3 Index

I mentioned "index" multiple times before, and you probably grasped somewhat sense of what it is. Index is a location of element in the vector. In R, index starts from 1 (not 0 like other programming languages). You can use index to call specific elements in the vector by indicating interested index in []. Here's the example:

```r
x <- c(10, 20, 30, 40, 50)
x[3]  #Call the 3rd element in the vector x
```

```
[1] 30
```

But one thing to be careful is that you cannot call two indicies at once. Try x[3,5] and see if it works.
Instead, you can use c() to connect multiple indices.

```r
x <- c(10, 20, 30, 40, 50)
x[c(2,4)]  #Call 2nd and 4th elements in the vector x
```

```
[1] 20 40
```

4 Matrix

Vector was a 1-dimensional data, meaning each vector can hold one type. For example, if we create vector of heights, all elements must be height numbers. However, what if we want need to hold details about the data? For example, height by gender?

Matrix is a 2-dimensional data structure, holding rows and columns. Just like vector, matrix must hold one type of data.

4.1 Creating a matrix

There's two ways to declare matrix.

1. Dividing vector into rows and columns with matrix()

```
c <- 1:10

m <- matrix(c, nrow = 2, ncol = 5)
m
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10
```

Here, did you notice the order of elements? R fills the matrix by column first, then row. If you want to fill by row first, use byrow = TRUE option.

```
m <- matrix(c, nrow = 2, ncol = 5, byrow = TRUE)
m
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    6    7    8    9   10
```

2. Combining vectors with rbind() and cbind() rbind() combines vectors by row, while cbind() combines by column. This means that rbind() will stack vectors on top of each other, while cbind() will place them side by side.

```
v1 <- c(1, 2, 3)
v2 <- c(4, 5, 6)
m_row <- rbind(v1, v2)
m_row
```

```
   [,1] [,2] [,3]
v1    1    2    3
v2    4    5    6
```

```
m_col <- cbind(v1, v2)
m_col
```

```
     v1 v2
[1,]  1  4
[2,]  2  5
[3,]  3  6
```

One thing to consider is the recycling rule. Whenever you're working with data sets, having different dimensions will cause shorter data set to repeat.

```
v1 <- c(1,2,3)
v3 <- c(7, 8)
m_row2 <- rbind(v1, v3)
```

```
Warning in rbind(v1, v3): number of columns of result is not a multiple of
vector length (arg 2)
```

```
m_row2
```

```
   [,1] [,2] [,3]
v1    1    2    3
v3    7    8    7
```

With a same logic, you can create matrix by combining vector with matrix or matrix with matrix using bind() functions.

```r
c <- 1:10
rbind(c, 11:20)
```

```
  [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
c    1    2    3    4    5    6    7    8    9    10
    11   12   13   14   15   16   17   18   19    20
```

4.2 Index of matrix

Just like vector, you can access specific elements of matrix using index. However, since matrix has two dimensions, you need to specify both row and column index.

When you index, you use matrix[row, column] format.Emptying either means "all of them," and you can also use range like how we created 1,2,3,4 with 1:4.

```r
test <- matrix(1:25, nrow=5, ncol=5)
test
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    6   11   16   21
[2,]    2    7   12   17   22
[3,]    3    8   13   18   23
[4,]    4    9   14   19   24
[5,]    5   10   15   20   25
```

```r
test[2, 3]  # Access element at 2nd row, 3rd column
```

```
[1] 12
```

```r
test[ , 4]  # Access all rows in 4th column
```

```
[1] 16 17 18 19 20
```

```r
test[5, 1:3]  # Access 1st-3rd column in 5th row
```

```
[1]  5 10 15
```

There is special about matrix in R. If you use minus, like [, -2], it means "Don't display (whatever you decide)". This means that you can manipulate matrix! Look at example below. Do you notice that 6-10 are missing?

```
test[ , -2]  # Access all rows except 2nd column
```

```
     [,1] [,2] [,3] [,4]
[1,]    1   11   16   21
[2,]    2   12   17   22
[3,]    3   13   18   23
[4,]    4   14   19   24
[5,]    5   15   20   25
```

```
test <- test[ , -2]  # By saving "test matrix without 2nd column", you can remove 2nd column
```

Like how matrix fills in order (like column first, then row), when you remove rows or columns, you can manipulate matrix beyond simply removing specific rows or columns. Look at example below. For example, 1:5 mean "1st to 5th row." Then what will happen if we do 5:1?

```
test[5:1, ]
```

```
     [,1] [,2] [,3] [,4]
[1,]    5   15   20   25
[2,]    4   14   19   24
[3,]    3   13   18   23
[4,]    2   12   17   22
[5,]    1   11   16   21
```

Because we're displaying test matrix from 5th row to 1st row, the order of rows get reversed! Pause and Think: what would test[order(a[, 2]),] do?

Like how we filtered out the matrix, we can manipulate them using logical operations. In this case, it only displays elements that's TRUE. Check example below.

```
people <- c("John", "Jane", "Jim", "Jill", "Jack")
Age <- c(28, 34, 29, 42, 23)
Height <- c(175, 160, 180, 165, 170)

Information <- cbind(people, Age, Height) # Imagine what output it would be!
Information[ , c(T,F,T)]  # Display only 1st and 3rd column (those with TRUE)
```

```
     people Height
[1,] "John" "175"
[2,] "Jane" "160"
[3,] "Jim"  "180"
[4,] "Jill" "165"
[5,] "Jack" "170"
```

```r
Information[Information[ ,2] > 30, ]  # Display only rows where Age (2nd column) is greater t
```

```
     people Age  Height
[1,] "Jane" "34" "160"
[2,] "Jill" "42" "165"
```

However, there is issue with the structure of matrix when we combine multiple vectors. Let's say you want to investigate average bloodpressure of a patient

```r
# Data with visiting date, bloodpressure, and patient id
record <- matrix(c("2023-01-01", 120, 123,
                   "2023-01-02", 130, 124,
                   "2023-01-01", 125, 456,
                   "2023-01-02", 135, 456), nrow=4, ncol=3, byrow=TRUE)
record
```

```
     [,1]         [,2]  [,3]
[1,] "2023-01-01" "120" "123"
[2,] "2023-01-02" "130" "124"
[3,] "2023-01-01" "125" "456"
[4,] "2023-01-02" "135" "456"
```

```r
Avg_BP <- mean(as.numeric(record[, 2]))  # Think, why do we need as.numeric() ?
Avg_BP
```

```
[1] 127.5
```

But you realized that you forgot the heights of patients, and you create new matrix.

```r
# Data with visiting date, height, bloodpressure, and patient id
record_fix <- matrix(c("2023-01-01", 150, 120, 123,
                       "2023-01-02", 150, 130, 124,
                       "2023-01-01", 150, 125, 456,
                       "2023-01-02", 150, 135, 456), nrow=4, ncol=4, byrow=TRUE)
Avg_BP <- mean(as.numeric(record_fix[, 2]))  # Run the same code again
Avg_BP
```

```
[1] 150
```

After fixing the matrix, running same code that gave us average blood pressure is giving us wrong result! This is because adding new column shifted location of existing information. Then how can we track which information is where? This is where rownames() and colnames() takes place to assign titles to rows and columns.

```r
colnames(record_fix) <- c("Date", "Height", "BloodPressure", "PatientID")
record_fix
```

```
     Date         Height BloodPressure PatientID
[1,] "2023-01-01" "150"  "120"         "123"
[2,] "2023-01-02" "150"  "130"         "124"
[3,] "2023-01-01" "150"  "125"         "456"
[4,] "2023-01-02" "150"  "135"         "456"
```

Now, we can access information by their names, not by their index. This way, even if we add new information, we can still access existing information without worrying about shifting index.

```r
Avg_BP <- mean(as.numeric(record_fix[ , "BloodPressure"]))
Avg_BP
```

```
[1] 127.5
```

Great fix! By the way, review old matrices like "Information" or m_row, and compare to "record." Do you notice two have colnames while one doesn't? This is where creating matrix by combining vectors (rbind, cbind) is better than creating matrix by dividing single vector (how we made record). When you use rbind() and cbind(), they recognize names of vectors, thinking "these values belong under the name of the vector!" As result, functions automatically assign colnames() or rownames() using vector names.

One last thing, matrix is 2x2. What if you only give one index like how we did in vector indexing? In matrix, inputing one index means matrix will behave like a vector. So, like how matrix fills by column first and then rows, matrix[i] will return index after scanning column and row order.

```
last <- matrix(1:9, nrow=3, ncol=3)
last
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
last[5] # Since it's 3x3, 5th index will be [2,2]. 1st column (3 elements), and 2nd column's
```

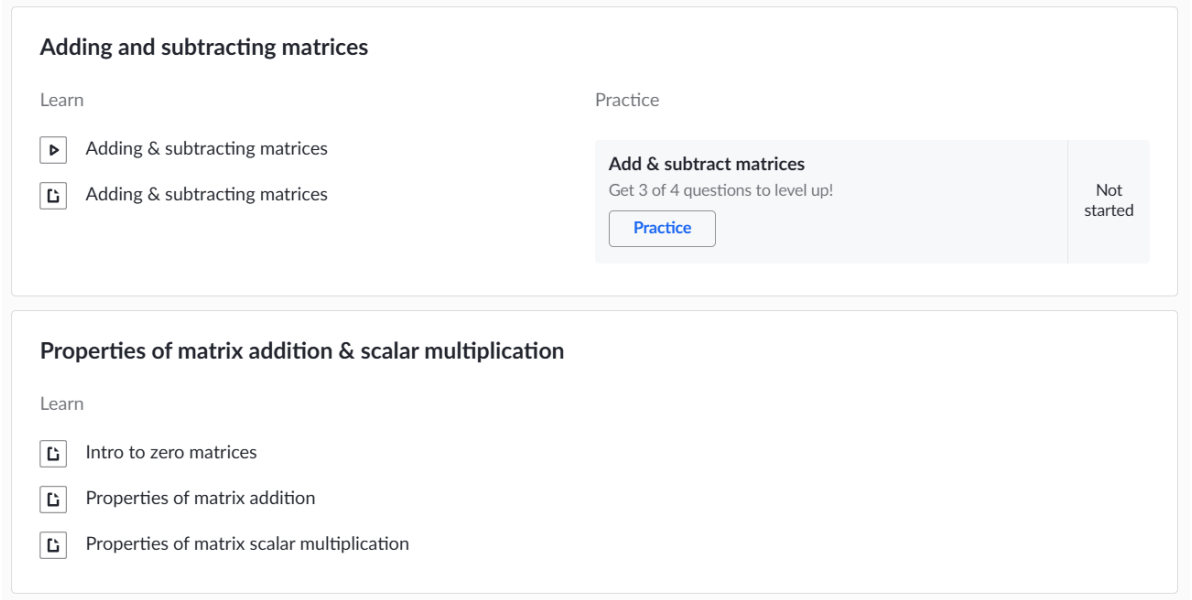
```
[1] 5
```

```
last[last > 5]  # Returns all elements greater than 5 as 1-dimensional vector
```

```
[1] 6 7 8 9
```

4.3 Calculation of matrix

Before we get into calculation of matrix, you should know the math behind matrix calculation. Please review concepts regarding matrix, adding/subtracting/multiplying matrices. Others are good to know but optional. Link is **here**



Now let's try calculating matrix in R. First, creating two matrices with same dimension.

```
A <- matrix(1:9, nrow=3, ncol=3)
B <- matrix(9:1, nrow=3, ncol=3)
A
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
B
```

```
     [,1] [,2] [,3]
[1,]    9    6    3
[2,]    8    5    2
[3,]    7    4    1
```

```
A + B
```

```
     [,1] [,2] [,3]
[1,]   10   10   10
[2,]   10   10   10
[3,]   10   10   10
```

```
B - A
```

```
     [,1] [,2] [,3]
[1,]    8    2   -4
[2,]    6    0   -6
[3,]    4   -2   -8
```

```
A * B
```

```
     [,1] [,2] [,3]
[1,]    9   24   21
[2,]   16   25   16
[3,]   21   24    9
```

```
A / B
```

```
          [,1]      [,2]     [,3]
[1,] 0.1111111 0.6666667 2.333333
[2,] 0.2500000 1.0000000 4.000000
[3,] 0.4285714 1.5000000 9.000000
```

Great! R is doing element-wise calculation. This means that R is adding/subtracting/multiplying/dividing each element in same position. For example, A[1,1] + B[1,1], A[1,2] + B[1,2], and so on.

One question, do matrix follow recycling rule? Try making two matricies with different dimension and adding them. You'll see the error - R cannot perform calculations that violates fundemental rules of math. However, when you're using vector and matrix, vectors can be recycled.

```
C <- c(0, 10, 100)

A + C
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]   12   15   18
[3,]  103  106  109
```

However, we should keep in mind that vector should be shorter than the number of elements in the matrix. If vector is longer, what are we adding leftover vector elements with?

4.4 Functions regarding matrix

###nrow(), ncol(), dim()

How do we know size of unknown matrix? nrow() and ncol() returns number of rows and columns, while dim() returns both as vector. dim() returns column size first! Don't get confused with [row, column] in indexing.

```
m <- matrix(1:12, nrow=3, ncol=4)
nrow(m)
```

```
[1] 3
```

```
ncol(m)
```

```
[1] 4
```

```
dim(m)
```

```
[1] 3 4
```

4.4.1 rownames(), colnames()

Earlier, we saw how we can assign row names and column names by rownames() <- c(names). However, can we do the opposite? Like, checking names of cols and rows instead of assigning? Yes! Instead of assigning through <-, you can use rownames(matrix) or colnames(matrix) to check names of rows and columns.

```
colnames(m)
```

```
NULL
```

```
rownames(m)
```

```
NULL
```

Here, we get Null (empty). Why is that? Look at how m looks like and think about it!

4.4.2 t()

Transpose matrix is matrix with switched rows and columns (for example, 2x3 becomes 3x2). In R, you can do this by t() function.

```
m
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
t(m)
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
[4,]   10   11   12
```

###Moving between matrix and vector

Sometimes, you may want to convert matrix into vector or vice versa. You can do this by as.vector() or c(). as.vector() unstrings matrix into single line (= vector). And do you remember how giving one input feature in indexing make matrix behave like vector? c() function does the same thing. If you do c(matrix) without telling what else to combine with, c() will combine matrix itself into one string, making it vector.

```
Testmat <- matrix(1:9, nrow=3, ncol=3)
as.vector(Testmat)
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
c(Testmat)
```

```
[1] 1 2 3 4 5 6 7 8 9
```

4.4.3 Apply()

Often, we need to perform calculation throughout row or column of the matrix (e.g. finding average height by each person, finding total score by each student). Instead of writing all elements individually, we can use apply() function.

apply() has three main arguments (inputs). X (target), Margin (which direction do you want to apply the function? 1 is row, 2 is column), and Function (work you want to perform)

```
Scores <- matrix(c(90, 85, 88,
                   78, 92, 80,
                   85, 87, 90), nrow=3, byrow=TRUE)
colnames(Scores) <- c("Math", "Science", "English")
Scores
```

```
     Math Science English
[1,]   90      85      88
[2,]   78      92      80
[3,]   85      87      90
```

```
#Let's find average score of each subjects
math_avg <- (Scores[1,1] + Scores[2,1] + Scores[3,1]) / 3
math_avg
```

```
    Math
84.33333
```

```
#Doing this for each column is annoying. Let's use apply()
exam_avg <- apply(Scores, 2, mean) # Find mean (function) of matrix (Scores) by column (2)
exam_avg
```

```
    Math  Science  English
84.33333 88.00000 86.00000
```

4.5 Array

Vector is 1-dimensional, matrix is 2-dimensional. What if we want to hold more than 2 dimensions? For example, what if we want to hold height and weight by gender? This is where array comes in. Array is multi-dimensional data structure, meaning it can hold 3 or more dimensions. However, like vector and matrix, array can only hold one type of data.

4.5.1 Creating an array

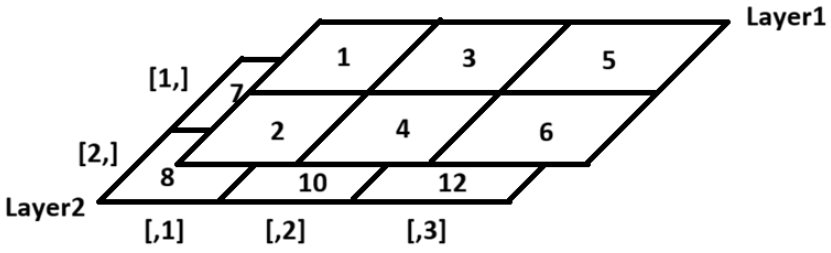
We can declare by matrix in a similar way to how we declared vectors. 1. Define matricies you want to stack 2. Use array() function to stack them * Remember, dimension should be identical to be able to stack them.

```
mat1 <- matrix(1:6, nrow=2, ncol=3)
mat2 <- matrix(7:12, nrow=2, ncol=3)
arr <- array(c(mat1, mat2), dim = c(2, 3, 2)) # 2 rows, 3 columns, 2 layers
arr
```

```
, , 1

     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2

     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12
```

Unlike how matrix or vectors were single layer, you'll see that 3D array has layers, shown as ", , 1" and ", , 2". Arrays fill data just like matrix. Column first, then rows (next columns), then layers. In the same way, you can index them. Array[row, column, layer]. If you expand it to 4,5,6th dimensions? You just keep adding commas! For example, array[row, column, layer, 4th dimension, 5th dimension]

It is difficult to imagine it, so I'll visualize 3D array we made (arr) below.



Just like colnames and rownames in matrix, you can assign names to each dimension using dimnames() function.

```
dimnames(arr) <- list(
  Row = c("R1", "R2"),
  Column = c("C1", "C2", "C3"),
  Layer = c("L1", "L2")
)
arr
```

```
, , Layer = L1

    Column
Row  C1 C2 C3
  R1  1  3  5
  R2  2  4  6

, , Layer = L2

    Column
Row  C1 C2 C3
  R1  7  9 11
  R2  8 10 12
```

Now, you can index using names too!

```
arr["R1", "C2", "L1"]
```

```
[1] 3
```

However, there is one thing that you should be careful. Let's look at the example below

```
arr[1, 2:3, 1:2] # Access 1st row, 2nd-3rd column, all layers
```

```
      Layer
Column L1 L2
    C2  3  9
    C3  5 11
```

We asked them to display first row, 2nd-3rd column, all layers: 3D data. However, output is 2x2 matrix, a 2D data. Like this, when R can display result in lower dimension, it will do it. If you want to keep the original dimension, use drop = False argument. If you look at the example below, you'll see that output is seperated into two layers again.

```
arr[1, 2:3, 1:2, drop = FALSE]
```

```
, , Layer = L1

    Column
Row  C2 C3
  R1  3  5

, , Layer = L2

    Column
Row  C2 C3
  R1  9 11
```

4.5.2 Calculation of Array

Omitted. Everything is same as Matrix calculation.

4.5.3 Advanced caluclation

4.5.3.1 Solving equations with matrix

Matrix and Array can be used in a lot of places. Due to its compact representation of high dimnesional data, we often use matrix to represent equations with multiple dimension. For example,

3x + 2y = 5 4x + 6y = 12

can be written as [3, 2, 5]; [4, 6,12] (; is used to seperate rows). So, we condensed multiple equations into one 2x3 matrix. This also means that we can use matrix to solve complex equations.

Using the matrix above as example, we have two parts: variables (x and y), and constants (5 and 12). We can separate them into two matrices:

```r
A <- matrix(c(3, 2,
              4, 6), nrow=2, byrow=TRUE) # Coefficient matrix
B <- matrix(c(5, 12), nrow=2) # Constant matrix
colnames(A) <- c("x", "y")
```

Now, you can just use solve() to find values of x and y!

```r
solution <- solve(A, B)
solution
```

```
  [,1]
x  0.6
y  1.6
```

4.5.3.2 Eigenvalues and Eigenvectors

This is core concept of linear algebra, which will help you understand more advanced machine learning algorithms. However, it's difficult to understand without proper math background, so we'll come back to this later. For now, watch this playlist to get introduction to linear algebra. I highly recommend this.

Link: Linear Algebra by 3Blue1Brown

4.6 Summary of Vector, Matrix, and Array.

Throughout Chapter 3 and 4, we reviewed basic concepts of vector (1D), matrix (2D), and array (Multiple D). While we defined three different data structure, are they really different? Think about this. We declared matrix by combining vectors into two dimensions. Then, we declared 3D array by stacking matrices. However, way to higher dimension is just repeating same process: adding a new axis by combining existing structure. So, doesn't that mean matrix is vector and array is also vector in the end? You're right, it is.

So array is often seen as 'generalized vector' or 'generalized matrix.' While vector is tied to 1D and matrix is tied to 2D, array can build as much as dimensions you want using vector. Next time, we'll explore the list (like a shopping list or to-do list). You'll be able to verify using code how vector to array are all same thing. Then, keep this in your mind and see you in the next chapter!

5 List

5.1 Object and Class

So far we looked at vector, matrix, and array. However, one limitation was that all elements in them need to be same data type. Revisit "Information" matrix where we stored patient's visit date, blood pressure, and patient ID. All elements were character type ("text") despite of bp/ID being number because "Date" was in text format.

But why is that really the case? Why can we declare vectors with text, numbers, or logic, but not two at the same time?

To answer this, we need to understand what object and type are.

5.1.1 What is Object?

Single component we decide to store was called variable. Then how about some kind of produces like vectors? Single unit of whatever data that R can recall, like vectors, matrix, and functions, are called "Objects."

In stored data's perspective, Objects are divded into two types. "Atomic" and "Generic (or List)" List, we'll talk about it later.

Atomic object is what we've been reviewing: A vector, data we can store with single data type. Some examples are vectored made out of logical, integer, character, float (double), complex, etc.

5.1.1.1 Characteristics of Object

All object has two intrinsic characteristics: Type and Length.

Type is literally type of data we discussed in Chapter 2. Nothing more to say. Throughout previous chapters, we talked about how to declare the vectors. However, we never talked about how to check data type of random vector. We use function called typeof() for that.

```
a <- c(T, F, T, T); typeof(a)
```

```
[1] "logical"
```

```
b <- c(1.2, 0.3, 3.5, 2.3); typeof(b)
```

```
[1] "double"
```

Length is another characteristic of object. It tells how many elements are in the object. For example, vector with 4 elements will have length of 4. We can check length of object with length() function.

```
length(a)
```

```
[1] 4
```

Intrinsic characteristics are intrinsic because they can't be changed once object is created. For example, if we create a vector with character type, we can't change it to numeric type unless we manually code it to be so.

But Question here:

```
a <- c(1, 2, 3, 4)
b <- matrix(c(1:4), nrow = 2, ncol = 2)
a
```

```
[1] 1 2 3 4
```

```
b
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Consider two following data. Both have same length (4), and same data tpye (int). There intrinsic characteristics are same, but they look different (one is 1D vector, another is 2D matrix). How do R distinguish different objects with identical intrinsic characteristics?

5.1.2 Class

Class solves the question. Class is an "atrribute" that you can assign to the object to tell R how to treat the object.

For example, vector "a" and matrix "b" both have same length (4) and same data type (int). However, R treats them differently because their class is different. Vector "a" is stored with class "numeric" while matrix "b" has class "matrix." We can check class of objects by class().

```
class(a) # This was vector
```

```
[1] "numeric"
```

```
class(b) # This was matrix
```

```
[1] "matrix" "array"
```

5.1.3 Attributes

Another question is, "Okay, now we know that it's the class that differentiate vector and matrix. But, how do R know the matrix is 2x2?, not 4x1?"

When we declare objects, R saves all the accessory information (people call it meta data. It's anything besides intrinsic characteristics, like class) as "Attributes." You can check all attributes using attributes() function. Let's try it.

```
attributes(a)
```

```
NULL
```

```
attributes(b)
```

```
$dim
[1] 2 2
```

As you can see, vector "a" has no attributes, meaning that it's just a simple 1D structure. However, matrix "b" has dimension of 2,2 which means 2x2. This dimension attribute ($dim) tells R that this object is 2D structure with 2 rows and 2 columns.

5.1.3.1 Editing attributes

Earlier, we said that intrinsic characteristics can't be changed once object is created. However, attributes can be changed after creation. Let's try changing dimension attribute of vector "a" to make it 2x2 matrix. We use attr() function to edit them.

```
a
```

```
[1] 1 2 3 4
```

```
attr(a, "dim")
```

```
NULL
```

```
# Like rownames(), function(target, detail) displays detail of the target. Here, we're askin

attr(a, "dim") <- c(2, 2) # Like assigning rownames, now "<-" assigns (2,2) to dim attribute
a # Now a is 2x2 matrix!
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```
class(a) # And class of a is now matrix!
```

```
[1] "matrix" "array"
```

There are many attributes, such as dimnames() (name of each dimension), names() (name of each element in 1D structure), etc. You can check all attributes using attributes() function. You can look them up and play with it!

5.2 Summary & List

So what was the answer to the question at the beginning of "Object and Class," what was the point of whole section about them?

Consider that intrinsic characteristics are saved and doesn't change when objects are declared. This also means that different type of data are saved and recalled through different pathway. For example, integer vector are stored where only integer exists. In the same way, method

that vector is loaded for calculation is also something only integer can understand. Therefore, vector/matrix/array can hold only single data type.

However, sometimes we need to save multiple data type like patient information vector (visiting date in character, vital signs in numeric). This is where "Generic (because it's not atomic)" object, or "List" comes in. List is special object that can hold multiple data type at the same time. Each element in list can be different data type, and even different structure (vector, matrix, array, another list, etc)!

Each element in the list is called "Component."

5.3 Creating a List

Just like atomic objects, list are declared through list() function in two ways: unnamed list and named list.

- Unnamed list: list(Compoenent1, Component2, Component3))
- Named list: list(Name1 = Component1, Name2 = Component2, Name3 = Component3)

Let's try an example.

```
my_list <- list(Name = c("Patient1", "Patient2", "Patient3"), #Component 1
                Age =  c(30, 45, 28), #Component 2
                Diabetic = c(F, T, F), #Component 3
                Visit.Date = matrix(c("2023-12-1",
                                      "2024-2-16",
                                      "2025-3-20"),
                                      nrow = 3, byrow = TRUE) #Component 4
)

print(my_list)
```

```
$Name
[1] "Patient1" "Patient2" "Patient3"

$Age
[1] 30 45 28

$Diabetic
[1] FALSE  TRUE FALSE

$Visit.Date
```

```
     [,1]
[1,] "2023-12-1"
[2,] "2024-2-16"
[3,] "2025-3-20"
```

Now you see all Character, Numeric, Logical, and even Matrix are stored in single object. Here, we have each components named like "Name," but it will show up as [[1]], [[2]], etc., if they're unnamed.

One important thing to remember while creating a list is that component's name isn't automatically assigned like how creating vector using cbind() named columns whether you name them or not.

```
var1 = c(3,5)
var2 = c(7,9)

cbind(var1, var2) # Columns are named var1 and var2 automatically
```

```
     var1 var2
[1,]    3    7
[2,]    5    9
```

```
list(var1, var2)
```

```
[[1]]
[1] 3 5

[[2]]
[1] 7 9
```

So like the my_list, you must manually assign names to list, like variable1 = var1.

5.4 Indexing List

Unlike vectors, List use different notation for index: listname[[index]]. Check the exmaple below:

```
class(my_list[1])
```

```
[1] "list"
```

```
class(my_list[[1]])
```

```
[1] "character"
```

One is list, and another is character. What's the difference? To remember the difference, think of level of information you need to call.

- Single bracket [ ]: Index of one level deeper (like calling component in vector X). So In list, [] will call entire "Component" itself. For example, my_list[i] will print entire i-th component. However, this is still a list object because list had two factors: Name and value.

```
my_list[c(1, 2)] # Calls entire first and second component
```

```
$Name
[1] "Patient1" "Patient2" "Patient3"

$Age
[1] 30 45 28
```

- Double bracket [[ ]]: Index of two level deeper. So In list, [[]] will call specific "element" inside the component. For example, my_list[[i]] will return contents in i-th component. This time, this is vector since it only prints out values without name. Check the example, can you spot the difference?

```
my_list[[1]] # Calls elements inside first component (Patient1, Patient2, Patient3)
```

```
[1] "Patient1" "Patient2" "Patient3"
```

```
my_list[1]
```

```
$Name
[1] "Patient1" "Patient2" "Patient3"
```

Important thing you must remember is that [] recalls component as list (list with 1 component), while [[]] recalls component as vector. So, for example, my_list[1][2] won't work since my_list[1] has only first element (2nd element of my_list[1] is non-existent). However, my_list[[1]][2] will work since my_list[[1]] is vector with 3 elements.

```
my_list[1][2] # As you can see, it prints out NULL.
```

```
$<NA>
NULL
```

```
my_list[[1]][2] # Calls 2nd element in first component (Patient2
```

```
[1] "Patient2"
```

Like vectors, you can also use name to call specific component/element. A lot of times, people use its alternatives, $, to call component by name. It's easier to type. There output is same!

```
my_list[["Age"]]
```

```
[1] 30 45 28
```

```
my_list$Age
```

```
[1] 30 45 28
```

Also, like vectors, you can recall elements in component by adding another index.

```
my_list$Name[2:3] # Calls 2nd and 3rd element in "Name" component
```

```
[1] "Patient2" "Patient3"
```

One special point about indexing an list in R is that you can call component by portion of name as long as it's unique. Let's try it.

```
my_list$Visit
```

```
     [,1]
[1,] "2023-12-1"
[2,] "2024-2-16"
[3,] "2025-3-20"
```

Even though name of that component was "Visit.Date," R was able to recall it because no other component started with "Visit." However, be careful when you have multiple components with similar names.

5.5 Editing List

Let's say you created a list. However, what if you need to change information to update record? Let's look at how we can do it.

5.5.1 Adding Component

You can add component to list by simply adding a new component to the list.

```
my_list$Height
```

```
NULL
```

Right now, my_lit doesn't have "height component" that it returns NULL. Let's add it.

```
my_list$Height <- c(170, 180, 160) # Adding Height component
my_list
```

```
$Name
[1] "Patient1" "Patient2" "Patient3"

$Age
[1] 30 45 28

$Diabetic
[1] FALSE  TRUE FALSE

$Visit.Date
     [,1]
[1,] "2023-12-1"
[2,] "2024-2-16"
[3,] "2025-3-20"

$Height
[1] 170 180 160
```

Wow! By adding Height component, now we see that list is extended.However, think about this: what will happen if we add componenet to way far back of the last element?

```
my_list[[7]] <- c("A+", "B", "O-") # Adding Blood Type component at 6th index. Our Last compo
my_list[5:7]
```

```
$Height
[1] 170 180 160

[[2]]
NULL

[[3]]
[1] "A+" "B"  "O-"
```

As you can see, we skipped 6th component slot and added bloodtype to 7th. R will automatically fill skipped componenet in the list with NULL.

5.5.2 Editing Component

In the same way with adding the component, we can edit elements within the component by overwriting existing component (assigning new content to desired index).

```
my_list$Height <- c(4,5,6) # Changing Height into 4, 5, 6
my_list$Height
```

```
[1] 4 5 6
```

```
my_list[[5]] <- c(150, 160, 170) # You can also edit using number index
my_list$Height
```

```
[1] 150 160 170
```

Do you remember how we could recall partial list (component as list) using []? In the same way, you can edit multiple components of list using []. However, remember that assignment (new edits) must be list, since list[] acts as a list, not vector.

```
my_list[1:2]
```

```
$Name
[1] "Patient1" "Patient2" "Patient3"

$Age
[1] 30 45 28
```

```
my_list[1:2] <- list(Name = c("NewPatient1", "NewPatient2", "NewPatient3"),
                             Age = c(35, 50, 40)) # Editing Name and Age component at once
my_list[1:2]
```

```
$Name
[1] "NewPatient1" "NewPatient2" "NewPatient3"

$Age
[1] 35 50 40
```

Now, name and age of patients are changed at once!

One thing to be careful is that recycling rule applies in the list too. So if I try to add a component with different length, R will recycle the values.

```
my_list[9:11] <- list(c("Text"), 12) # Adding Weight component with length
```

```
Warning in my_list[9:11] <- list(c("Text"), 12): number of items to replace is
not a multiple of replacement length
```

```
my_list[9:11] # Now, you'll see "Text" twice because of recycling
```

```
[[1]]
[1] "Text"

[[2]]
[1] 12

[[3]]
[1] "Text"
```

5.5.3 Removing Component

Recall how skipping component resulted in Null, empty. We can use this to remove component from the list! Simpy assign Null to the component you want to remove. It's like editing a list, but with empty component.

```
my_list[8:11] <- NULL # Removing my_list[9:11] we just added. But think, why [8:11], not[9:1
my_list[[7]] <- NULL # Removing blood type component
```

Now, try calling blood type by my_list[[7]]. It will return error since it doesn't exist anymore.

5.5.4 Vector to list, list to Vector

Recall how we could change data type among vectors, like number to character? In the same way, we can convert vector to list, and list to vector.

```
Test_Vector <- c(10, 20, 30, 40)
Test_List <- as.list(Test_Vector) # Converting vector to list
Test_List # List to vector will be as.vector()
```

```
[[1]]
[1] 10

[[2]]
[1] 20

[[3]]
[1] 30

[[4]]
[1] 40
```

Also, we can connect different lists using c() like vectors.

```
Another_List <- list("Text1", "Text2")
Combined_List <- c(Test_List, Another_List)
Combined_List
```

```
[[1]]
[1] 10

[[2]]
[1] 20

[[3]]
[1] 30

[[4]]
[1] 40

[[5]]
[1] "Text1"

[[6]]
[1] "Text2"
```

One thing to be careful is the hierarchy of list. When you use c(), you're putting two vectors next to each other (equally). This results a list with single layer (component1, componenet2, etc, like example above). However, if you create a list by combining other lists, combined list will be compoenent of new list, creating multiple layers.

```
Combined_List2 <- list(Sublist1 = Test_List, Sublist2 = Another_List)
Combined_List2
```

```
$Sublist1
$Sublist1[[1]]
[1] 10

$Sublist1[[2]]
[1] 20

$Sublist1[[3]]
[1] 30

$Sublist1[[4]]
[1] 40


$Sublist2
$Sublist2[[1]]
```

```
[1] "Text1"

$Sublist2[[2]]
[1] "Text2"
```

Now, you'll see that both Test_list and Another_List became components under name of Sublist1, and Sublist2, respectively (2 layered list). Then, could we transform multi-layered list into vector? Yes, we can use unlist() function for that.

```
unlist(Combined_List2)
```

```
Sublist11 Sublist12 Sublist13 Sublist14 Sublist21 Sublist22
     "10"      "20"      "30"      "40"   "Text1"   "Text2"
```

5.6 Functions with List

Many functions we learned can be applied to list. However, apply() doesn't work on list because list is a generic object with different components, while apply() requires same data type as it perform same work throughout the object.

5.6.1 lapply()

lapply works by apply(list, function). Let's look at the example.

```
Sample1 <- list(A = c(1,2,3), B = c(4,5,6), C = c(7,8,9))
Sample1
```

```
$A
[1] 1 2 3

$B
[1] 4 5 6

$C
[1] 7 8 9
```

If we want to find mean of each component, using mean() requires us to calculate through three lines of code (mean() per component). Using, lapply does the same in single line.

```r
lapply(Sample1, mean)
```

```
$A
[1] 2

$B
[1] 5

$C
[1] 8
```

Here, remember that lapply() applies function to each componenet and return the result in identical structure to the input (so list in this case).

5.6.2 sapply()

Many times, we want the calculation to return in simplified structure like vector so that we don't have to index list every time (which is more complex and time consuming). sapply() does the same thing with lapply(), but return the result in vector or matrix.

```r
sapply(Sample1, mean)
```

```
A B C
2 5 8
```

```r
sapply(Sample1, range)
```

```
     A B C
[1,] 1 4 7
[2,] 3 6 9
```

Now, instead of multi components from lapply ($A, $B, $C), we have single vector with three elements. This is easier to use in many cases. If result requires multiple rows/columns, like range(), output will be matrix.

5.6.3 mapply()

So lapply() and sapply() apply function to single list. But what if we have multiple lists and want to apply function to them at once? mapply() is the answer. mapply() works by mapply(function, list1, list2, …). Let's look at example.

```
mList <- list(A = c(1,2,3), B = c(4,5,6), C = c(7,8,9))
mList2 <- list(D = c(10, 11, 12, 13), E = c(14, 15), F = c(16, 17, 18))

mapply(c, mList, mList2) # Perform c() of each component in mList and mList2
```

```
$A
[1]  1  2  3 10 11 12 13

$B
[1]  4  5  6 14 15

$C
[1]  7  8  9 16 17 18
```

As you can see, each component in mList and mList2 are combined using c(). Two points to notice here: 1. mapply() is order sensitive. If you put mList2 first, mList2 will be at the front of the result. 2. When combining components with different names, names of first list are prioritized (Here, six components are combined into three components. Since A, B, C are from first list, they're kept as names). 3. mapply() performs dimension reduction

What do we mean by #3? Look at the output below:

```
mList3 <- list(G = c(10, 11, 12), H = c(13, 14, 15), I = c(16, 17, 18))
mapply(c, mList, mList3)
```

```
      A  B  C
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
[4,] 10 13 16
[5,] 11 14 17
[6,] 12 15 18
```

Here, we have a single matrix instead of list with three components. Why is this? Review the length of each component.

- mList and mList2: Since mList and mList2's component pairs (A/D, B/E, C/F) had different lengths, mapply() is required to print output as seperate components: a list.
- mList and mList3: However, in mList and mList3, all componenets (A/G, B/H, C/I) had same length (3). Therefore, mapply() was able to combine them into single matrix instead of list with three components, reducing the dimension.

Like this, mapply() will try to be useful and print most simplified structure possible.

5.7 Summary

List, which combines multiple data types into single object, is absolute must-know concept in R as many real world data mixes data types (like title and values). Also, this concept of list will be foundation of the highlight of data structure, a dataframe (we'll cover this in next chapter). Then, see you in next chapter!

And if you read this far, email me. I'll unlock next chapters.

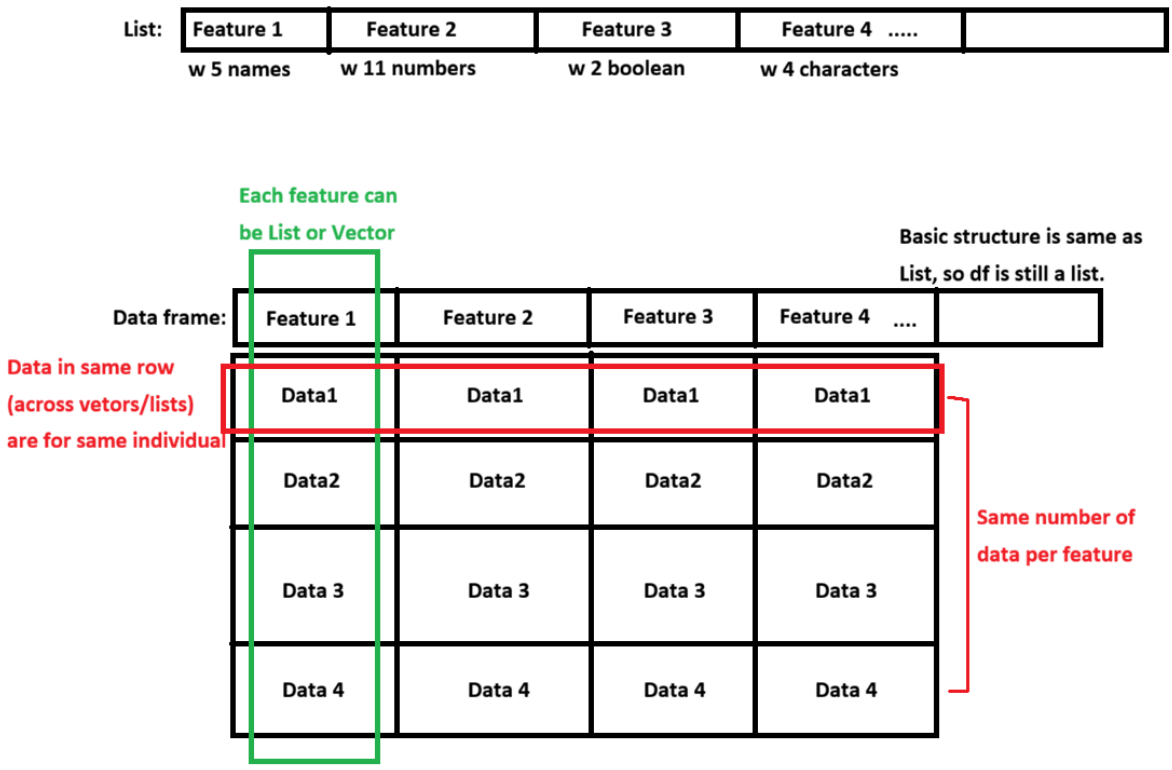
6 Data Frame

So far we reviewed many types of object, from vector to list. Now, how do we find a 'pattern' of collection of data? It can be regression, clustering, or any other methods, but there's something that never changes: All valid data points have result (Output) corresponding to the observation (Input). In other words, you must have pairs of x and y values. Think about it, can you draw best fit line between patient's age and blood pressure, can you draw a line only using age?

However, data structures we saw so far have limitation with giving us (X,Y) pairs:

- Matrix: While matrix gives nice 2d, row-column structure, it could not handle multiple data types.
- List: This was solution to limitation of matrix. While it holds mix of data at once, it lacks strict structure. As result, it is difficult to guarantee that your list component of "Ages" and "Diagnoses" have same number of entries to form clean, row-by-row data pairs.

So what's the solution? How do we keep perfect two dimensional (m x n) data structure that stores mixed data type? We use "data frame" (or df), and this is absolute standard of structure in data analysis. It builds top of the list (so techincally df is still a list) and adds solid 2d structure. Not sure about how that looks like? here's the figure:



6.1 Categorial Variables and Factors

6.1.1 Factors and Levels

Before we get into exploring components in df, we first need to understand how data are stored into each cells. Let's say we have people are voting on the survey: What is your blood type? 1)A 2)B 3)O 4)AB

Here, what we care about bloodtype (A/B/O/AB). Let's create a list with responses.

```
results <- c(1,3,2,4,3,4, 5)

mean(results)
```

```
[1] 3.142857
```

In the survey, 1~4 meant to represent corresponding to the blood type. However, calculating average returned 3.14. What does mean, people have 3.14 blood type in general? It doesn't

make sense! Also, there was no option '5' but R didn't tell us that something is wrong. Why is this happening? It's because contents in the list thought they were numerical values.

factor() is a data class (if you forgot what class, object, and attributes were, review earlier chapter!) that contains overall question. Level is a attribute that contains rules/options within the factor. Let's check what that means by creating a factor.

```
factoredresult <- factor(results, levels = 1:4)
# Create a data structure with values of 'results' and options of 'levels'

factoredresult
```

```
[1] 1    3    2    4    3    4    <NA>
Levels: 1 2 3 4
```

```
mean(factoredresult) #Will this return 3.14 again?
```

```
Warning in mean.default(factoredresult):
NA

[1] NA
```

Now, look at the output. Do you see levels (options) we assigned? Since available levels are 1~4, 5 in the list returns NA. Also, because list is a factor with levels instead of a numbers, mean(factoredresult) is NA. It correctly reflects the fact that we can't average survey options!

But looking at numbered levels might be confusing. If we have list of 132423124123213… with thousands of entires, manually reviewing the list will be nightmare. To resolve readability, we can use levels() to assign level with custom names.

```
factoredresult
```

```
[1] 1    3    2    4    3    4    <NA>
Levels: 1 2 3 4
```

```
levels(factoredresult) <- c("A", "B", "O", "AB")
#levels in the list was 1, 2, 3, 4. Now, we're giving them a name for each!

factoredresult
```

```
[1] A    O    B    AB   O    AB   <NA>
Levels: A B O AB
```

Clean!

6.1.2 Ordered Levels

Sometimes, you might have a ordered levels. For example, a data set where levels are "very bad", "bad" "normal", "good", "very good"

While you can declare factors() for this, there is one headache when getting a statistics: you can't filter out people with more than certain satisfaction level. For example, you won't be able to view individuals with normal or higher satisfactions, because R takes level names as text and that they don't know their rankings/orders. Let's try it

```
satisfaction <- c("verybad", "verybad", "verygood", "good", "normal", "bad", "good")

factoredsat <- factor(satisfaction, levels = c("verybad", "bad", "normal", "good", "verygood"
factoredsat
```

```
[1] verybad  verybad  verygood good     normal   bad      good
Levels: verybad bad normal good verygood
```

```
factoredsat > "bad"
```

```
Warning in Ops.factor(factoredsat, "bad"):  (factors)
'>'    .
```

```
[1] NA NA NA NA NA NA NA
```

As you can see, it returns NA because R doesn't know relative orders of the levels. To resolve this issue, we use argument called ordered.

```
orderedsat <- factor(satisfaction, ordered = TRUE, levels = c("verybad", "bad", "normal", "g
orderedsat
```

```
[1] verybad  verybad  verygood good     normal   bad      good
Levels: verybad < bad < normal < good < verygood
```

```
orderedsat > "bad"
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE FALSE  TRUE
```

Unlike how last cell just printed out all levels, this time level are ordered by <. But remember, ordered rank levels from smallest to largest in input order. If you put levels = c("good", "bad"), it will remember as bad > good.

And now, we can filter data with threshold because levels are ordered. Do you see how it says True or False based on data in the factor (in the cell below)?

```
orderedsat
```

```
[1] verybad  verybad  verygood good     normal   bad      good
Levels: verybad < bad < normal < good < verygood
```

```
orderedsat > "bad" #Checking which data has satisfaction with normal or higher
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE FALSE  TRUE
```

6.1.3 Editing Level orders

While assigning factor() and levels are good, table and graph will print levels in input order. For example, if we say levels = c(2,4,3,1), then table will have column ordered in 2,4,3,1.

```
factoredresult
```

```
[1] A    O    B    AB   O    AB   <NA>
Levels: A B O AB
```

```
table(factoredresult)
```

```
factoredresult
 A  B  O AB
 1  1  2  2
```

Then, what if we want to display something on the first column? we use relevel(). ref argument make assigned level to be displayed first while keeping the rest same order.

```
factoredresult2 <-relevel(factoredresult, ref = "O")

factoredresult2
```

```
[1] A    O    B    AB   O    AB   <NA>
Levels: O A B AB
```

```
table(factoredresult2)
```

```
factoredresult2
 O  A  B AB
 2  1  1  2
```

Now, do you see how levels are listed in O A B AB, instead of A B O AB?

6.2 Creating a data frame

We know what df is, and that column is feature while rows are individuals. Then, let's make one!

Remember: df has m x n structure. This means that all column (features) must have same number or rows (observations). When we create a df, data on the same row will become a dataset for that individual.

To declare df, we use command data.frame(factor1, factor2, factor3,...)

```
name <- c("Alex", "Banana", "Clown", "Donkey", "Elex")
gender <- c("M", "F", "M", "M", "F")
height <- c(1, 2, 4, 3, 4)

tests <- data.frame(name, gender, height)
tests
```

```
    name gender height
1   Alex      M      1
2 Banana      F      2
3  Clown      M      4
4 Donkey      M      3
5   Elex      F      4
```

As we discussed, df has characteristic of both list and matrix. This means that we can add more data or generate df through cbind() or rbind()! Or, since df is still a list, we can add columns just like how we did it in previous chapter.

```r
bt <- factor(c("A", "O", "B", "O", "AB"))
cbind(tests, bloodtype = bt) # Again, remember that length should be same as other columns.
```

```
    name gender height bloodtype
1   Alex      M      1         A
2 Banana      F      2         O
3  Clown      M      4         B
4 Donkey      M      3         O
5   Elex      F      4        AB
```

```r
rbind(tests, c(name = "Frona", gender = "F", height = "6", bloodtype = "A"))
```

```
Warning in rbind(deparse.level, ...): number of columns of result, 3, is not a
multiple of vector length 4 of arg 2
```

```
    name gender height
1   Alex      M      1
2 Banana      F      2
3  Clown      M      4
4 Donkey      M      3
5   Elex      F      4
6  Frona      F      6
```

```r
tests$COVID19 <- c(T, F, F, T, T)
tests
```

```
    name gender height COVID19
1   Alex      M      1    TRUE
2 Banana      F      2   FALSE
3  Clown      M      4   FALSE
4 Donkey      M      3    TRUE
5   Elex      F      4    TRUE
```

Q1. Why do you see warning in rbind()? Why aren't we seeing bloodtype column after adding new row to tests using rbind()? Send me an answer to email! (Hint: declaring variable)

There is ONE VERY IMPORTANT concept you need to remember when adding data into data frame (also matrix). They are UNPACKED. What does it mean by that?

```r
test_id <- c(1, 2)
bp_matrix <- matrix(c(97, 76, 102, 134), nrow = 2, ncol = 2)
test_df <- data.frame(ID = test_id, bp = bp_matrix)

test_df
```

```
  ID bp.1 bp.2
1  1   97  102
2  2   76  134
```

Do you see how 2x2 matrix is added to df? Earlier we said column in df are each features. So when we add matrix into df, it feels like entire matrix should become a single column in df (since whole matrix is a feature we're trying to add). However, this didn't happen. When data are combined into single df (or matrix), R discect each data into all individual columns. So we don't have one matrix in one column of df, but 2 vectors (from matrix) across 2 columns of data frame.

Remember, data frame and matrices are unpacked.

6.3 Indexing Data Frame

Recall that data frame is still a list. So, indexing goes same way as list does.

```r
tests
```

```
    name gender height COVID19
1   Alex      M      1    TRUE
2 Banana      F      2   FALSE
3  Clown      M      4   FALSE
4 Donkey      M      3    TRUE
5   Elex      F      4    TRUE
```

```r
tests[["height"]]
```

```
[1] 1 2 4 3 4
```

```r
tests$name
```

```
[1] "Alex"   "Banana" "Clown"  "Donkey" "Elex"
```

```
tests[[1]] ## $name and [[1]] are same thing because 1st column is name
```

```
[1] "Alex"   "Banana" "Clown"  "Donkey" "Elex"
```

```
tests$name[1:3]
```

```
[1] "Alex"   "Banana" "Clown"
```

One detail to remember here is the difference between indexing using [[]] and []. tests[] recalls the column of data (tests df) we're looking at. This means that [1] extracts first column of 'data frame,' which is a portion of 'data frame.' So, [] returns data frame. However, [[]] recalls 'content only' of data. This means that [[1]] extracts feature (list/vector) stored in first column of data frame, which returns list/vector but not data frame. Do you see how one is still list (data frame) while another becomes vector (char)?

```
a <- tests[1]
attributes(a)
```

```
$names
[1] "name"

$row.names
[1] 1 2 3 4 5

$class
[1] "data.frame"
```

```
typeof(a)
```

```
[1] "list"
```

```
b <- tests[[1]]
attributes(b)
```

```
NULL
```

```
typeof(b)
```

```
[1] "character"
```

Also, list had similar structure to matrix, where number of rows of all features are identical. Because of this characteristic, you can index or perform operations on data frame just like how we treat matrix.

```
tests[1:2, ]
```

```
    name gender height COVID19
1   Alex      M      1    TRUE
2 Banana      F      2   FALSE
```

```
tests[-3]
```

```
    name gender COVID19
1   Alex      M    TRUE
2 Banana      F   FALSE
3  Clown      M   FALSE
4 Donkey      M    TRUE
5   Elex      F    TRUE
```

```
tests[1:2, c(1,4)]
```

```
    name COVID19
1   Alex    TRUE
2 Banana   FALSE
```

```
tests$height > 2
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
tests[tests$height != 2, c("name", "height")]
```

```
    name height
1   Alex      1
3  Clown      4
4 Donkey      3
5   Elex      4
```

Also, we can sort dataframe like how we did in matrix.

```
tests[order(tests$height, decreasing = T), ]
```

```
    name gender height COVID19
3  Clown      M      4   FALSE
5   Elex      F      4    TRUE
4 Donkey      M      3    TRUE
2 Banana      F      2   FALSE
1   Alex      M      1    TRUE
```

```
tests[order(tests$height, tests$COVID19), ] #What do you think order(A, B) does?
```

```
    name gender height COVID19
1   Alex      M      1    TRUE
2 Banana      F      2   FALSE
4 Donkey      M      3    TRUE
3  Clown      M      4   FALSE
5   Elex      F      4    TRUE
```

Last, if you been running codes with me, you probably felt you're getting tired with repeating object name over and over, like tests[order(tests$height, tests$COVID19),]. You had to state df tests 3 times to filter the data. Now, we have alternative option, subset().

subset() allows you to omit the object name, and use feature name directly.

```
subset(tests, height > 2)
```

```
    name gender height COVID19
3  Clown      M      4   FALSE
4 Donkey      M      3    TRUE
5   Elex      F      4    TRUE
```

```r
subset(tests, height > 2, c(1:2))
```

```
    name gender
3  Clown      M
4 Donkey      M
5   Elex      F
```

6.4 Functions in data frame

Nothing new. Since it has characteristic of both list and matrix, all functions you use in list and matrix can be applied to data frames too. Review them!

6.5 Reading, Editing, Saving Data Frame

Since we know how to work with data frame, let's talk about now we load the table or data frame, edit them, and rewrite them from your computer.

6.5.1 Working with Text Files

First, let's create example text file format df. Open your notepad, and copy paste following. Save the file as "Test_txt" in the directory your project is in.

```
Name Scores
A 25
B 31
C 92
```

If you're not sure if the file is in your directory, you can check your working directory using getwd(). You can also use list.files() to see files exist in your directory and check if you saved file in correct location.

```r
getwd() #get working directory
```

```
[1] "C:/Users/louis/Downloads/IHC/EHR/Pics/R Guide"
```

```
list.files() #list files exist in working directory
```

```
 [1] "_quarto.yml"                   "cover.png"
 [3] "DataFrame_6.html"              "DataFrame_6.qmd"
 [5] "DataFrame_6.rmarkdown"         "DataFrame_6_files"
 [7] "Dimension-Reduction_9_files"   "Dimension Reduction_9.qmd"
 [9] "docs"                          "Editing Data Frame_7.qmd"
[11] "ggplot()_8.qmd"                "images"
[13] "index.html"                    "index.qmd"
[15] "List_5.html"                   "List_5.qmd"
[17] "Matrix_4.html"                 "Matrix_4.qmd"
[19] "R Guide.Rproj"                 "references.bib"
[21] "Revised_CSV"                   "Revised_CSV2"
[23] "site_libs"                     "Structure_of_R_Studio_1.html"
[25] "Structure_of_R_Studio_1.qmd"   "Test_excel.csv"
[27] "Test_txt.txt"                  "Variables_Calculations_2.html"
[29] "Variables_Calculations_2.qmd"  "Vectors_3.html"
[31] "Vectors_3.qmd"                 "Vitals.csv"
```

```
list.files(pattern = "txt") #list files with contains "txt" in the name only
```

```
[1] "Test_txt.txt"
```

When you load your file, the file should be in your working directory. If not, you must put full location where file exists. You will use following format to load the text file: read.table(name or location, header)

```
testfile <- read.table("Test_txt.txt", header = T) #If file is in the working directory
```

```
Warning in read.table("Test_txt.txt", header = T): 'Test_txt.txt'
readTableHeader
```

```
testfile
```

```
  Name Scores
1    A     25
2    B     31
3    C     92
```

```
textfile2 <- read.table("C:/Users/louis/Downloads/IHC/EHR/Pics/R Guide/Test_txt.txt", header
```

```
Warning in read.table("C:/Users/louis/Downloads/IHC/EHR/Pics/R
Guide/Test_txt.txt", : 'C:/Users/louis/Downloads/IHC/EHR/Pics/R
Guide/Test_txt.txt'  readTableHeader
```

```
textfile2
```

```
  Name Scores
1    A     25
2    B     31
3    C     92
```

Here, let's look at what header does. When we write a table, we always state what row and columns are, right? In programming languages, those 'title' for columns and rows are called headers. That said, let's compare two

```
testfile
```

```
  Name Scores
1    A     25
2    B     31
3    C     92
```

```
testfile2 <- read.table("Test_txt.txt", header = F)
```

```
Warning in read.table("Test_txt.txt", header = F): 'Test_txt.txt'
readTableHeader
```

```
testfile2
```

```
    V1     V2
1 Name Scores
2    A     25
3    B     31
4    C     92
```

Do you see how Name and Score are counted as column names when header is true, but counted as data when header is false? Instead, header = F assigned random names to fill out the missing column names.

Now, open our txt file and add row names. So it would look like following.

```
Name Scores
1 A 25
2 B 31
3 C 92
```

Now, run the code in previous cell again with header = T. Then, run another with header = F. Now you'll run into error and data table doesn't load. Why is that? (Hint: Where does row names fits if row names are not headers?) So it's always important to check if data has headers!

Typically in data analysis, you want to remove row names from raw data. It's because R (or other program languages) automatically assignes row names as it loads the data. So if we manually assign index for each row, you might run into headers issue (like you just saw). Or, because R automatically assign index/rowname, R will think manually assigned index must be actual data and ruin your data set even before any analysis.

6.5.2 Working with CSV data

Well we reviewed how to load the data tables, but most of data are normally in excel files. Especially, a csv file (comma-seperated values), meaning each cells are seperated using comma.

In the same way, let's create sample csv file first. Open your excel, write same content as data table above, and save it as csv file in your working directory. Now, again, you should see it in your "files" tab, bottom right of R Studio, if it saved correctly. Let's try to load this csv. We use read.csv() for that.

```
testcsv <- read.csv(file = "Test_excel.csv")
testcsv
```

```
  X Name Score
1 1    A    25
2 2    B    31
3 3    C    92
```

While you can also use header = T, you must tell sep = ',' This is because of difference between text file and csv file. In text file, space are used as indicator that tells R "we use space to differentiate each cells." Similarly, CSV comma to separate cells. So, we just state that comma is used to separate cells, not space. To check this point, let's do quick practice:

Load csv without sep = ',' argument. Then, try to calculate average of score column. Can you get the answer or do you get NA?

Another point is row name (or index of data). When you load the testcsv, indices (1,2,3) have randomly assigned column name X. Why? (Hint: We just talked about this). If you figured out why, let's remove them.

```
edited_csv <- testcsv[, -1]
edited_csv
```

```
  Name Score
1    A    25
2    B    31
3    C    92
```

Since we made change, let's save it into csv file. We'll use write.csv(data you're saving, file = "file name").

```
write.csv(edited_csv, file = "Revised_CSV")
```

Once you run that code, you'll see a csv file generated in your working directory. But wait, if you open the file, do you see how row name is saved? This will create issue we discussed earlier if we try to load that csv next time. To prevent this, we use argument 'row.names = F' (don't save row names in generated CSV file).

```
write.csv(edited_csv, file = "Revised_CSV2", row.names = F)
```

If you open CSV2, we don't have row names saved and R will re assign it when we load CSV file again!

We use same method for writing table.

```
write.table(saving data, file name, row.name = F)
```

6.5.3 Saving Multiple Objects

We reviewed how to save csv. But what if we have 100 data structures to save? Repeating write.csv 100 times will be both tiring and time consuming. For this situations, we have functions that exports/imports all objects created in current environment (variables/objects saved from codes you run so far). Let's say you made vector X, list Y, and data frame Z. Then, we can export them into single file using follow command.

```
#I didn't do it here, but make objects and try it yourself!
Let's pretend you made X, Y, Z.

save(X,Y,Z, file = "File name")

Then, you'll see that file called (File name.RData) is created. You can load it using follow
load("File name.RData")
```

6.6 Summary

Now, we learned how to create data frame, a structured data format that has perfectly matching dimension. Each features have same number of variables saved, allowing us to perform data analysis while being able to track which row was who. Next Chapter, we'll explore how we use package called 'dplyr' to edit data frames more conveniently. See you soon!

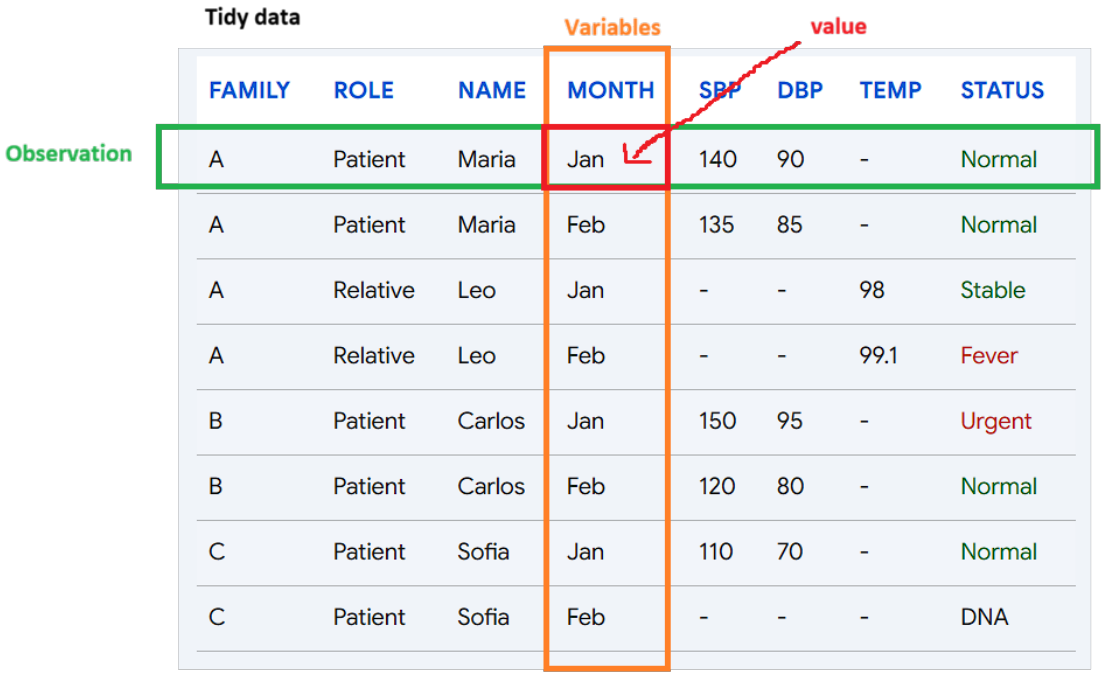
7 Editing Data Frame

In data analysis, you always must pre-process the data. This is to make sure that outliers won't influence quality of your analysis, and to check general statistics/summary of data for detailed analysis pipeline.

In the previous chapter, we learned how to create data frame. While we could perform calculations or editing df using operations we reviewed in Chapter 3-5, they often require repetitive typing or heavy manual step-by-step calculations. In this chapter, we'll learn package (like an extension) called dplyr which allows us to perform df edit in simple ways.

7.1 tidy data (Organized Data)

Last chapter, we just said data analysis requires data with features of same size so that we can perform analysis like regression (and that df was the solution). However, there are few more details to care. Let's look into more details about how we want data to be organized.



1. Each row of a data matrix has a one-to-one relationship with an observation.
2. Every column is a single variable
3. Every cell is a single measured value

Why are these essential in data science? Let's look at example df below:

```
# The Ultimate Nightmare Dataframe
nightmare_df <- data.frame(
  Record_ID = c("Fam_A", "Pat_B", "Pat_C"),
  Jan_Triage_Data = c(
    "Maria: 140/90 mmHg, Leo: 98 F",
    "Carlos: 150/95 mmHg (Urgent)",
    "Sofia: 110/70 mmHg"
  ),
  Feb_Triage_Data = c(
    "Maria: 135/85 mmHg, Leo: 99.1 F",
    "Carlos: 120/80 mmHg (Normal)",
    "Did Not Attend"
  )
)

nightmare_df
```

```
  Record_ID                Jan_Triage_Data                   Feb_Triage_Data
1     Fam_A Maria: 140/90 mmHg, Leo: 98 F Maria: 135/85 mmHg, Leo: 99.1 F
2     Pat_B  Carlos: 150/95 mmHg (Urgent)    Carlos: 120/80 mmHg (Normal)
3     Pat_C            Sofia: 110/70 mmHg                   Did Not Attend
```

- Case1: Calculating patient volume Let's try to find how many patients we saw. Since we have patient names in each cell, can we find it by adding all cells (or counting how many cells we have)? There is actually no way to do this because first row contains two patients while second and third rows are single patients.

To calculate patient volume, each observation (row) must have only one patient (so that we can count number of rows and find how many patients we saw). As result, we come up with important condition, "Each row of a data matrix has a one-to-one relationship with an observation."

- Case2: Tracking bp measurement over time This time, let's try to draw a line graph using bp in each months. In this case, our variable is month and observation will be measured bp. Now the question is, can you find a column or row that contains month? Answer is no because they're used as features (columns).

Variable is a concept we are measuring. Jan and Feb are specific data point (values) of a concept called 'Month.' By using months are colnames, we cannot extract 'month' variable needed for tracking bp measurement over time no matter what column or row we pick. Some might say "If we use row, then we can extract bp data per month." But each rows are 'observation': A set of values of individual accross multiple different features (variables), not all data in single varaible (If you're confused about this, feel free to email me).

So being unable to extract month data connects to rule number 2, "Every column is a single variable"

- Case3: Extracting bp of single patient Now, let's try to extract bp measurement of single patient. Since each row contains triage data of visiting patients, can we say nightmare_df$Jan_Triage_Data[3] will give us bp measurement? No. While row 3 is an observation specific to Sofia's triage data, single cell is containing all names, bp, urgency, etc. So even we extract that cell, how do R know which portion within that cell is what we asked for? Rather than looking the data as Name (character) + bp measurement (numeric), it'll be seen as one gigantic garbage block of text (character). This example leads to condition three, "Every cell is a single measured value"

Then, how would tidy data look like? Let's look at the figure below.

Untidy data

ID	JAN	FEB
Fam_A	Maria: 140/90 mmHg, Leo: 98 F	Maria: 135/85 mmHg, Leo: 99.1 F
Pat_B	Carlos: 150/95 mmHg (Urgent)	Carlos: 120/80 mmHg (Normal)
Pat_C	Sofia: 110/70 mmHg	Did Not Attend

Tidy data

FAMILY	ROLE	NAME	MONTH	SBP	DBP	TEMP	STATUS
A	Patient	Maria	Jan	140	90	-	Normal
A	Patient	Maria	Feb	135	85	-	Normal
A	Relative	Leo	Jan	-	-	98	Stable
A	Relative	Leo	Feb	-	-	99.1	Fever
B	Patient	Carlos	Jan	150	95	-	Urgent
B	Patient	Carlos	Feb	120	80	-	Normal
C	Patient	Sofia	Jan	110	70	-	Normal
C	Patient	Sofia	Feb	-	-	-	DNA

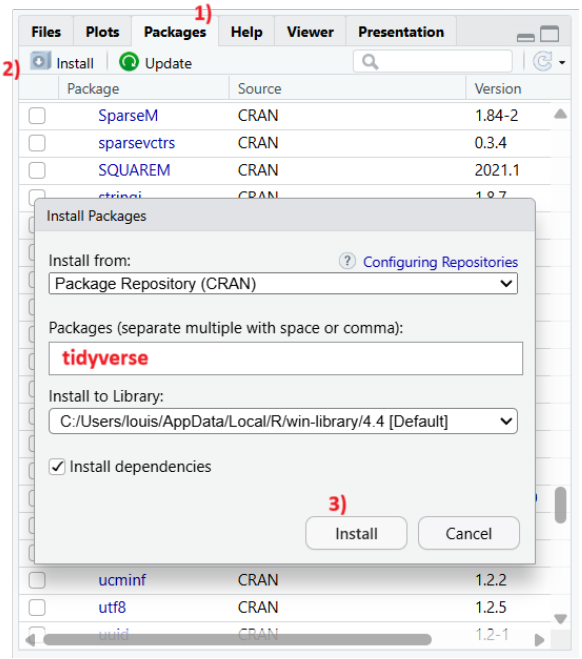
If we review 3 cases again, - Case1: We can calculate patient volume by counting columns - Case2: We can draw a line graph using Month column as x axis,and SBP/DBP columns as y axis - Case3: We can pick any cell in bp columns and will get clean bp measurement only

So untidy data might look for compact and organized, it's tidy data that actually allows us a statistical operations. One huge advantage of tidy data is that structure is formalized (one feature per column, one value per cell) that we can perform same operations throughout different column without customizing different calculation method every time. If we want to find average for each column, we can apply mean() to all numeric columns! That's something we can't do with untidy data.

7.2 tidyverse package

Again, package is like an extension that allows you to perform commands in simple manner. tidyverse is a package that contains professional and organized methods to load, analyze, and visualize tidy data.

First, let's install tidyverse. In the Console, type "install.packages("tidyverse")". If it gives you error, then we can install it directly through Packages panel.



Once you installed it, we need to load it. As we use R more, we'll install lots of packages. Loading all existing packages will be resources and time consuming, so as alternative, we state what packages we want to use for code we're running. Good news? We need to load them once per script, so we don't need to repeat it per cell. Let's load packages using library() command.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.2.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

Take a look at 'Attaching core tidyverse packages.' "tidyverse" is package for handling all tidy data, which consist specific applications (packages within tidyverse). Since we're looking at how to manipulate tidy data, we'll only discuss about dplyr. As you go through this manual, we'll talk about other packages like ggplot2 (vizualization) too!

- Side note: Since we loaded tidyverse, all packages within tidyverse are also loaded. This means that we don't need to run library(dplyr) again.

7.3 What does dplyr do?

I've been mentioning that dplyr allows you to edit df, but how does it actually do it? There are 5 methods:

1. Selecting Rows

- filter(): Filter rows that meets condition
- slice(): Extract selected rows
- They differ as filter() looks for 'values' that meets condition, while slice() looks for 'rows' that you pick.

2. Selecting Column

- select(): Like slice(), it extracts selected columns.

3. Arranging Rows

- arrange(): Ordering rows using conditions you assign (e.g., decreasing order of column 1)

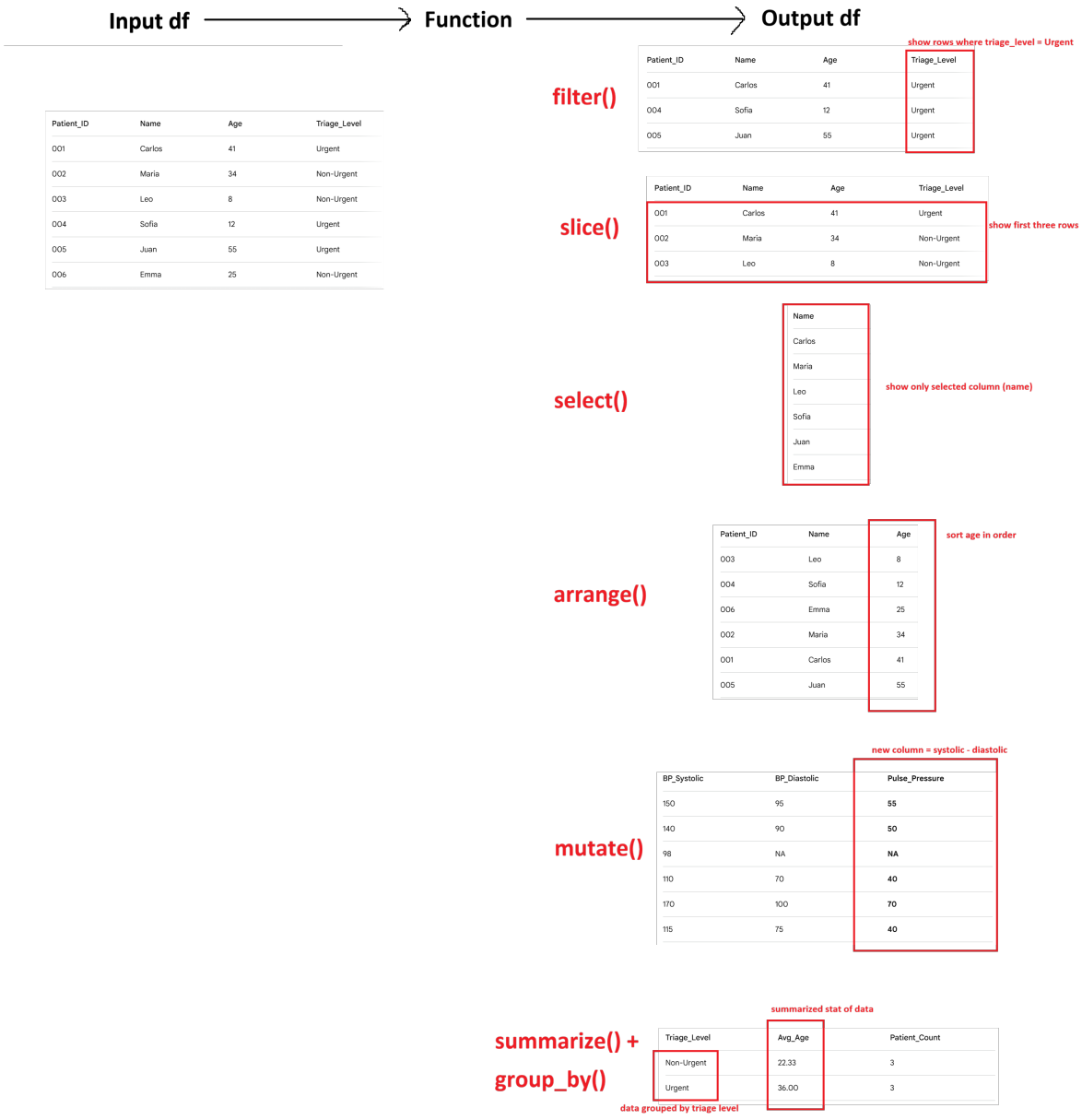
4. Adding Columns

- mutate(): Add new column using existing columns (e.g., New column = col1 + col2)

5. Summarizing Data

- summarize(): Squash selected dataset into single summarized row
- group_by(): Group dataset into buckets of your choice
- For example, summarize(age) will return average age of selected range. On the other hand, group_by(age) will set grouped filters of dataset, for example children and adult.

Let's look at the figures below in case explanations were not clear. Vizualization below shows how dplyr are like a function you learn in math courses.



Some might question, why do we need dplyr? For instance, summarize() does same thing that mean() function does. Can't we just use what R have? That is true, technically you can do everything that dplyr using basic R functions. However, there are some tricky situations. Remember how indexing df sometimes gives us df while other times it gives us list/vectors ([] vs [[]])? Basic R funcitions' input and output formats are different case by case, causing issues in analysis (because dimension might change and two different data might be incompetible for

regression). However, dlpyr input and ouput structure of data stable and consistent.

7.4 Loading Example Dataset

In R, and dlpyr, there are multiple example dataset available for us to practice coding and analysis. one of them is called 'mpg,' data collected by U.S EPA from 1998 to 2008 regarding vehicle type and their fuel efficiency. To practice dlpyr functions, let's first load mpg data.

```
mpg
```

```
# A tibble: 234 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
 2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
 3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
 6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
 7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
# i 224 more rows
```

Each column consist following data: -manufacturer : maker of the car -model : name of the car -displ : engine displacement -year : year of the model -cyl : number of engine cylinder -trans : type of car transmission -drv : driving system. f = forward, r = rear, 4 = four wheel drive -cty : fuel efficiency in city -hwy : fuel efficiency in highway -fl : type of fuel -class : type of car (compact, suv, etc)

7.5 Selecting Rows

7.5.1 filter()

First, let's try out extracting rows that meets your column condition.

You use filter() by

```
filter(dataframe, condition)
```

```r
filter(mpg, manufacturer == "dodge")
```

```
# A tibble: 37 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 dodge        caravan 2~   2.4  1999     4 auto~ f        18    24 r     mini~
 2 dodge        caravan 2~   3    1999     6 auto~ f        17    24 r     mini~
 3 dodge        caravan 2~   3.3  1999     6 auto~ f        16    22 r     mini~
 4 dodge        caravan 2~   3.3  1999     6 auto~ f        16    22 r     mini~
 5 dodge        caravan 2~   3.3  2008     6 auto~ f        17    24 r     mini~
 6 dodge        caravan 2~   3.3  2008     6 auto~ f        17    24 r     mini~
 7 dodge        caravan 2~   3.3  2008     6 auto~ f        11    17 e     mini~
 8 dodge        caravan 2~   3.8  1999     6 auto~ f        15    22 r     mini~
 9 dodge        caravan 2~   3.8  1999     6 auto~ f        15    21 r     mini~
10 dodge        caravan 2~   3.8  2008     6 auto~ f        16    23 r     mini~
# i 27 more rows
```

```r
filter(mpg, year > 2001)
```

```
# A tibble: 117 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 2 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 3 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 4 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
 5 audi         a4 quattro   2    2008     4 auto~ 4        19    27 p     comp~
 6 audi         a4 quattro   3.1  2008     6 auto~ 4        17    25 p     comp~
 7 audi         a4 quattro   3.1  2008     6 manu~ 4        15    25 p     comp~
 8 audi         a6 quattro   3.1  2008     6 auto~ 4        17    25 p     mids~
 9 audi         a6 quattro   4.2  2008     8 auto~ 4        16    23 p     mids~
10 chevrolet    c1500 sub~   5.3  2008     8 auto~ r        14    20 r     suv
# i 107 more rows
```

What if we have multiple conditions? We just add more conditions, like

```
filter(datafram, condition 1, condition2, etc)
```

```
filter(mpg, manufacturer == "dodge", year > 2001)
```

```
# A tibble: 21 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 dodge        caravan 2~   3.3  2008     6 auto~ f        17    24 r     mini~
 2 dodge        caravan 2~   3.3  2008     6 auto~ f        17    24 r     mini~
 3 dodge        caravan 2~   3.3  2008     6 auto~ f        11    17 e     mini~
 4 dodge        caravan 2~   3.8  2008     6 auto~ f        16    23 r     mini~
 5 dodge        caravan 2~   4    2008     6 auto~ f        16    23 r     mini~
 6 dodge        dakota pi~   3.7  2008     6 manu~ 4        15    19 r     pick~
 7 dodge        dakota pi~   3.7  2008     6 auto~ 4        14    18 r     pick~
 8 dodge        dakota pi~   4.7  2008     8 auto~ 4        14    19 r     pick~
 9 dodge        dakota pi~   4.7  2008     8 auto~ 4        14    19 r     pick~
10 dodge        dakota pi~   4.7  2008     8 auto~ 4         9    12 e     pick~
# i 11 more rows
```

```
filter(mpg, year > 2001 | cty >= 30) #What does this mean? Review operator if you don't reme
```

```
# A tibble: 119 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 2 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 3 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 4 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
 5 audi         a4 quattro   2    2008     4 auto~ 4        19    27 p     comp~
 6 audi         a4 quattro   3.1  2008     6 auto~ 4        17    25 p     comp~
 7 audi         a4 quattro   3.1  2008     6 manu~ 4        15    25 p     comp~
 8 audi         a6 quattro   3.1  2008     6 auto~ 4        17    25 p     mids~
 9 audi         a6 quattro   4.2  2008     8 auto~ 4        16    23 p     mids~
10 chevrolet    c1500 sub~   5.3  2008     8 auto~ r        14    20 r     suv
# i 109 more rows
```

One thing to remember is that operator's priority does matter here. Again, anything in () calculates first, and & (and) executes before | (or). So, (Condition1 | Condition2) & Condition3 will filter in order of "data that meets condition3 among data that satisfies condition1 or condition2," while Condition1 | Condition2 & Condition3 will filter in order of "data that satisfies (condition2 and condition3) or condition 1" They are different.

```
filter(mpg, (model=="audi" | cty >= 20) & year == 2001)
```

```
# A tibble: 0 x 11
# i 11 variables: manufacturer <chr>, model <chr>, displ <dbl>, year <int>,
#   cyl <int>, trans <chr>, drv <chr>, cty <int>, hwy <int>, fl <chr>,
#   class <chr>
```

```
filter(mpg, model=="audi" | cty >= 20 & year == 2001)
```

```
# A tibble: 0 x 11
# i 11 variables: manufacturer <chr>, model <chr>, displ <dbl>, year <int>,
#   cyl <int>, trans <chr>, drv <chr>, cty <int>, hwy <int>, fl <chr>,
#   class <chr>
```

Check the result. Do you see how in the second sentence, all Audi cars are returned because our logic is "made by Audi" or "cty and year condition are met," while first sentence didn't return all Audi because the logic is (audi or cty >= 20) "AND" model year is 2001?

7.5.2 slice()

slice() was filtering rows of location you want. We use it by

```
slice(dataframe, location1, location2, etc)
```

Again, one thing to keep in mind that positive number like 1 or 2 select 1st and 2nd row, while negative number like -1 select all but 1st row.

To easily understand slice(), let's first extract all Audi cars.

```
audicars <- filter(mpg, manufacturer == "audi")
head(audicars)
```

```
# A tibble: 6 x 11
  manufacturer model displ  year   cyl trans      drv     cty   hwy fl    class
  <chr>        <chr> <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
1 audi         a4      1.8  1999     4 auto(l5)   f        18    29 p     compa~
2 audi         a4      1.8  1999     4 manual(m5) f        21    29 p     compa~
3 audi         a4      2    2008     4 manual(m6) f        20    31 p     compa~
4 audi         a4      2    2008     4 auto(av)   f        21    30 p     compa~
5 audi         a4      2.8  1999     6 auto(l5)   f        16    26 p     compa~
6 audi         a4      2.8  1999     6 manual(m5) f        18    26 p     compa~
```

Now, how would you extract 1~3rd row and 5~7th rows? What if you want everything but 1~3rd rows and 5~7th rows?

```
slice(audicars, 1:3, 5:7)
```

```
# A tibble: 6 x 11
  manufacturer model displ  year   cyl trans      drv     cty   hwy fl    class
  <chr>        <chr> <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
1 audi         a4      1.8  1999     4 auto(l5)   f        18    29 p     compa~
2 audi         a4      1.8  1999     4 manual(m5) f        21    29 p     compa~
3 audi         a4      2    2008     4 manual(m6) f        20    31 p     compa~
4 audi         a4      2.8  1999     6 auto(l5)   f        16    26 p     compa~
5 audi         a4      2.8  1999     6 manual(m5) f        18    26 p     compa~
6 audi         a4      3.1  2008     6 auto(av)   f        18    27 p     compa~
```

```
slice(audicars, -(1:3), -(5:7))
```

```
# A tibble: 12 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 2 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 3 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
 4 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
 5 audi         a4 quattro   2    2008     4 auto~ 4        19    27 p     comp~
 6 audi         a4 quattro   2.8  1999     6 auto~ 4        15    25 p     comp~
 7 audi         a4 quattro   2.8  1999     6 manu~ 4        17    25 p     comp~
 8 audi         a4 quattro   3.1  2008     6 auto~ 4        17    25 p     comp~
 9 audi         a4 quattro   3.1  2008     6 manu~ 4        15    25 p     comp~
10 audi         a6 quattro   2.8  1999     6 auto~ 4        15    24 p     mids~
11 audi         a6 quattro   3.1  2008     6 auto~ 4        17    25 p     mids~
12 audi         a6 quattro   4.2  2008     8 auto~ 4        16    23 p     mids~
```

7.5.2.1 Useful variations

1. slice_random()

   There are useful feature in slice(). When we talked about evaluating ML models, we needed some fresh/random testing set. But how can we select rows randomly? we have it!

```
slice_random(dataframe, n = 5)      #This command extracts 5 rows randomly from dataframe!
slice_random(dataframe, prop = 0.6) #This command extracts 60% of rows randomly form datafra
```

2. slice_head() (and tail())

   What if you want to extract first or last 5 rows? You can use slice(df, 1:5), but what if dataframe's size is unknown or so big? Instead of using slice(df, 99999999994:99999999999), we have head() and tail() like how we previewed data!

```
slice_head(dataframe, n= 5)
slice_tail(dataframe, prop = 0.4)
```

3. slice_max() (and min())

   How about if you want to see a row with max/min value of specific value?

```
slice_min(dataframe, column, n = 2, with_ties = F) #with_ties dictates whether to print all t
```

```
#Do you see how we have top 3 rows with highest cty among Audi cars?
slice_max(audicars, cty, n = 3, with_ties = F)
```

```
# A tibble: 3 x 11
  manufacturer model displ  year   cyl trans      drv     cty   hwy fl    class
  <chr>        <chr> <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
1 audi         a4      1.8  1999     4 manual(m5) f        21    29 p     compa~
2 audi         a4      2    2008     4 auto(av)   f        21    30 p     compa~
3 audi         a4      2    2008     4 manual(m6) f        20    31 p     compa~
```

```
#If we don't have with_ties = F, it's not really top 3 anymore. Is it?
slice_max(audicars, cty, n = 3, with_ties = T)
```

```
# A tibble: 4 x 11
  manufacturer model      displ  year   cyl trans  drv     cty   hwy fl    class
  <chr>        <chr>      <dbl> <int> <int> <chr>  <chr> <int> <int> <chr> <chr>
1 audi         a4           1.8  1999     4 manua~ f        21    29 p     comp~
2 audi         a4           2    2008     4 auto(~ f        21    30 p     comp~
3 audi         a4           2    2008     4 manua~ f        20    31 p     comp~
4 audi         a4 quattro   2    2008     4 manua~ 4        20    28 p     comp~
```

7.6 Selecting Columns

7.6.1 select()

Format is same as filter(). However, one thing to be careful is to state column names without " ". Normally column names are character that we have to use" ", but in dplyr, column and row names themselves are variable.

```
select(audicars, model, year, cty) #Here, we extract selected columns only
```

```
# A tibble: 18 x 3
   model       year   cty
   <chr>      <int> <int>
 1 a4          1999    18
 2 a4          1999    21
 3 a4          2008    20
 4 a4          2008    21
 5 a4          1999    16
 6 a4          1999    18
 7 a4          2008    18
 8 a4 quattro  1999    18
 9 a4 quattro  1999    16
10 a4 quattro  2008    20
11 a4 quattro  2008    19
12 a4 quattro  1999    15
13 a4 quattro  1999    17
14 a4 quattro  2008    17
15 a4 quattro  2008    15
16 a6 quattro  1999    15
17 a6 quattro  2008    17
18 a6 quattro  2008    16
```

One interesting thing in select is that now col names are variables! Meaning, we can use : operator. For example, column names were ordered in manufacturer, model, displ, etc., order. Also, this means that we can use both numeric indexing and varaible name to state which column we want to select.

```
audicars
```

```
# A tibble: 18 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
```

	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>	<int>	<chr>	<chr>
1	audi	a4	1.8	1999	4	auto~	f	18	29	p	comp~
2	audi	a4	1.8	1999	4	manu~	f	21	29	p	comp~
3	audi	a4	2	2008	4	manu~	f	20	31	p	comp~
4	audi	a4	2	2008	4	auto~	f	21	30	p	comp~
5	audi	a4	2.8	1999	6	auto~	f	16	26	p	comp~
6	audi	a4	2.8	1999	6	manu~	f	18	26	p	comp~
7	audi	a4	3.1	2008	6	auto~	f	18	27	p	comp~
8	audi	a4 quattro	1.8	1999	4	manu~	4	18	26	p	comp~
9	audi	a4 quattro	1.8	1999	4	auto~	4	16	25	p	comp~
10	audi	a4 quattro	2	2008	4	manu~	4	20	28	p	comp~
11	audi	a4 quattro	2	2008	4	auto~	4	19	27	p	comp~
12	audi	a4 quattro	2.8	1999	6	auto~	4	15	25	p	comp~
13	audi	a4 quattro	2.8	1999	6	manu~	4	17	25	p	comp~
14	audi	a4 quattro	3.1	2008	6	auto~	4	17	25	p	comp~
15	audi	a4 quattro	3.1	2008	6	manu~	4	15	25	p	comp~
16	audi	a6 quattro	2.8	1999	6	auto~	4	15	24	p	mids~
17	audi	a6 quattro	3.1	2008	6	auto~	4	17	25	p	mids~
18	audi	a6 quattro	4.2	2008	8	auto~	4	16	23	p	mids~

```
select(audicars, year:drv)
```

```
# A tibble: 18 x 4
    year   cyl trans      drv
   <int> <int> <chr>      <chr>
 1  1999     4 auto(l5)   f
 2  1999     4 manual(m5) f
 3  2008     4 manual(m6) f
 4  2008     4 auto(av)   f
 5  1999     6 auto(l5)   f
 6  1999     6 manual(m5) f
 7  2008     6 auto(av)   f
 8  1999     4 manual(m5) 4
 9  1999     4 auto(l5)   4
10  2008     4 manual(m6) 4
11  2008     4 auto(s6)   4
12  1999     6 auto(l5)   4
13  1999     6 manual(m5) 4
14  2008     6 auto(s6)   4
15  2008     6 manual(m6) 4
16  1999     6 auto(l5)   4
17  2008     6 auto(s6)   4
```

```
18  2008     8 auto(s6)   4
```

```
select(audicars, 4:7)
```

```
# A tibble: 18 x 4
    year   cyl trans      drv
   <int> <int> <chr>      <chr>
 1  1999     4 auto(l5)   f
 2  1999     4 manual(m5) f
 3  2008     4 manual(m6) f
 4  2008     4 auto(av)   f
 5  1999     6 auto(l5)   f
 6  1999     6 manual(m5) f
 7  2008     6 auto(av)   f
 8  1999     4 manual(m5) 4
 9  1999     4 auto(l5)   4
10  2008     4 manual(m6) 4
11  2008     4 auto(s6)   4
12  1999     6 auto(l5)   4
13  1999     6 manual(m5) 4
14  2008     6 auto(s6)   4
15  2008     6 manual(m6) 4
16  1999     6 auto(l5)   4
17  2008     6 auto(s6)   4
18  2008     8 auto(s6)   4
```

However, there is important point you should remember: New output will follow the order of variable you put. Let's look at the example below

```
select(audicars, year, cty)
```

```
# A tibble: 18 x 2
    year   cty
   <int> <int>
 1  1999    18
 2  1999    21
 3  2008    20
 4  2008    21
 5  1999    16
 6  1999    18
 7  2008    18
```

```
 8  1999    18
 9  1999    16
10  2008    20
11  2008    19
12  1999    15
13  1999    17
14  2008    17
15  2008    15
16  1999    15
17  2008    17
18  2008    16
```

```
select(audicars, cty, year)
```

```
# A tibble: 18 x 2
     cty  year
   <int> <int>
 1    18  1999
 2    21  1999
 3    20  2008
 4    21  2008
 5    16  1999
 6    18  1999
 7    18  2008
 8    18  1999
 9    16  1999
10    20  2008
11    19  2008
12    15  1999
13    17  1999
14    17  2008
15    15  2008
16    15  1999
17    17  2008
18    16  2008
```

Do you see how order of output columns changed?

7.7 Adding Columns

7.7.1 mutate()

mutate() is creating new column at the end of the dataframe using existing column. For example, let's extract fuel efficiency of cars

```
fueldata <- select(mpg, cty, hwy)
fueldata
```

```
# A tibble: 234 x 2
     cty   hwy
   <int> <int>
 1    18    29
 2    21    29
 3    20    31
 4    21    30
 5    16    26
 6    18    26
 7    18    27
 8    18    26
 9    16    25
10    20    28
# i 224 more rows
```

now, if we want to calculate average of two and save it into same data frame, we can use mutate() using following manner

```
mutate(dataframe, new variable (column name) = equation)
```

```
mutate(fueldata, avg = (cty + hwy)/2)
```

```
# A tibble: 234 x 3
     cty   hwy   avg
   <int> <int> <dbl>
 1    18    29  23.5
 2    21    29  25
 3    20    31  25.5
 4    21    30  25.5
 5    16    26  21
```

```
 6    18    26  22
 7    18    27  22.5
 8    18    26  22
 9    16    25  20.5
10    20    28  24
# i 224 more rows
```

If you want to only keep new column and disregard all existing columns, you can use transmute() instead.

```
transmute(fueldata, mean = (cty + hwy)/2)
```

```
# A tibble: 234 x 1
    mean
   <dbl>
 1  23.5
 2  25
 3  25.5
 4  25.5
 5  21
 6  22
 7  22.5
 8  22
 9  20.5
10  24
# i 224 more rows
```

7.8 Summarizing Data

7.8.1 summarize()

summarize() shows statistical overview of all values in column into single row. Some common stats were: -n() : number of entires -sum() : sum of all entires -mean() : average of all entires -median() : center of all entires -sd() : standard deviation -var() : variance -max()/min(): maximum and minimum value

In summarize(), format is same as others.

```
summarize(dataframe, name1 = equation1, name2 = equation2, ...)
```

```
summarize(mpg, avg = mean(cty), med = median(cty), std = sd(cty))
```

```
# A tibble: 1 x 3
    avg   med   std
  <dbl> <dbl> <dbl>
1  16.9    17  4.26
```

Now you see the statistics of average city fuel efficiency of all cars registered in the mpg data frame!

7.8.2 group_by()

While summarize() shows overview of entire dataset, there is one limitation: it summarize everything. What if we want summary by marker? Like, what if we want to see average cty of Audi and Sonata serpeately instead of all cars? This is when group_by() becomes useful.

group_by() puts data into each brackets. What does that mean?

```
perMaker <- group_by(mpg, manufacturer)
perMaker
```

```
# A tibble: 234 x 11
# Groups:   manufacturer [15]
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
 2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
 3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
 6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
 7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
# i 224 more rows
```

If you compare perModel, grouped data, with mpg (raw dataset), there is no difference. Why? group_by assign 'groups' to each data, but itself doesn't perform any operations. So, how do we use it? It becomes handly when combined with summarization.

```r
summarize(perMaker, avg = mean(cty), med = median(cty), std = sd(cty))
```

```
# A tibble: 15 x 4
   manufacturer   avg   med   std
   <chr>        <dbl> <dbl> <dbl>
 1 audi          17.6  17.5 1.97
 2 chevrolet     15    15   2.92
 3 dodge         13.1  13   2.49
 4 ford          14    14   1.91
 5 honda         24.4  24   1.94
 6 hyundai       18.6  18.5 1.50
 7 jeep          13.5  14   2.51
 8 land rover    11.5  11.5 0.577
 9 lincoln       11.3  11   0.577
10 mercury       13.2  13   0.5
11 nissan        18.1  19   3.43
12 pontiac       17    17   1
13 subaru        19.3  19   0.914
14 toyota        18.5  18   4.05
15 volkswagen    20.9  21   4.56
```

Now, compare the difference with summarize(mpg, same arguments). Do you see the difference? Remember, group_by() is grouping data, nothing special. This means you can combine group_by() with other dplyr functions like mutate(), select(), and more!

```r
perMaker1 <- mutate(perMaker, rank = min_rank(desc(cty)))
select(perMaker1, 1:2, cty, rank) #Ranking cty from highest to lowest.
```

```
# A tibble: 234 x 4
# Groups:   manufacturer [15]
   manufacturer model        cty  rank
   <chr>        <chr>      <int> <int>
 1 audi         a4            18     6
 2 audi         a4            21     1
 3 audi         a4            20     3
 4 audi         a4            21     1
 5 audi         a4            16    13
 6 audi         a4            18     6
 7 audi         a4            18     6
 8 audi         a4 quattro    18     6
 9 audi         a4 quattro    16    13
```

```
10 audi          a4 quattro    20     3
# i 224 more rows
```

In any case you want to ungroup your grouping, you can use ungroup().

```
summarize(ungroup(perMaker), avg = mean(cty))
```

```
# A tibble: 1 x 1
    avg
  <dbl>
1  16.9
```

7.9 Pipline Operator

So far, we looked at what each functions do. However, what if we need to use multiple?

Let's say we want to do following: Find average cty per model, and display models with average cty > 20.

To do this, we need to go through multiple steps.

```
# 1. Grouping cars by model
perModel <- group_by(mpg, model)

# 2. Finding average cty per model
avgcty <- summarize(perModel, count = n(), avg = mean(cty))

# 3. Filtering model with average cty > 20
ctysatisfied <- filter(avgcty, avg > 20)

ctysatisfied
```

```
# A tibble: 5 x 3
  model      count   avg
  <chr>      <int> <dbl>
1 altima         6  20.7
2 civic          9  24.4
3 corolla        5  25.6
4 jetta          9  21.2
5 new beetle     6  24
```

Or, we could do this in one line

```
filter(summarize(group_by(mpg, model), count = n(), avg = mean(cty)), avg > 20)
```

```
# A tibble: 5 x 3
  model      count   avg
  <chr>      <int> <dbl>
1 altima         6  20.7
2 civic          9  24.4
3 corolla        5  25.6
4 jetta          9  21.2
5 new beetle     6  24
```

In the first method, we had to save multiple variables and data frames to get to the conclusion. It seems fine right now, but if we work with large data sets with hundreds of variables saved, remembering which is which every time will be headache.

In the second method, we could do it in one line, but we will get easily lost or make mistake repeating () and putting functions in the function which is in another function, and so on.

Pipeline operator, %>%, exist to resolve both cases: It allows us to perform multiple functions without losing readability or creating countless dummy variables. Think %>% as "sub category," or layer. For example, if we have A %>% B %>% C, it means "Perform B using data from (%>%) A", and "Perform C using data from B." If that was confusing, think like %>% means "Pass result to next calculation without making new variable."

```
mpg %>%
  group_by(model) %>%
  summarize(count = n(), avg = mean(cty)) %>%
  filter(avg > 20)
```

```
# A tibble: 5 x 3
  model      count   avg
  <chr>      <int> <dbl>
1 altima         6  20.7
2 civic          9  24.4
3 corolla        5  25.6
4 jetta          9  21.2
5 new beetle     6  24
```

Here, do you notice how group_by(model) is performed using data from mpg (mpg %>% group_by(model))? One thing to remember is how our grammer changed. Earlier, we had to state "group_by(mpg, model)" to tell "group data by model using mpg data"

Now, we only state "group_by(model)" to do the same because "mpg %>%" defines what data group_by(model) is using. In conclusion, %>% makes multi step calculation easy!

7.10 Summary

Today, we reviewed why we want tidy data for data analysis, and how we preprocess (manipulate) data using dlpyr package. Next time, we will learn how to vizualize data!

8 Vizualizing Data with ggplot2

Last chapter, we learned how to organize, clean, and manipulate tidy data. In this chapter, we will use another package in tidyverse, ggplot2 package, to learn how to vizualize data.

First, let's load ggplot2 package. Just like dplyr, we need to call for tidyverse.

```
library(tidyverse)
```

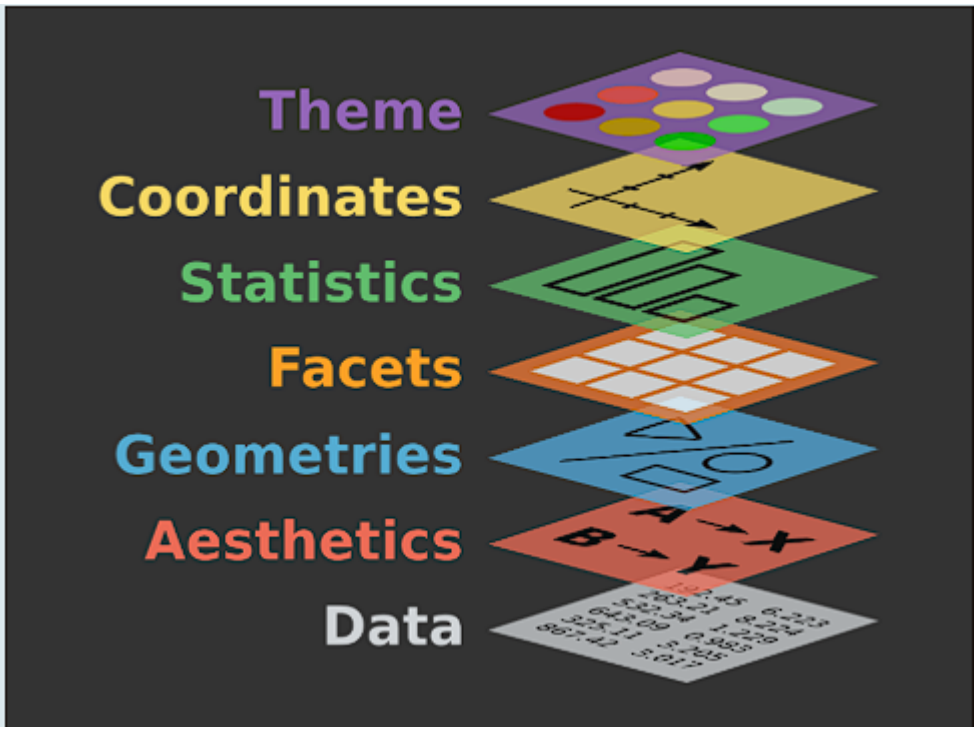
```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.2.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

8.1 Components in ggplot2()

ggplot is like a grammer, in a sense that you stack multiple layers to produce a figure.

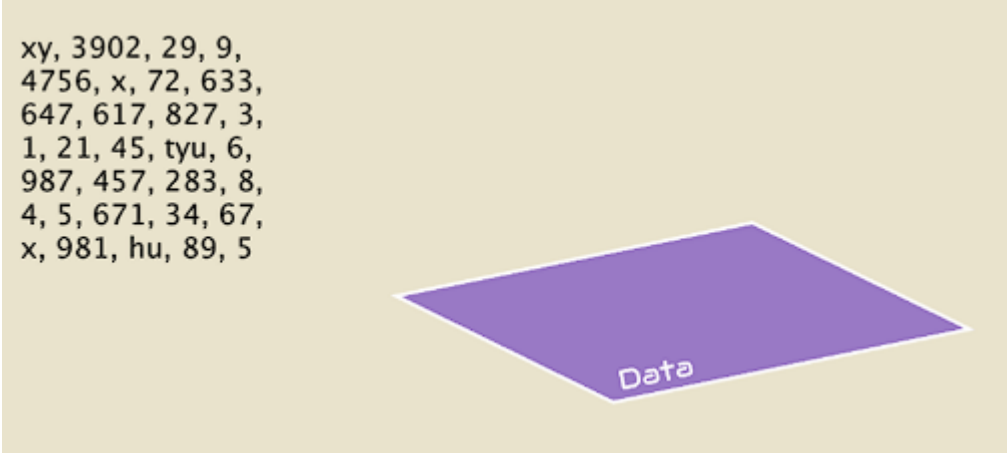
ggplot() itself tells R, "here's an empty canvas." But, you must define each components to come up with one figure. What variables are we using? What values are we plotting? What type of figure are we using?

There are multiple layers of component that defines those:

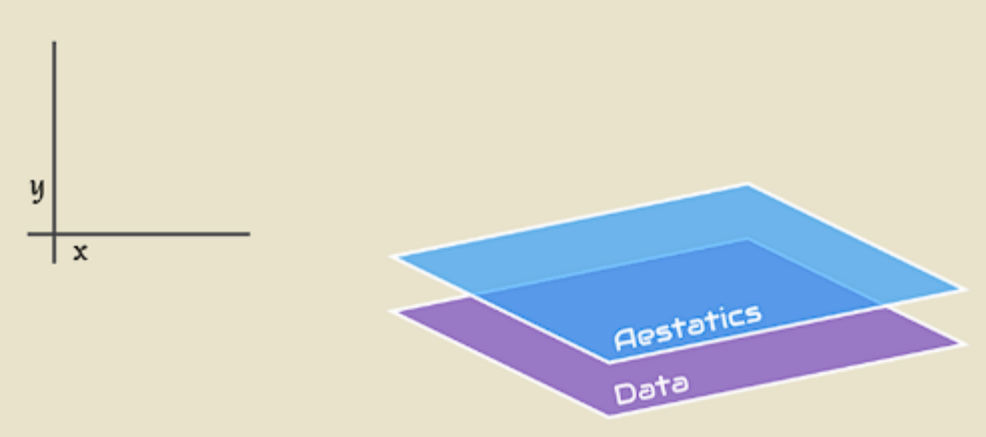


Three main (mandatory) components are:

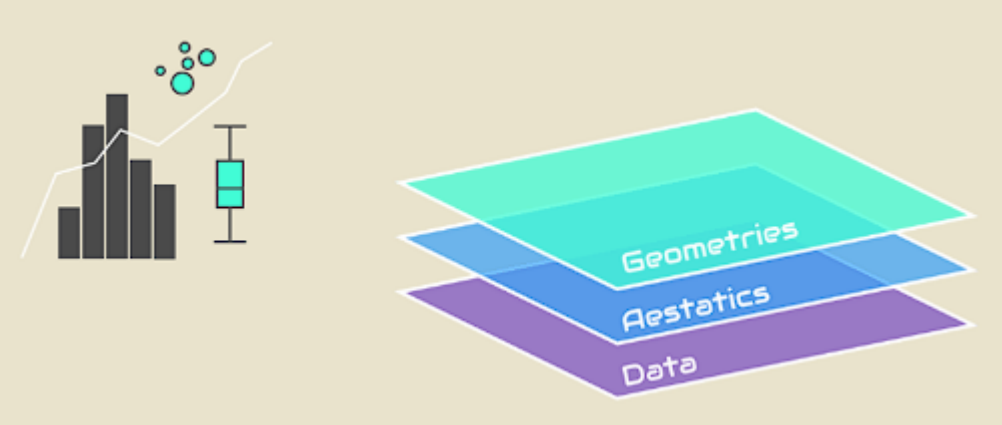
1. Data | data = : This is component where you state the data frame you will use to plot



2. Aesthetics | aes(x=,y=,color=, …) : This layer defines visual components of the plot. These includes axis, color and size of data points, and more.



3. Geometry | geom_x() where x can be multiple modes : Defines what method you will use to visualize, like line graph, bar graph, etc,.

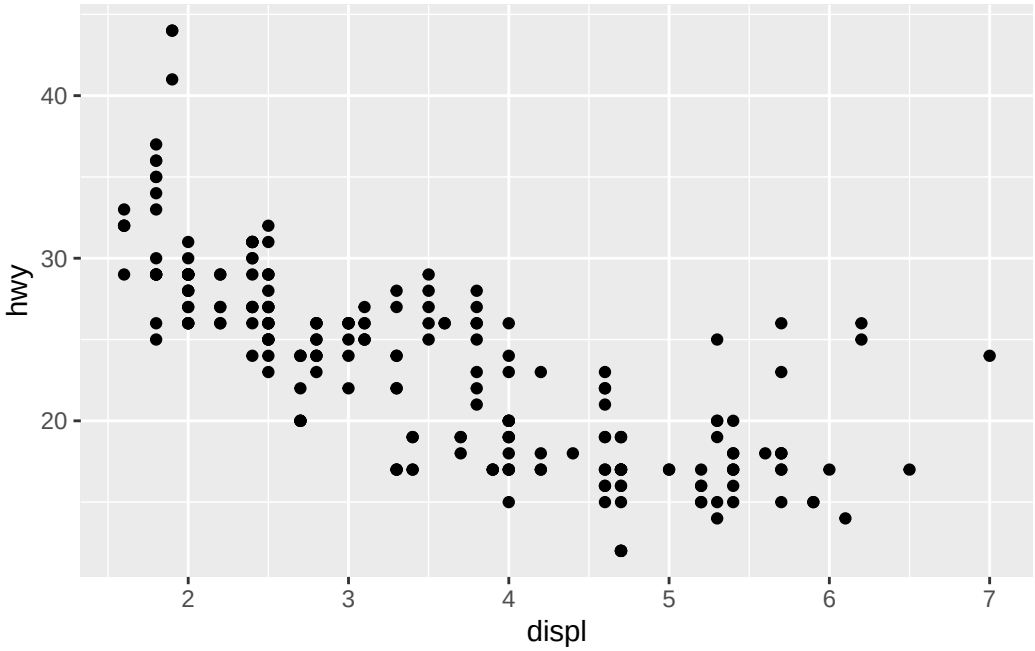


Others are accessory components, as they help readability of the graph but not mandatory.

8.1.1 Three Mandatory Components

Now, let's start drawing our first ggplot() using mpg dataset we used last time.

```
ggplot(data = mpg) +
  aes(x = displ, y = hwy) +
  geom_point()
```



First, let's understand the graphic.

Take a look at the downward trend of the dots. In this dataset, displ represents the car's engine displacement (engine size), and hwy represents its highway fuel efficiency.

Notice that they are inversely proportional. Does this make sense? Yes! As engine size increases on the X-axis, the engine requires more fuel per cycle to run. This burns more energy, resulting in significantly lower fuel efficiency (MPG) on the Y-axis.

Second, let's look at how this graph is built.

Remember how we're stacking different layers of functions to create one figure? We use "+" to connect different layers.

Then, Let's break down the three layers of our code:

Layer 1: The Canvas (ggplot(data = mpg))

This is our base sheet. We are telling R to initialize the chart and specifically use the raw mpg dataframe as our starting material.

Layer 2: The Blueprint (aes(x = displ, y = hwy))

This is our aesthetic sheet. We haven't drawn any shapes yet, but we are telling R to draw the gridlines: map engine size to the X-axis, and highway efficiency to the Y-axis.

Layer 3: The Paint (geom_point())

This is our geometry sheet. Now that the data and gridlines are set, we finally tell R to pick up the "dot" paintbrush and stamp the physical shapes onto the canvas.

Like this, ggplot() combines layers of component (ggplot(), geom(), etc) that contains details (data =, x =, etc).

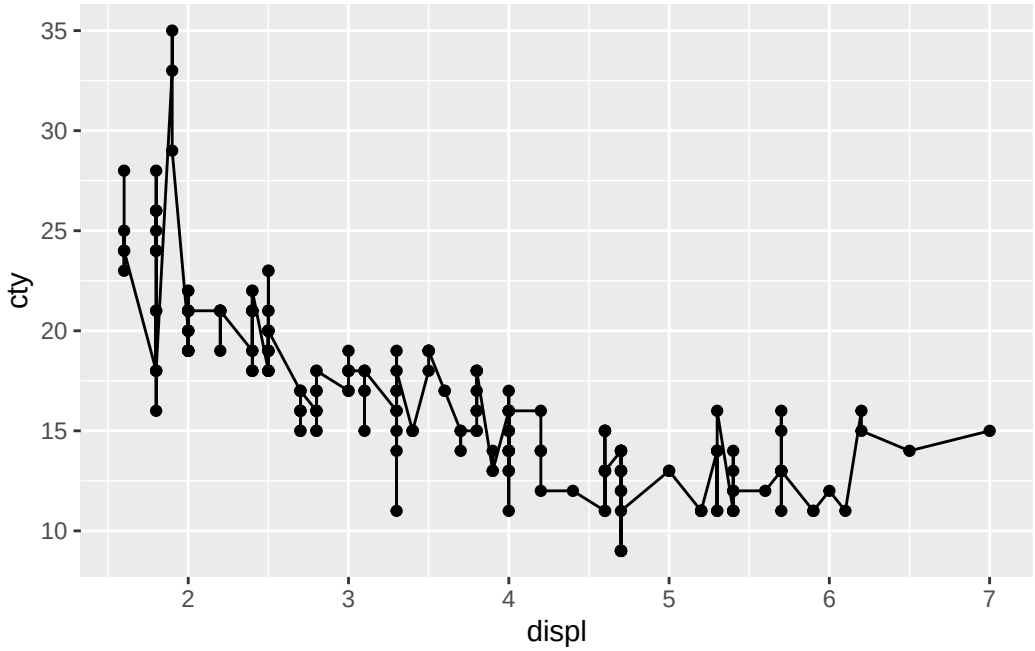
8.2 mapping Argument

When you run a stack of layers, R reads your code from top to bottom, but it doesn't actually draw anything until it reaches the very end. For our previous example, R reads the canvas ggplot(), reads the gridlines aes(), reads the paintbrush geom_point(), and then executes everything at once. Because of the way R reads the code, there is IMPORTANT point to remember.

Trap: it can hold only single layer of same-type.

Let's demonstrate the trap by adding second type of plot.

```
ggplot(data = mpg) +

# First plot
  aes(x = displ, y = hwy) +
  geom_point() +

# Second plot
  aes(x = displ, y = cty) +
  geom_line()
```



Since geom_point() uses first aes() (hwy) and geom_line() uses second aes() (cty), you would expect that code to give two distinct graphs. However, when you run the code, you recognize both dot and line uses cty as y values. Where did first aes() (hwy) go?

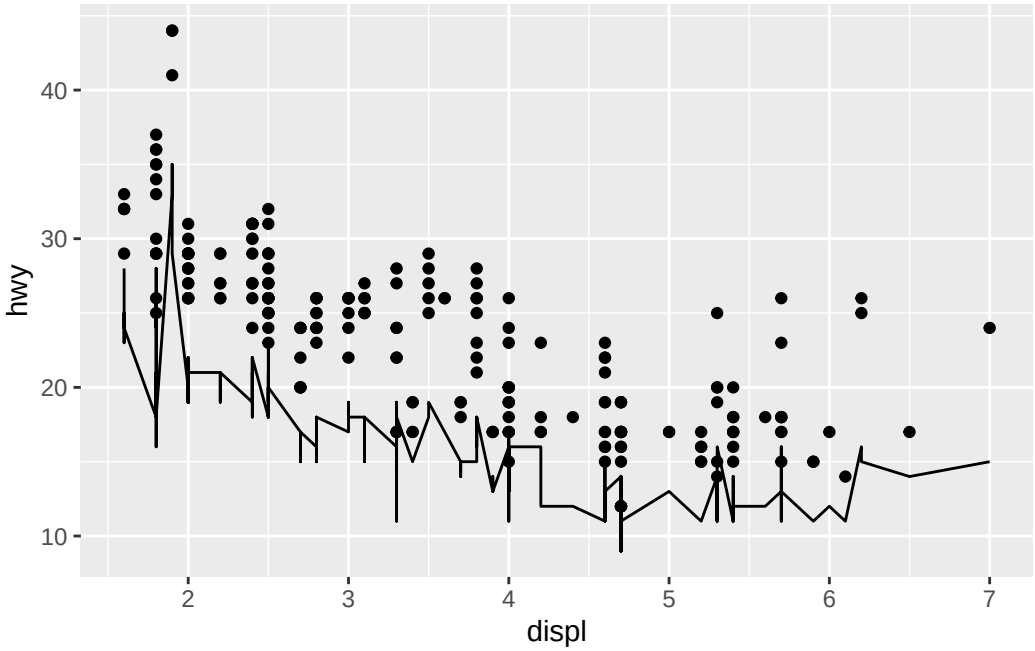
This happens because R updates master snapshot as it reads from top to bottom, and execute the everything at once it finishes reading the code. So, even R sees first aes(), second aes() overrides it (this is what it means by ggplot() only holds single layer of identical type). By the time R reads all code, we're left with second aes() (cty), geom_point(), and geom_line() that both graph uses same x&y values.

Then, how do we prevent this issue? We use argument called mapping.

Mapping, is a bridge that connects visuals to the data. So while aes() is a visualization layer, "mapping = aes()" allows visual layer to be a detail of data layers.

Connecting data and visual means we don't have to leave aes() as seperate layer. Instead, we could use it like a detail of other layers. Let's look at the example below.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  geom_line(mapping = aes(x = displ, y = cty))
```



Different to plot we saw earlier, where both geom_point() and geom_line() uses aes(x=displ, y=xty), each plot uses different y values as we intended. We can compare what ggplot() ends up with at the end of the code, we can understand why this happened.

1. aes() as seperate layer
   geom() #1 : geom_point() using aes() layer
   geom() #2 : geom_line() using aes() layer
   aes() : aes(x = displ, y = cty)
   Result : both geom() draws using aes(x=displ, y = cty)

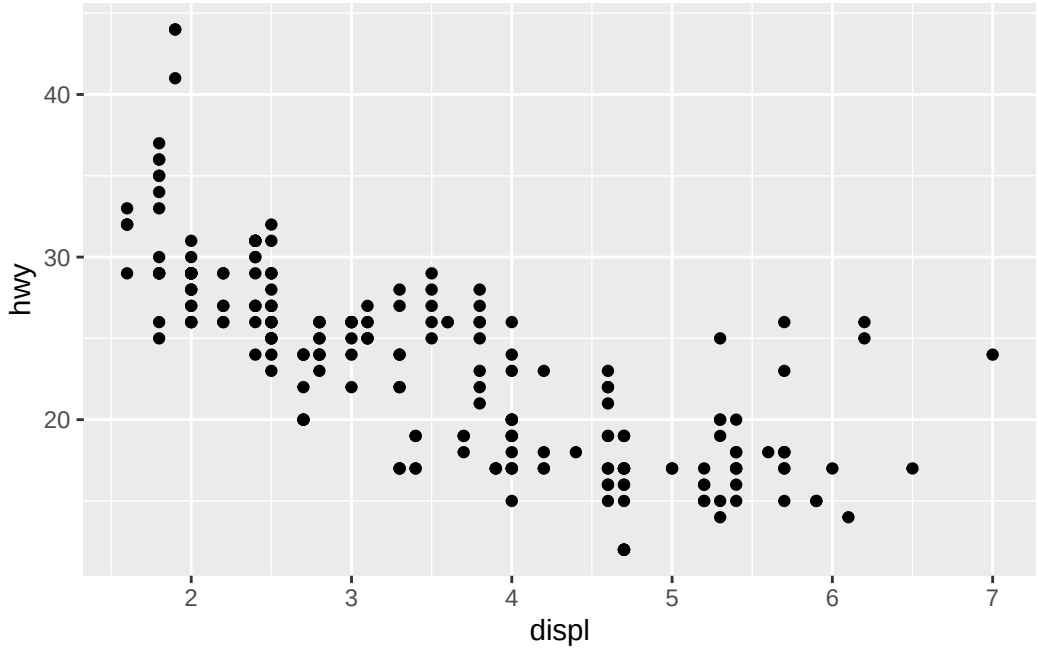
2. using mapping = aes()
   geom() #1 : geom_point() using mapping = aes(x = displ, y = hwy)
   geom() #2 : geom_line() using mapping = aes(x = displ, y = cty)
   Result : each geom() have unique aes()

By making aes() a detail rather than seperate layer(), now we can plot multiple geom() using multiple dataset.

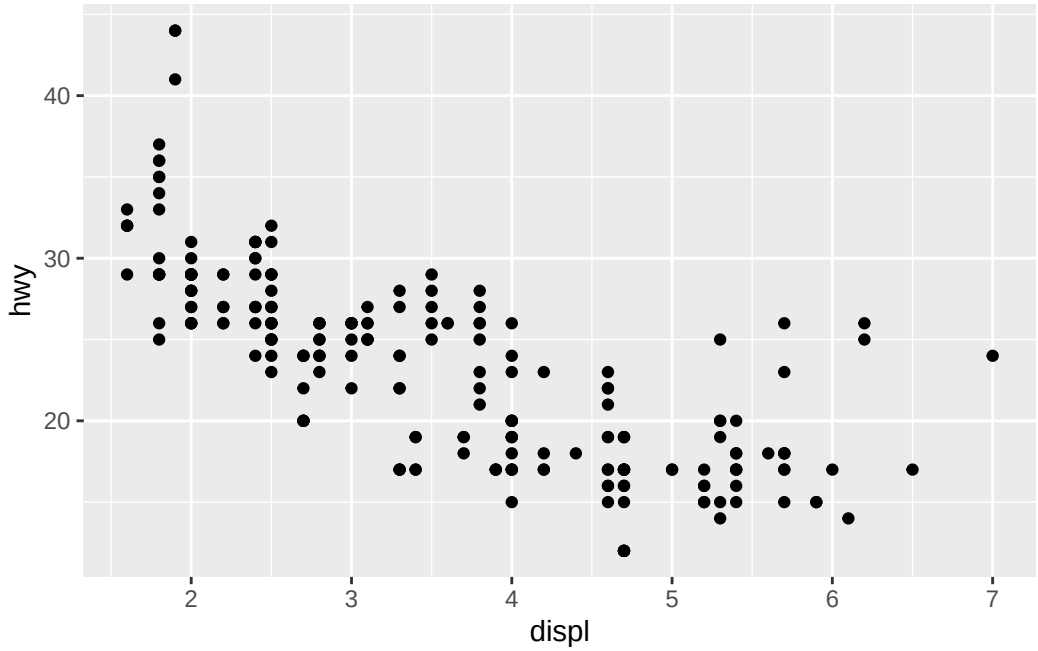
8.2.1 Global versus Local mapping

If we can make aes() as argument within geom(), can we also do the same within ggplot()? Answer is yes. Let's look at the example below.

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```




```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point()
```



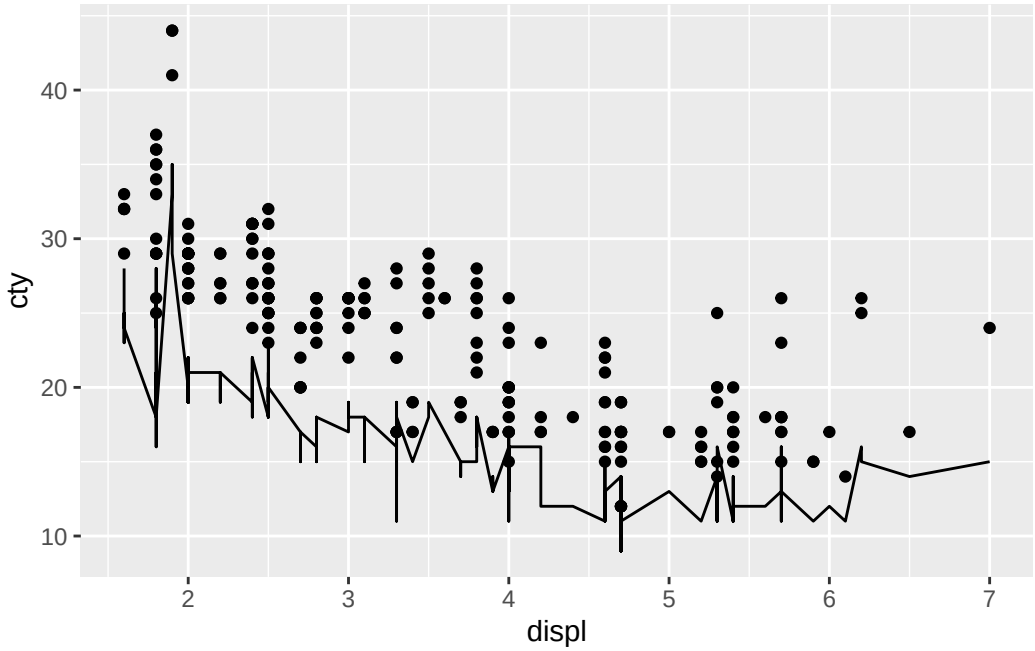
Compare two plots. Don't they look the same?

ggplot() is a global layer, while other layers are local layers. It means that any argument in ggplot() becomes a default for entire code. Now let's review the second ggplot() again. In ggplot(), aes(x=displ, y=hwy) sets global default aes() for everything else. So even though geom_point() didn't have aes() that defines x and y values, it looks back at ggplot() and use global aes() to draw graph.

On the other hand, in first ggplot(), there is no global aes() decalred. However, geom_point() has its own local aes() that it still can draw graph.

What if we have both global and local mapping?

```
ggplot(data = mpg, mapping = aes(x = displ, y = cty)) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  geom_line()
```



Here, we get different plots again. But it does make sense, let's look at line by line.

First line, we set global aes() with y values of "cty". In the second line, geom_point() has its own aes() with "y = hwy." Therefore, geom_point() use its local mapping. In the third line, geom_line() doesn't have any aes(). Now, it checks back at global mapping ad use aes() with "y = cty." Therefore, we get one with hwy as y value (geom_point with local aes) and another with cty as y value (geom_line with global aes).

One point of common confusion: global doesn't mean "universial law." It's just default. layers will prioritize local mapping if they have their own. Only when they lack settings they need, global mapping will be used.

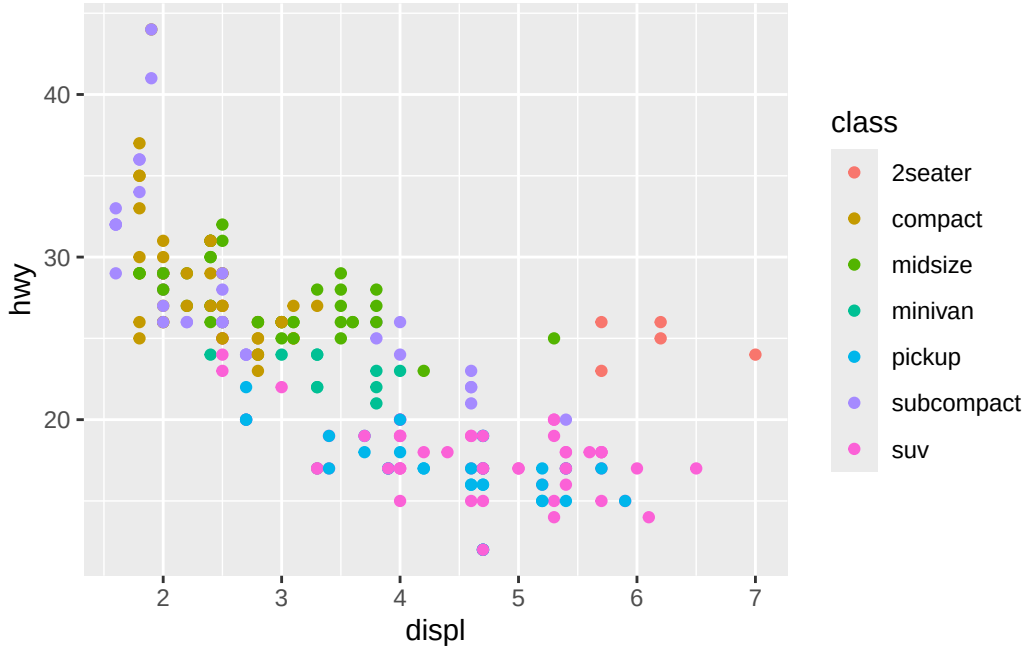
8.3 aes()

Now, let's look at details about aesthetic layer. As aes() defines visuals like shape, color, etc., there are many argument. Adding them is simple, as they're just extra argument within aes(). Below, we'll demonstrate how they look like.

8.3.1 color

If multiple different individuals are visualized on the same graph, you might want to differentiate them using color.

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class))
```



Class in mpg data frame was the type of cars. As shown in the figure, each data point with different class are plotted with different colors.

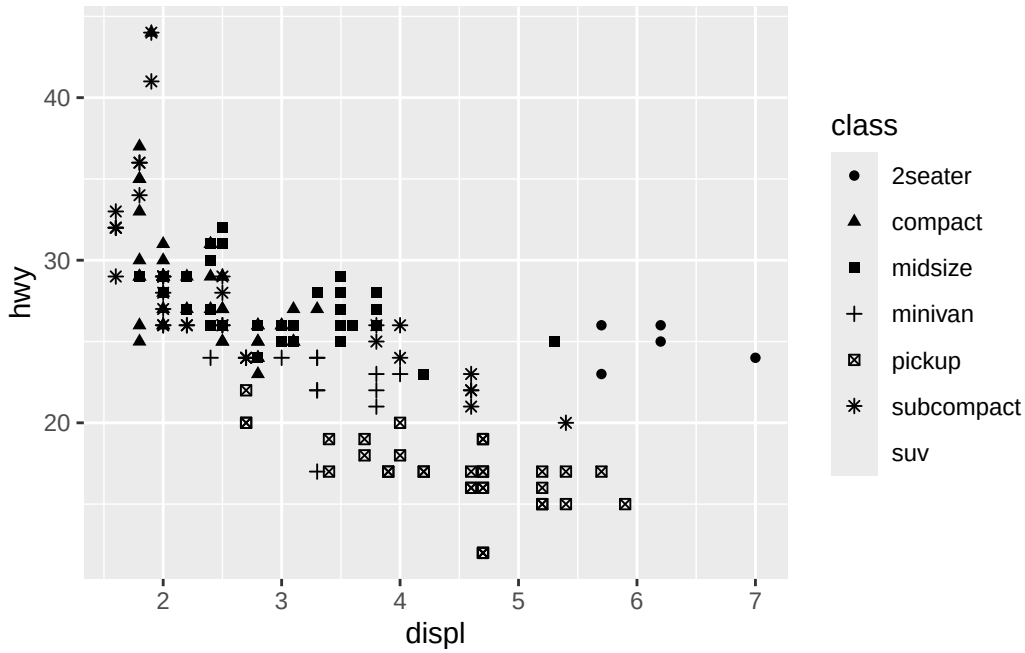
8.3.2 shape

similar to color, we can use shape argument to differentiate data points

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, shape = class))
```

```
Warning: The shape palette can deal with a maximum of 6 discrete values because more
than 6 becomes difficult to discriminate
i you have requested 7 values. Consider specifying shapes manually if you need
  that many of them.
```

```
Warning: Removed 62 rows containing missing values or values outside the scale range
(`geom_point()`).
```



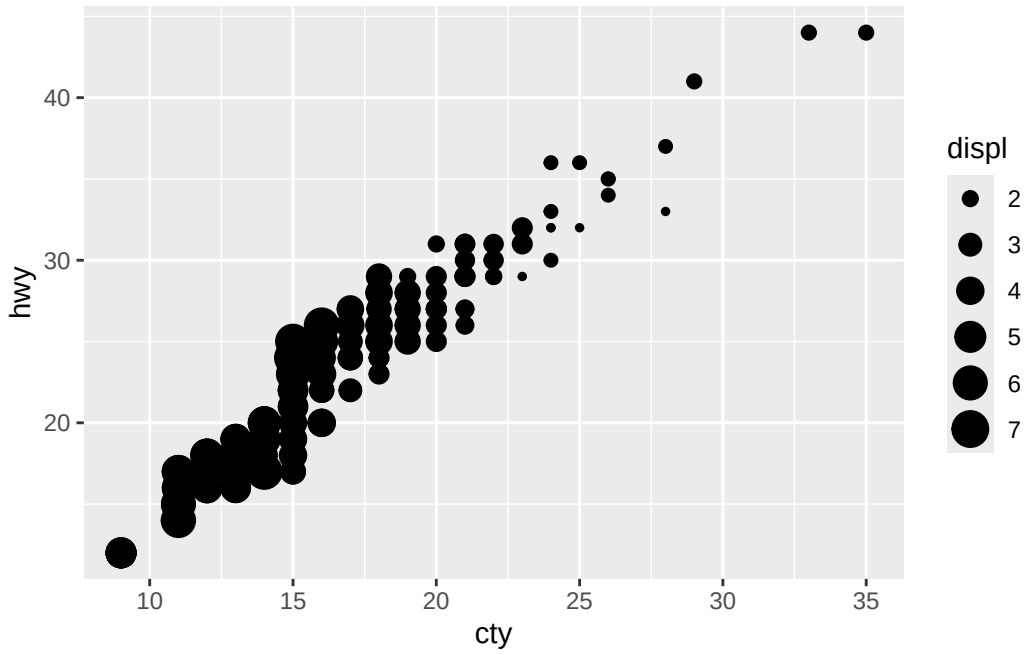
While shape differentiates the car class, do you see the warning sign? This is because there are finite number of shapes available. If we have 1000000 categories (e.g., continuous number range) but only have 60 shapes, we can't assign unique shapes to every single categories. In fact, we don't even have a shape assigned for suv. In that case, color might be better fit since there are much more available colors.

8.3.3 size and alpha

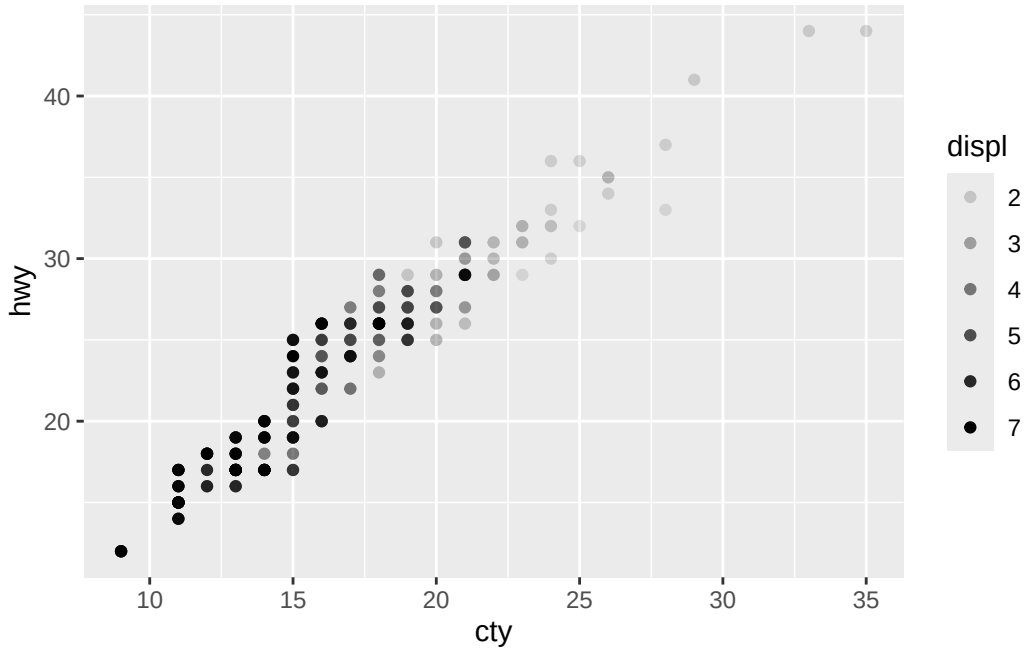
Size controls size of the data points, and alpha controls transparency of the data point. This time, let's use continuous data, displ.

```
# Too many shapes needed for continous data, so it will return error. Try running this code
ggplot() +
  geom_point(data = mpg, mapping = aes(x = cty, y = hwy, shape = displ))
```

```
# Using size
ggplot() +
  geom_point(data = mpg, mapping = aes(x = cty, y = hwy, size = displ))
```

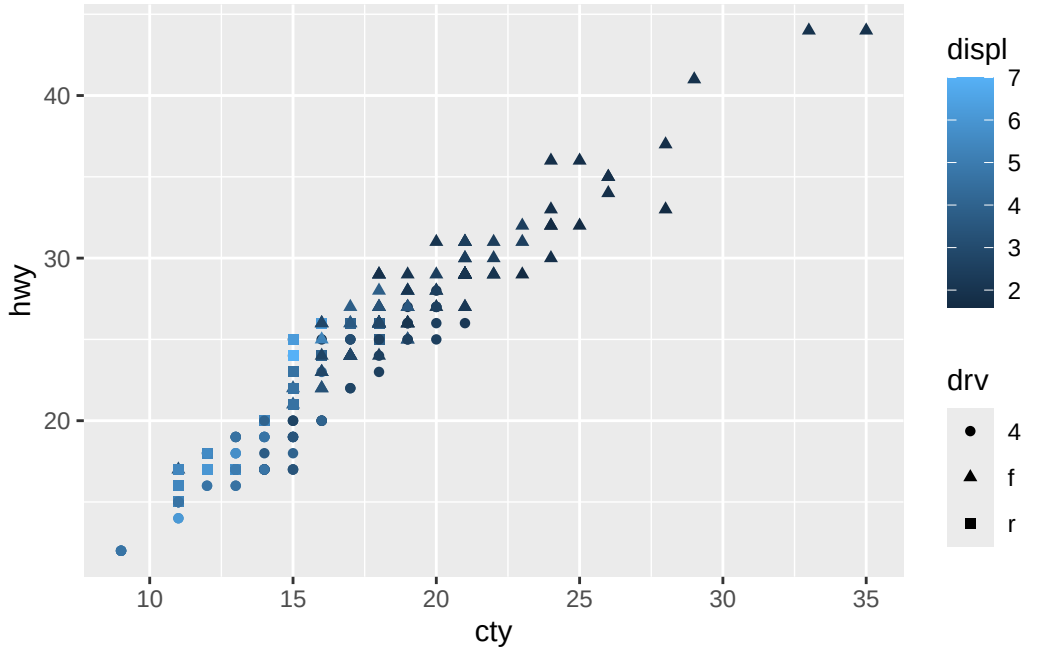



```
# Using alpha
ggplot() +
  geom_point(data = mpg, mapping = aes(x = cty, y = hwy, alpha = displ))
```



Of course, you can use multiple arguments at once.

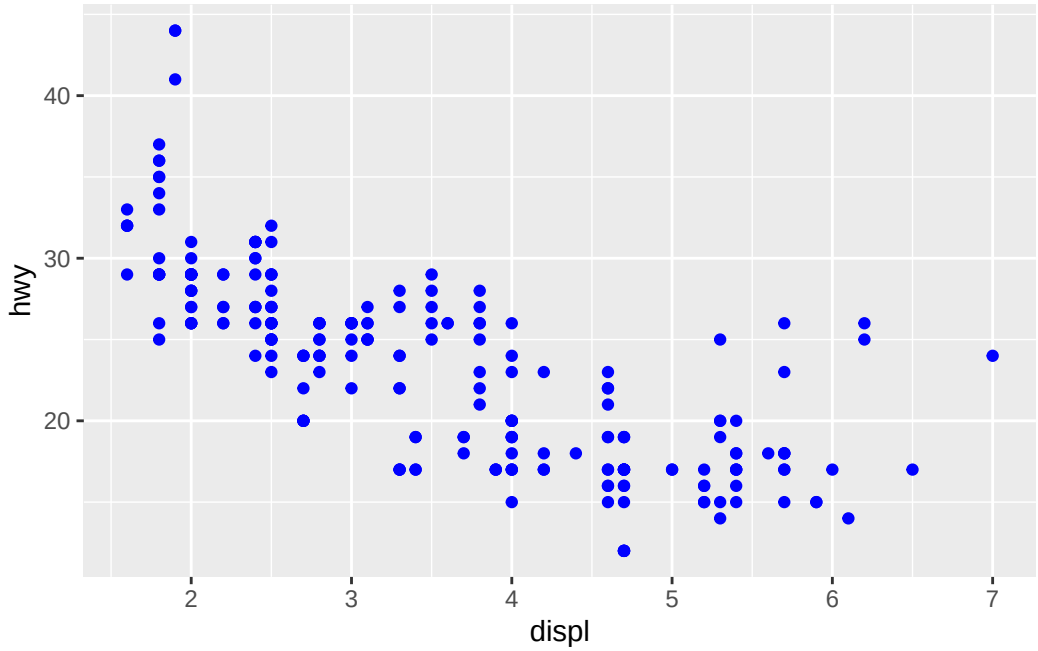
```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = cty, y = hwy, shape = drv, color = displ))
```



8.3.4 Manually Setting Arguments

So far, arguments set colors/size/etc for you automatically. For example, if we wanted color based on the type of displ, we would put color = in aes() (visual factor of each point).But sometimes, you want to set colors or size manually. Like, "plot all points blue regardless of what kind of data it is."

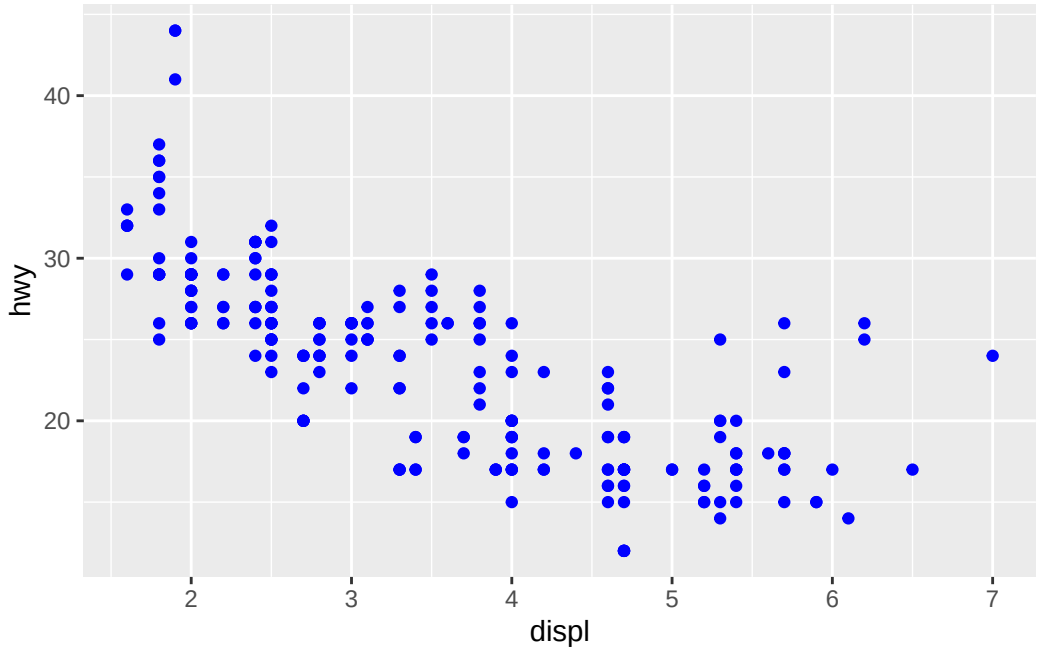
```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy), color = "blue")
```



In the plot above, look at where "color =" is located. When you put "color =" in aes(), which differentiates type of value, you're assigning colors per type. However, if you put "color =" outside of aes(), you're telling geom_point() "Ignore boundaries. Plot all points using blue."

What will happen if you accidentally put "color = blue" in aes()?

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy), color = "blue")
```



Why do you see red dots here? Because you put color argument in aes(), ggplot() thinks there is a column that saves information called "blue" and try to assign color based on values in "blue" (in reality, we don't even have column called "blue"). So, data points are assigned with random color instead of blue you intended.

8.3.5 group

So far, we only looked at geom_point(). While we will talk about other types of graphs more later on, let's look at geom_line().

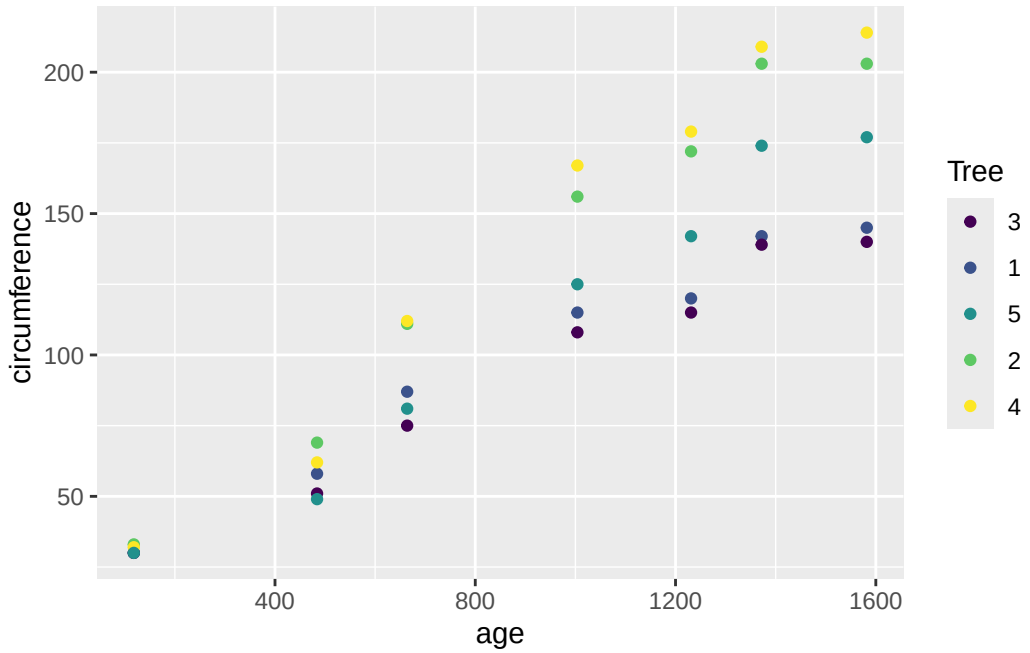
For better demonstration of why I'm bringing up geom_line in aes() section, let's use different data set other than mpg. 'Orange' data set tracks age and circumference of different trees.

```
Orange
```

```
   Tree  age circumference
1     1  118            30
2     1  484            58
3     1  664            87
4     1 1004           115
5     1 1231           120
6     1 1372           142
7     1 1582           145
```

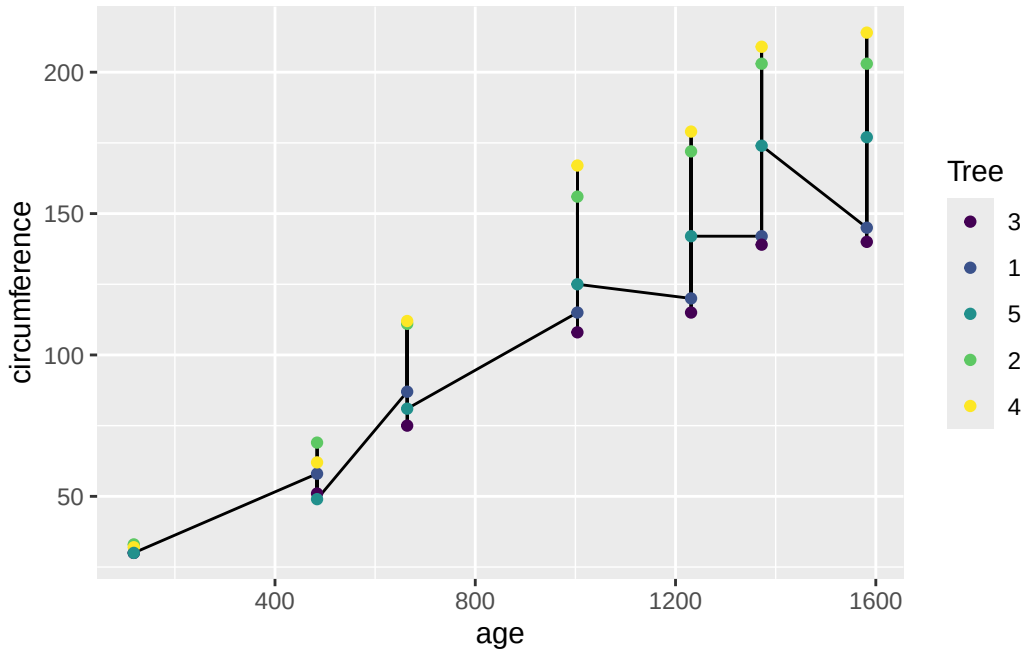
```
8      2  118            33
9      2  484            69
10     2  664           111
11     2 1004           156
12     2 1231           172
13     2 1372           203
14     2 1582           203
15     3  118            30
16     3  484            51
17     3  664            75
18     3 1004           108
19     3 1231           115
20     3 1372           139
21     3 1582           140
22     4  118            32
23     4  484            62
24     4  664           112
25     4 1004           167
26     4 1231           179
27     4 1372           209
28     4 1582           214
29     5  118            30
30     5  484            49
31     5  664            81
32     5 1004           125
33     5 1231           142
34     5 1372           174
35     5 1582           177
```

```
ggplot(data=Orange) +
  geom_point(mapping = aes(x = age, y = circumference, color = Tree))
```



Here, we see a clear linear relationship between age and circumference per tree. Let's draw a line graph to make this different growth per tree clear!

```
ggplot(data=Orange, mapping = aes(x = age, y = circumference)) +
  geom_line() +
  geom_point(mapping = aes(color = Tree))
```

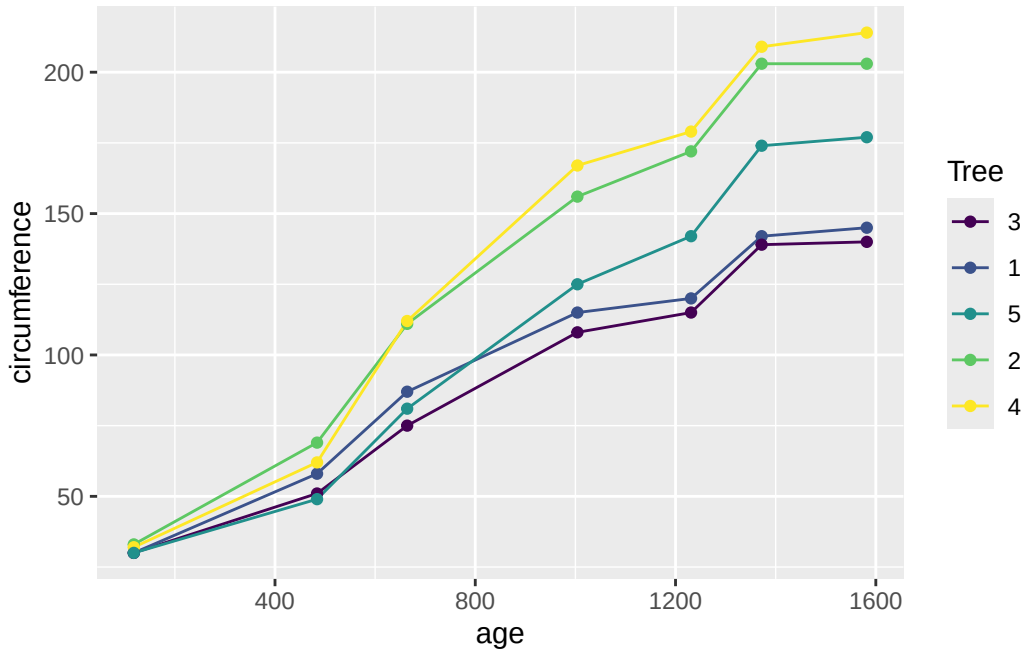


The line graph doesn't look like what we expected. Instead of multiple lines per tree, we see a single line that connects all existing values from smallest to largest x value (age). Why is this happening?

In human eyes and brains, we think "oh each trees are unique, so we should connect lines using data from same tree only!" However, code only does what they are told.So when we map aes(x = age, y = circumference), ggplot thinks "I should draw single line graph using all data we got!" not knowing the data is mix of different trees.

So, we need to tell ggplot() to draw line per tree. For this purpose, we use "group =" argument within aes(). Look at Orange data set again. In Trees, each are labeled with unique numbers. Now, let's draw another line graph using this point.

```
ggplot(data=Orange, mapping = aes(x = age, y = circumference)) +
  geom_line(mapping = aes(group = Tree, color = Tree)) +
  geom_point(mapping = aes(color = Tree))
```

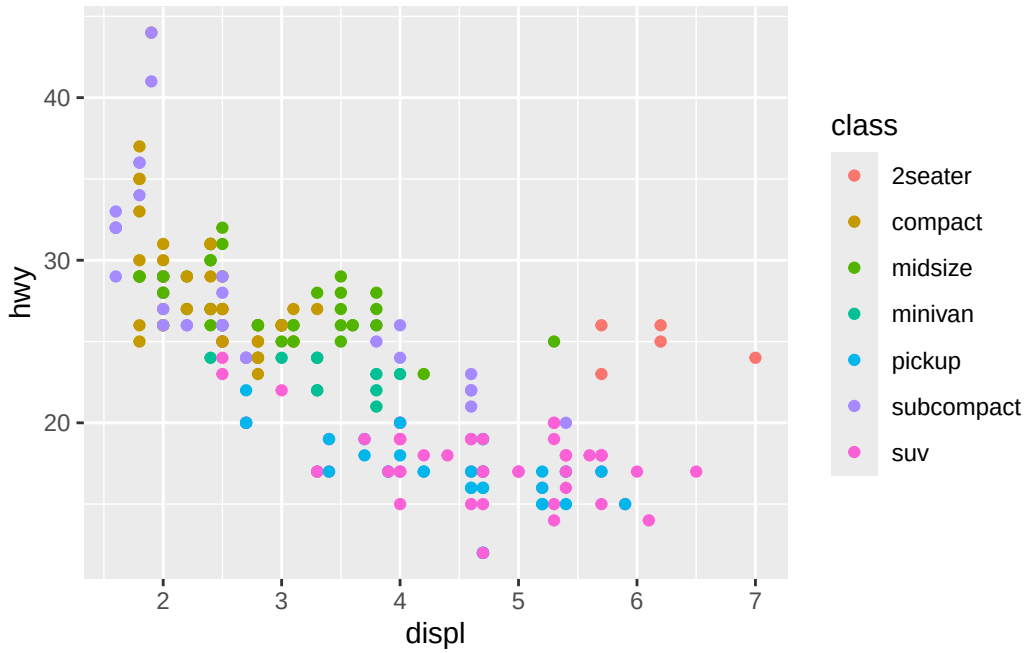


By using group argument, ggplot() knows which subset of data it should use to draw a accurate line graphs.

8.4 facet

Let's revisit mpg data.

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class))
```



Here, there were so many different car types that when we try to look for specific car's data, for example subcompact, it takes some time finding all data points that belongs to subcompact cars. Could we generate multiple plots using same axis, so that we can see plot per car class? That's where facet is used. When we segment a graph into multiple little graphs to plot subsets separately, we say we are plotting with facets.

Before we use facets, we need to understand operator "~" It means "modeled by." There are two ways to understand how it's used.

1: As in equation (generally): One way that ~ is used is when you state relationship between two. So, y ~ x mean "Use x to explain y, or"y is function of x" For example, y = x^2, we could say y ~ x.

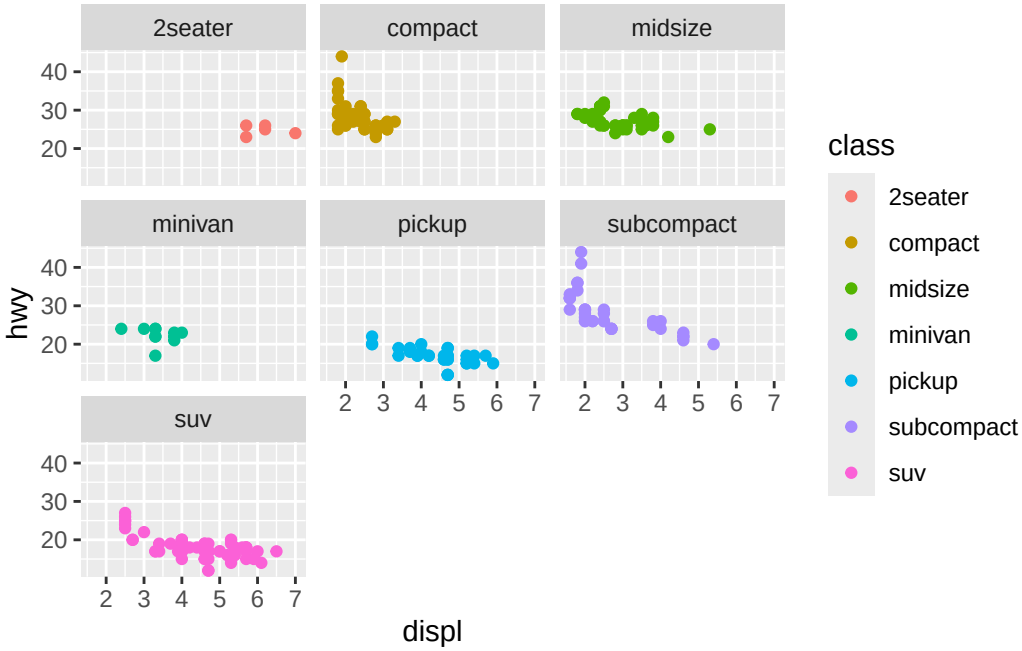
2: As in geometry (for ggplot()): It's similar to way #1. When you try to plot something in 2D graph, you need one variable for X axis and another variable for Y axis. It's like drawing y = x^2 onto 2D plot. So in this case, y~x means y (everything left to ~) is variable for rows (Y axis) and x (everything right to x ~) is variable for xolumn (X axis).

Then what would facet_wrap(~x) mean? ggplot() would draw facets of x, and x will be column.

8.4.1 1D facet

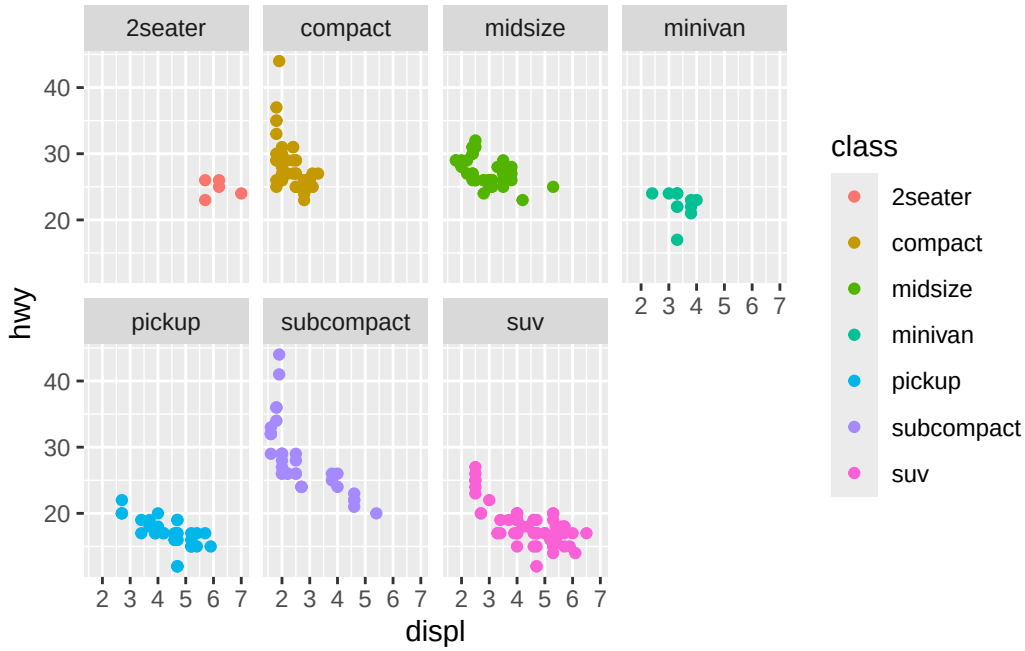
plotting with facets is divide into two steps: First, we draw draw graph with everything. Second, we divide plot (we call this wrapping) into facets.

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class)) + #First we draw
  facet_wrap(~class) #Second we divide
```



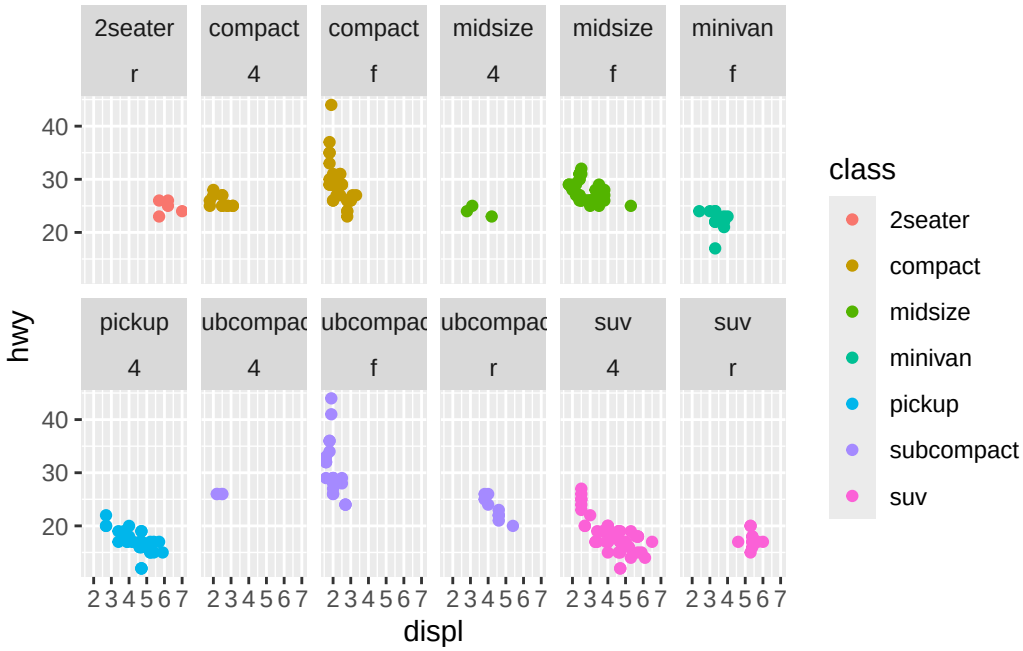
Now, you see same plot but per specific car class. Also, do you see how ~class makes class category on the column names (y~x)? If it's not clear, don't worry! 2D facet example later on will make it clear. If you want to set number of rows facets should be displayed, you can add nrow = argument.

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class)) + #First we draw
  facet_wrap(~class, nrow = 2) #Second we divide, but display them in two rows
```



If you want to use multiple variable per plot, you can connect them using "+" just like how we stack layers in ggplot().

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class)) +
  facet_wrap(~class + drv, nrow = 2)# drv was wheel motion type, r (rear), f (forward), 4 (f
```



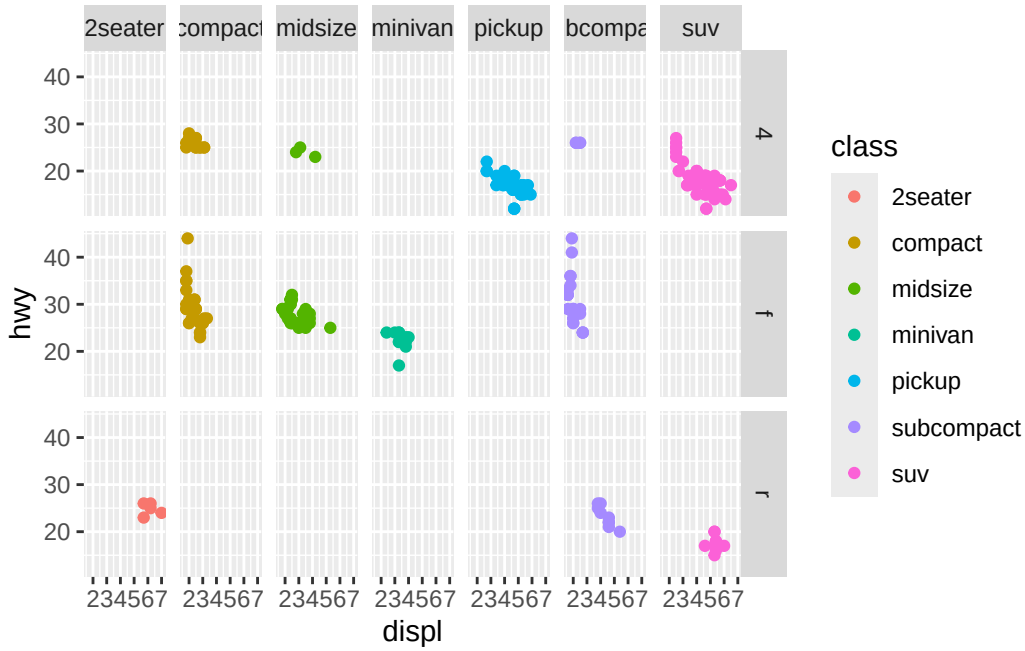
Now facets are not just based on the class, but combination of class and drv.

8.4.2 2D facet

But if you noticed, number of plots increase dramatically in 1D facet because you're displaying all possible combination of variables in single row, like a ribbon. However, readability of the ribbon will decrease exponentially as number of variable increases. For instance, graph we made just above, you need to read 12 different combination every time (which makes difficult to track which is which). So, we use 2D facet.

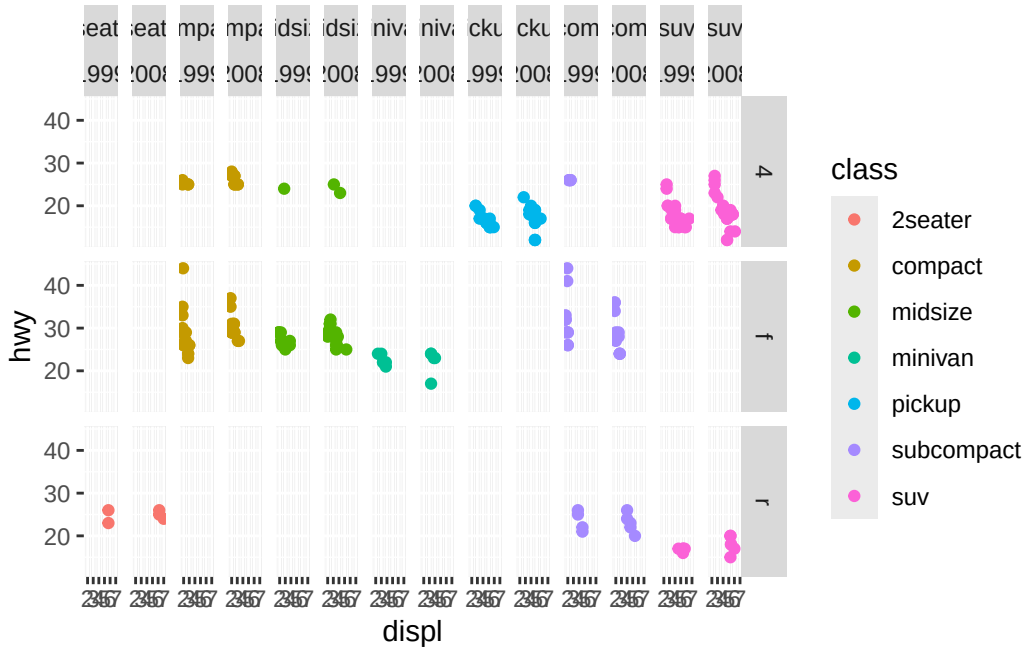
When you create 2D facet, facet_grid() is used instead of facet_warp(). Also, you use A~B instead of ~B (because you need variable for both row/column direction in 2D, where as 1D only need one variable).

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class)) +
  facet_grid(drv ~ class)
```



Just like 1D facet, you can use combination of different variables.

```
ggplot() +
  geom_point(data = mpg, mapping = aes(x = displ, y = hwy, color = class)) +
  facet_grid(drv ~ class + year)
```



8.5 geom()

Now, let's explore different types of plots in ggplot().

8.5.1 Simple statistics to ggplot()

There are lots of geom() functions that fits different situations. First, let's look at some geom() that is helpful turning summarized statistics into plot.

8.5.1.1 Categorial Data ()

8.5.1.1.1 geom_col()

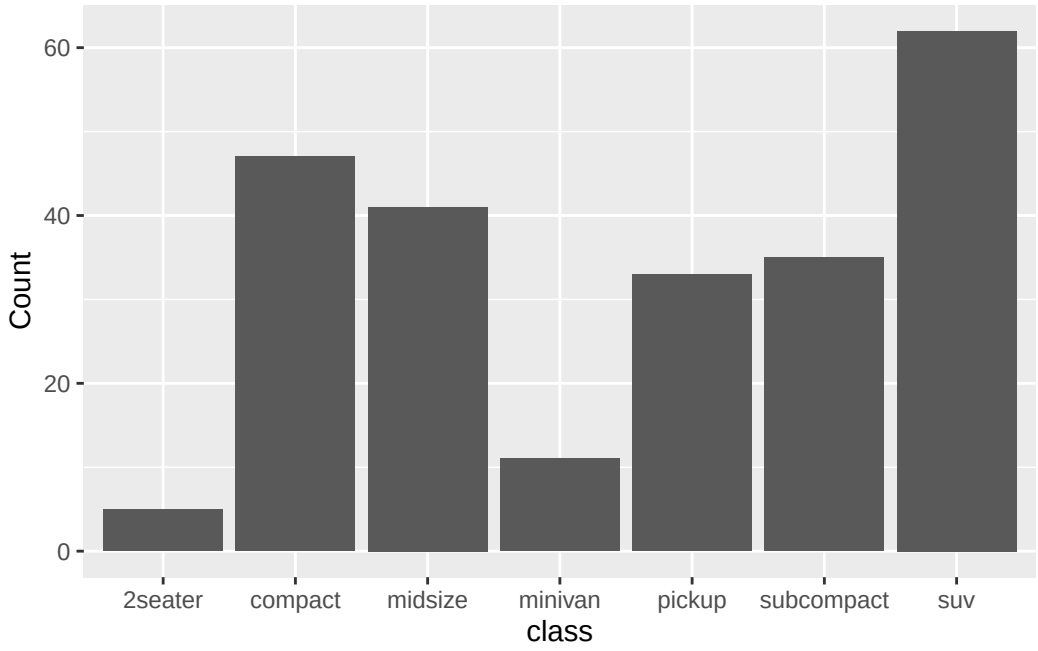
Here, let's see how we can plot number of cars in each car classes. How would you count number of cars in each class? Try it yourself! Hint: summarize()

```
class_count <- mpg %>%
  group_by(class) %>%
  summarize(Count = n())

class_count
```

```
# A tibble: 7 x 2
  class      Count
  <chr>      <int>
1 2seater        5
2 compact       47
3 midsize       41
4 minivan       11
5 pickup        33
6 subcompact    35
7 suv           62
```

One way we vizualize categorial data is bar graph (since x axis, categorial variable, are not continuous data we can connect using line) using geom_col().

```
ggplot(class_count) +
  geom_col(aes(x = class, y = Count))
```

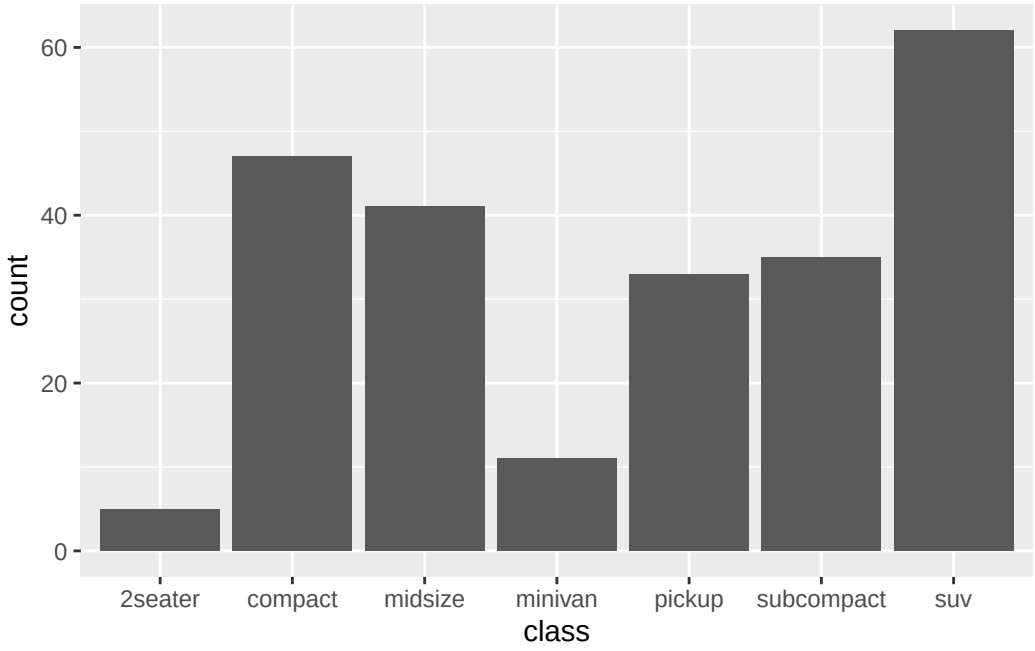


Nice looking bar graph! Except one minor hassle: for each bar graph, you need to calculate summarized stat every time. So, how do we make it easier?

8.5.1.1.2 geom_bar()

While both geom_col() and geom_bar() creates bar graph, there is one major difference: geom_bar() performs summarization for you. Let's look at how it works.

```
ggplot(mpg) +
  geom_bar(aes(x = class), stat = 'count')
```



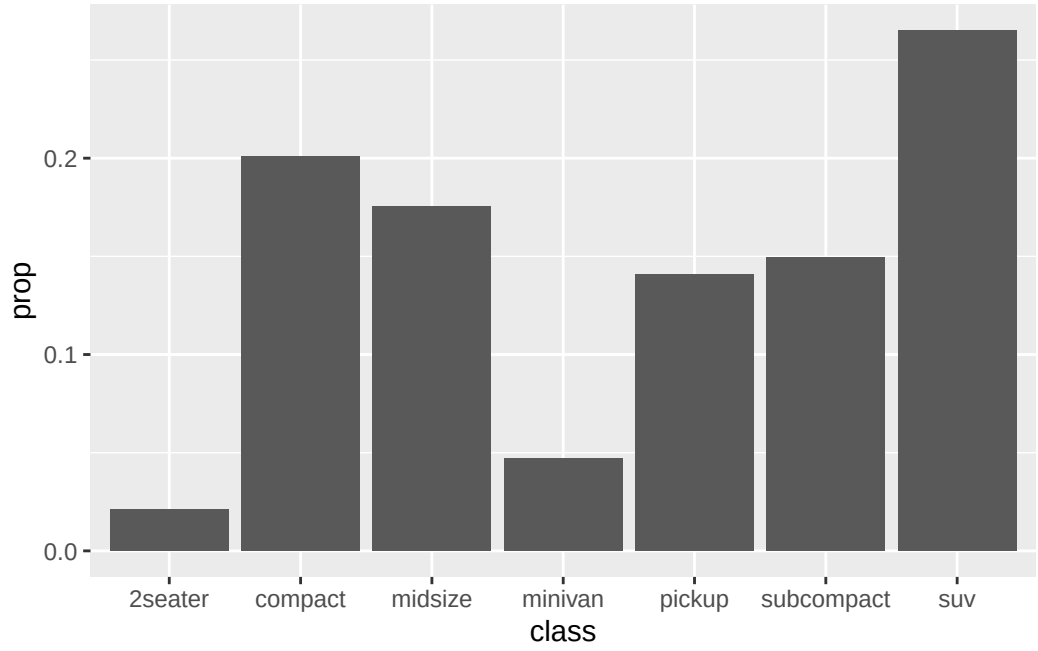
Same looking graph, but using mpg data directly without summarize()! What was happening in the geom_bar()?

In aes(x=class), we tell ggplot() that our categorial varialbe (or x axis) will be class. stat = 'count' makes geom_bar() to count how many times each category shows up in the data set. In this case, count() counts how many "suv" exist in the class column of mpg, how many "subcompact" exist in..., and so on.

What if you want to use percentage instead of absolute count on the y axis? we add after_stat(prop) argument. Why after_stat() instead of stat()? stat() allows us to choose column in original data set, while after_stat() allows us to choose column of summarized data by geom_bar().

```
Original Data (pick something in here using stat()) -> Summarized statistics (pick something
```

```
ggplot(mpg) +
  geom_bar(aes(x = class, y = after_stat(prop), group = 1))
```

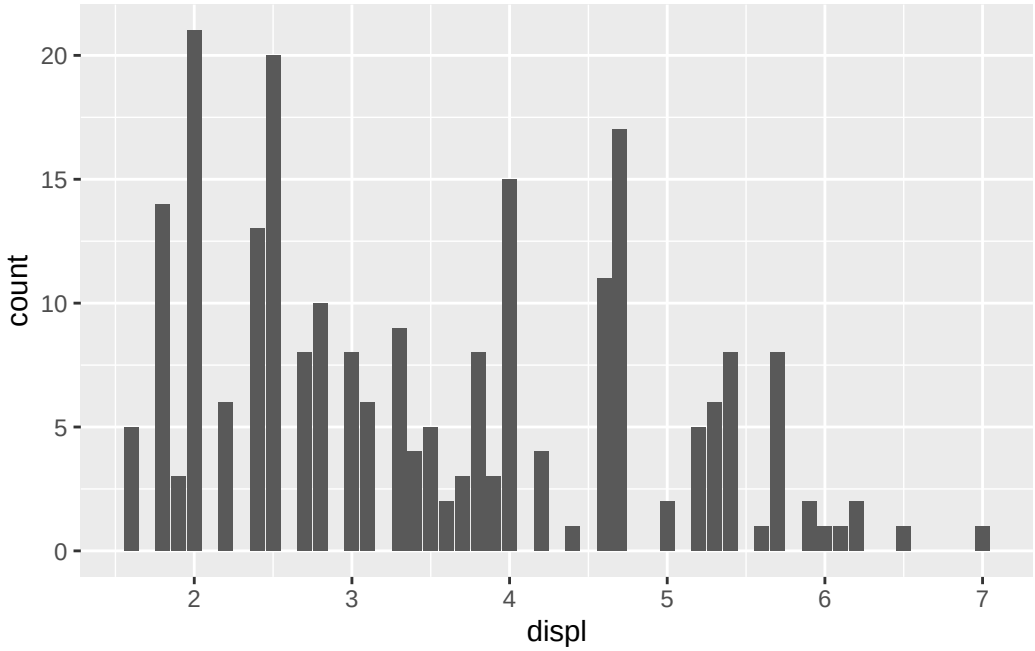


When you use prop, it is important to put 'group = 1.' It's because how geom_bar() calculates proportions. geom_bar() calculates them by looking at 'how many items meet condition/how many items in selected group' In that specific code, proportion is calculated by 'how many cars in class X/how many cars are there in selected group' Because we're calculating proportion from after_stat (where data are summarized in class - count manner), geom_bar() will look at 'how many cars are suv / how many cars are in suv category' : all class will return prob of 100%. So by labeling all cars same (giving tag '1' to everything), now geom_bar() calculates suv out of total cars.

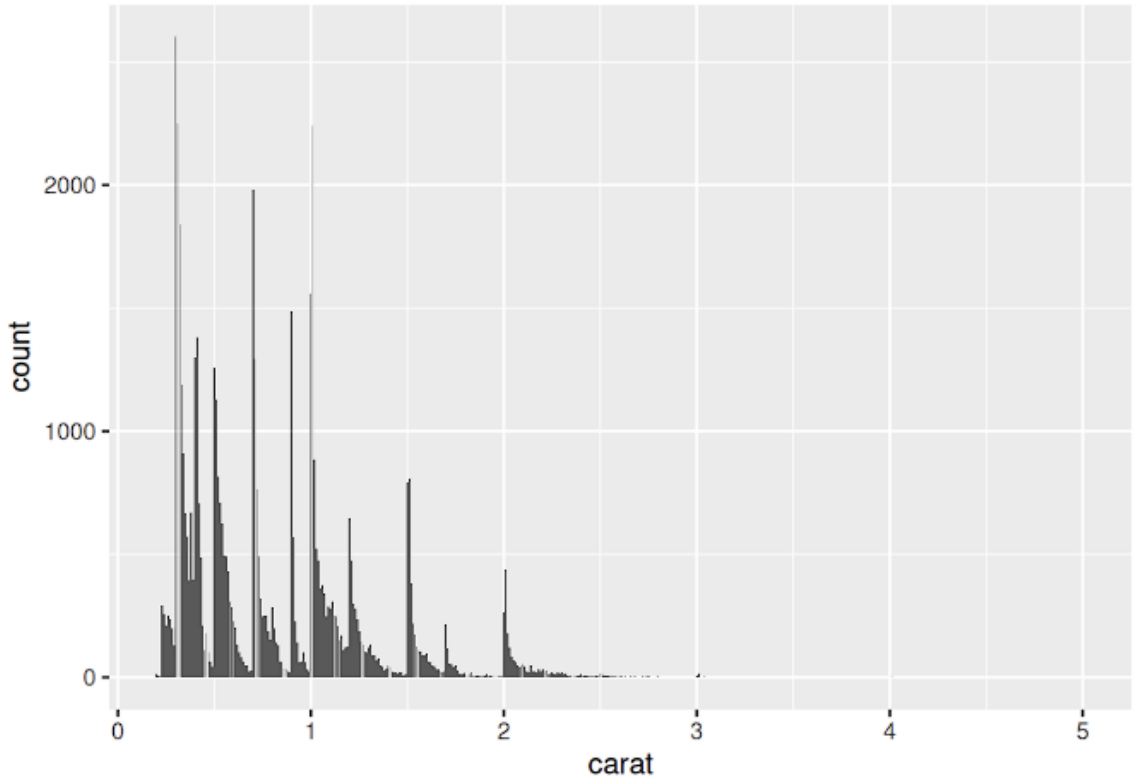
8.5.1.2 Numerical Data

Using geom_bar() on continuous data creates too many columns, which makes us difficult understanding which column is what.

```
ggplot(mpg) +
  geom_bar(aes(x = displ))
```



Well you might say you can read each column. Okay, then what if we draw bar graph of number of diamond per carat?



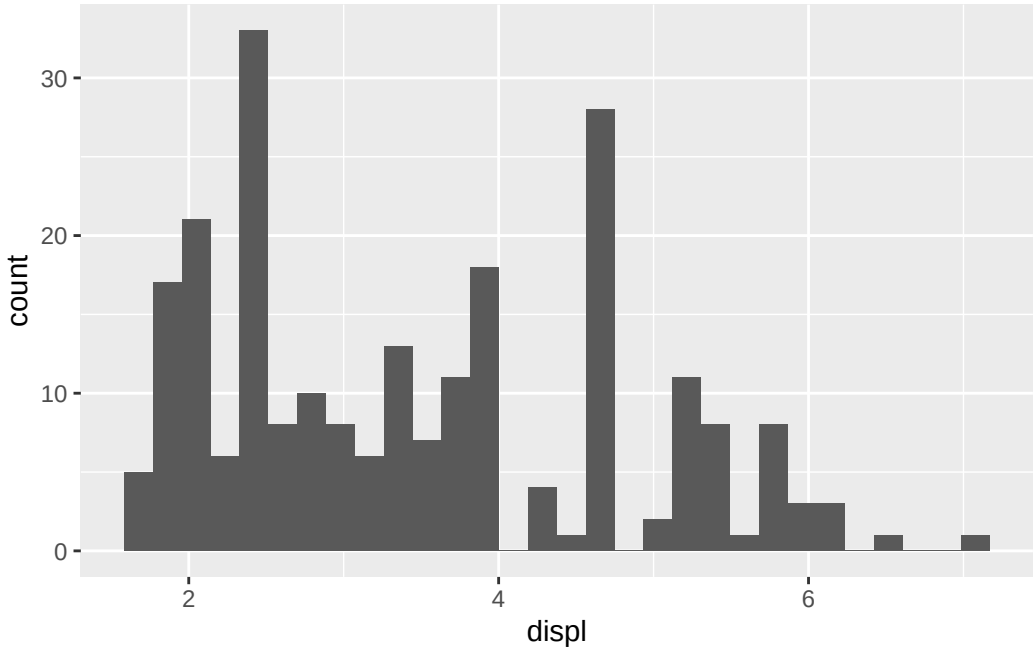
Now, can you even tell how many different caract exist in the data set? probably not.

8.5.1.2.1 geom_histogram()

One handy way to show count of continuous data is to use histogram. Instead of displaying column per value, you can group them and display together. For example, putting everything with displ value of 1~2 into single column. There are two ways to draw histogram. First, you can use geom_bar() with "stat='bin' " argument. Second, you can use geom_histogram().

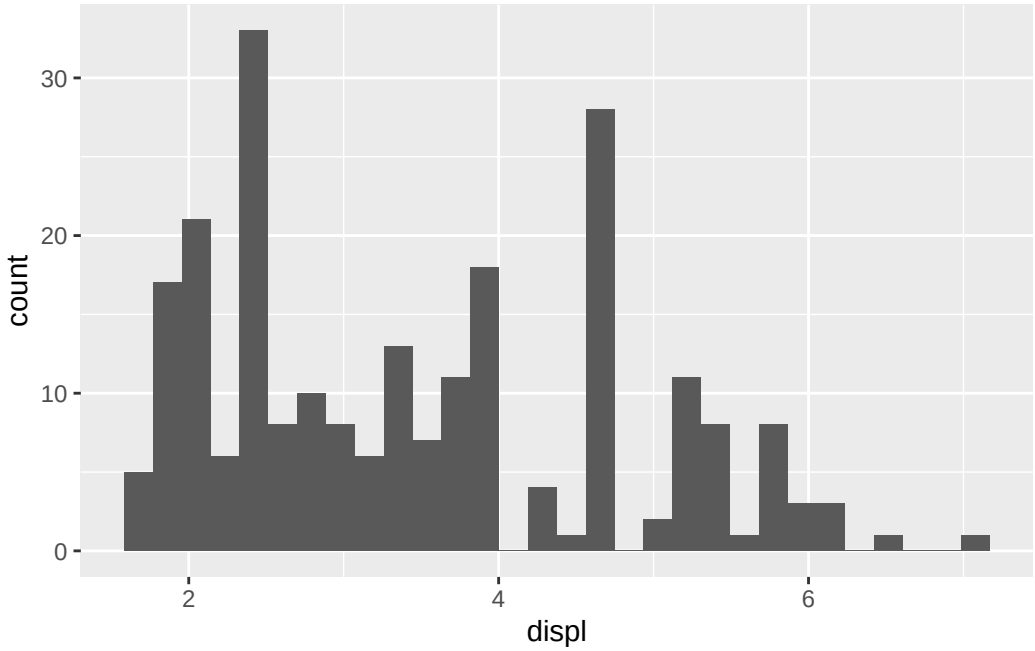
```
ggplot(mpg) +
  geom_bar(aes(x = displ), stat = 'bin')
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



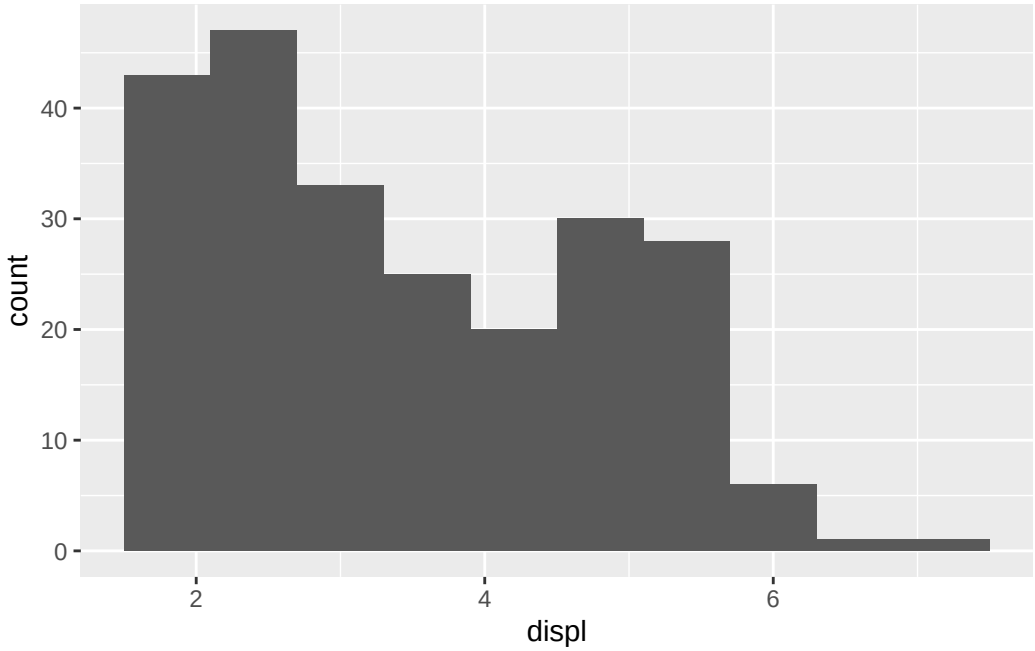

```
ggplot(mpg) +
  geom_histogram(aes(x = displ))
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

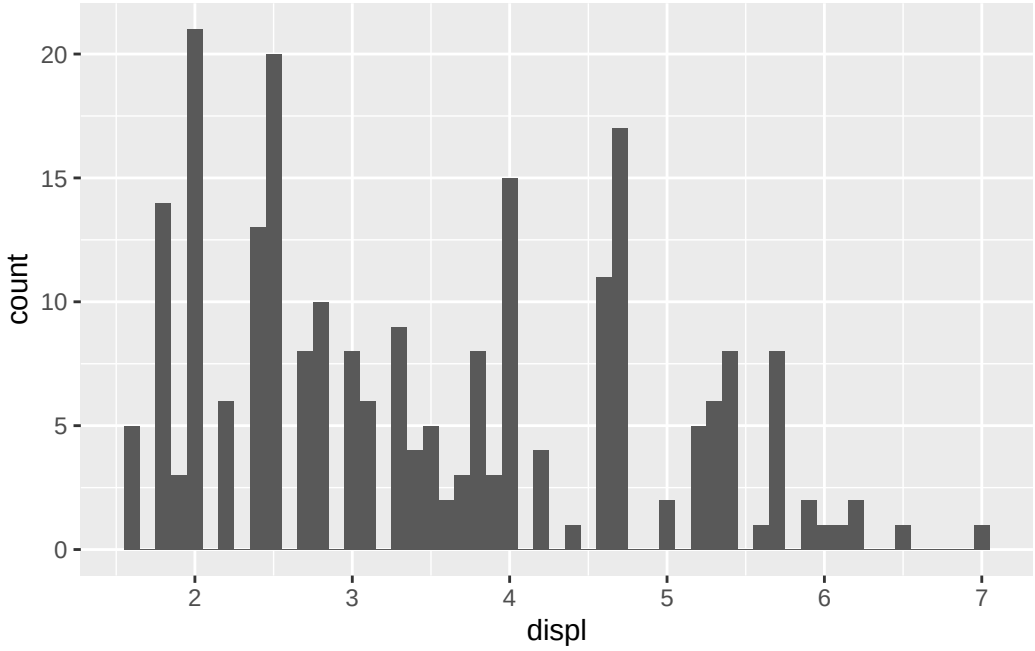


I prefer to use geom_histogram() because of its capability to cusomize: You can change number of column (bins), or binwidth (range of each bin).

```
ggplot(mpg) +
  geom_histogram(aes(x = displ), bins = 10) #Use only 10 bins
```



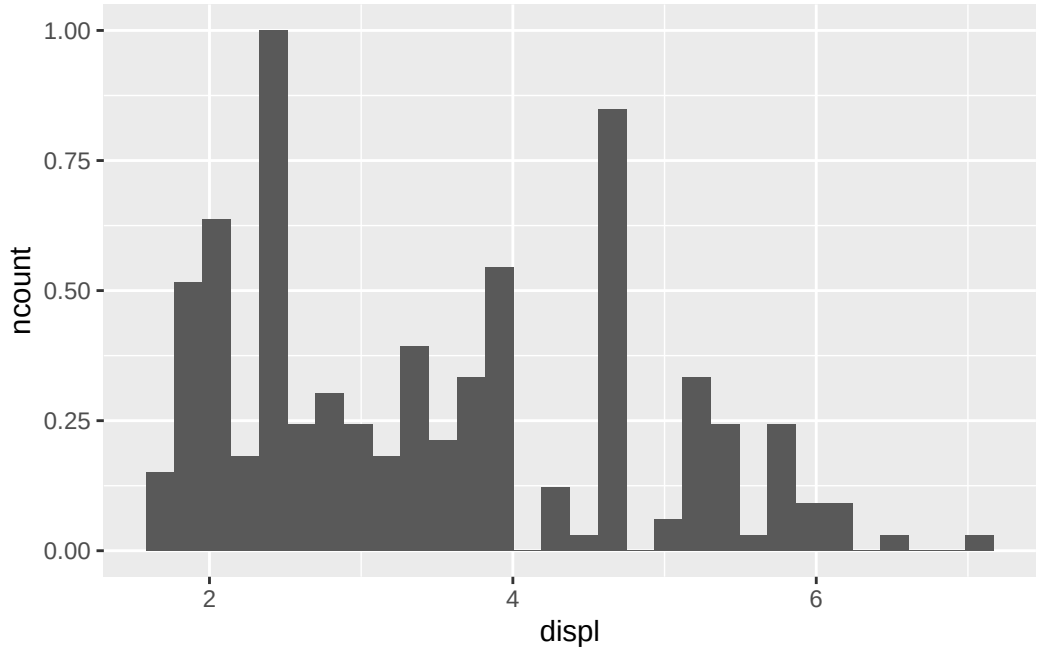

```
ggplot(mpg) +
  geom_histogram(aes(x = displ), binwidth = 0.1) #Group displ by increment of 0.2.
```



Just like after_stat(prob), you can also draw density/proportion graph using after_stat(density). If you want normalized density (maximum value = 1), you can use after_stat(ncount) and after_stat(ndensity). n + something as Normalized + something.

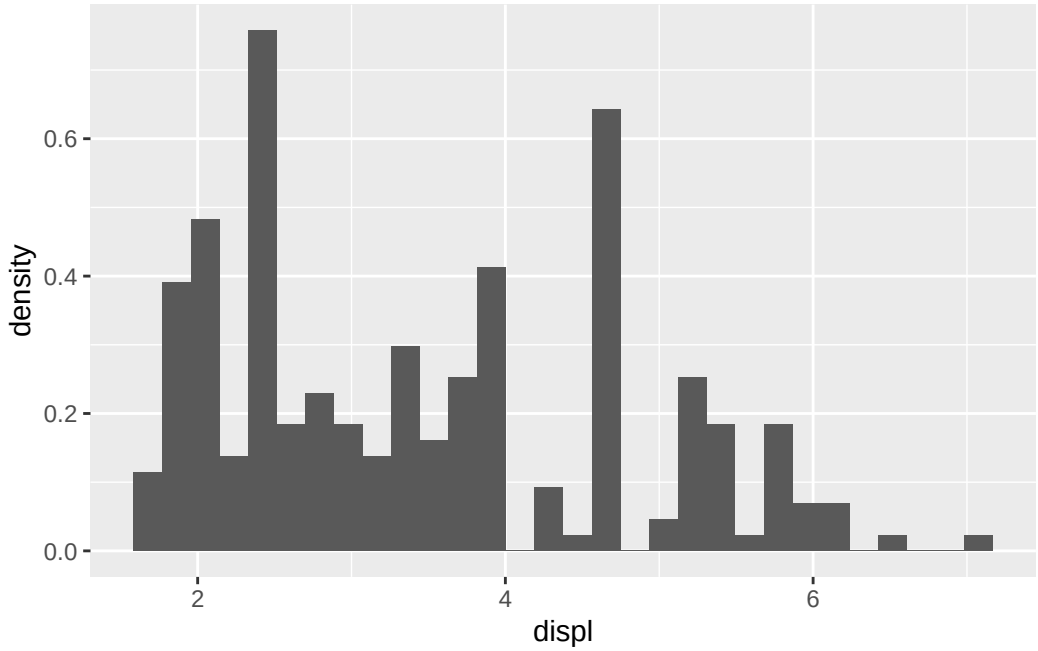
```
ggplot(mpg) +
  geom_histogram(aes(x = displ, after_stat(ncount)))
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```




```
ggplot(mpg) +
  geom_histogram(aes(x = displ, after_stat(density)))
```

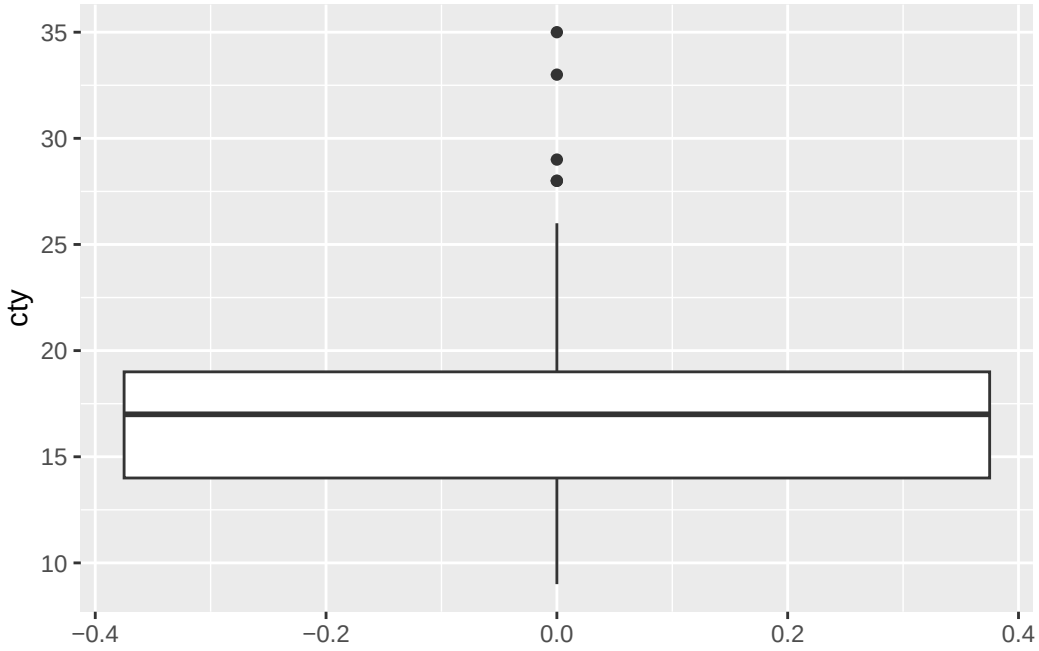
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



8.5.1.2.2 geom_boxplot()

Last, you can draw box plot that summarize quartiles of data set.

```
ggplot(mpg) +
  geom_boxplot(aes(y = cty))
```



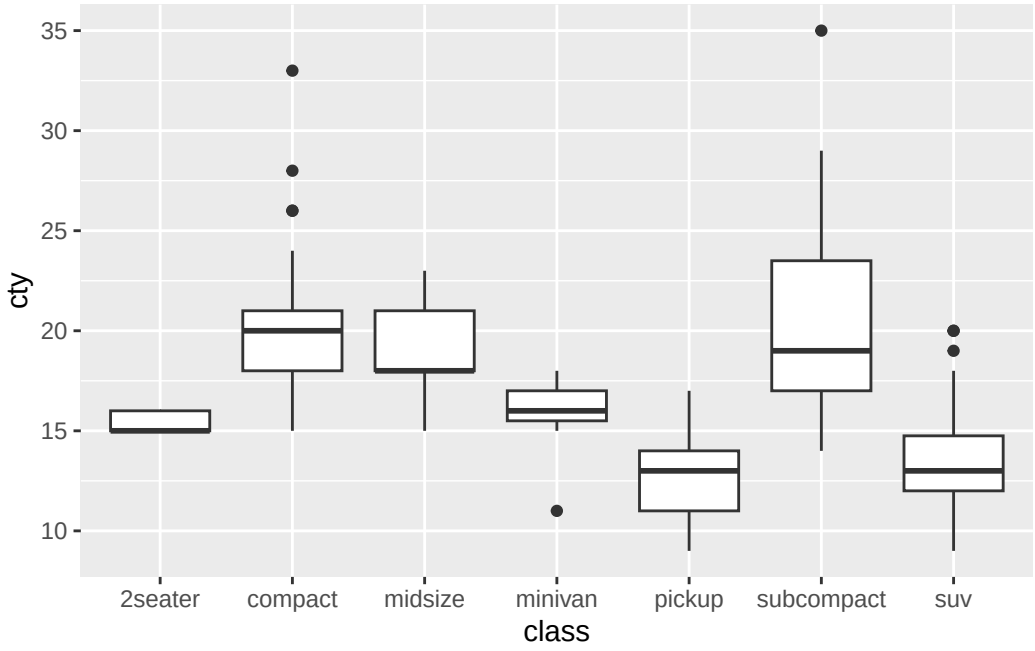
box plot is advantageous when you want to show the spread of the data.

y value: Data you want to show the spread
Mid line of the box: 50% percentile (median)
Lower line of the box: 25% percentile (Q1)
Upper line of the box: 75% percentile (Q3)
Distance between lower and upper line: Inter-Quartile Range (IQR)
Vertical line: data within 1.5IQR
Outliers: data outside of 'Q3 + 1.5IQR' or 'Q1 - 1.5IQR'
Dots: Data points that are outlier

If you want to draw boxplot where spread is shown in the x axis, you simply need to change aes() to 'x='

When do we use boxplots? It becomes useful when you compare numerical value's statistics among categorical data. Earlier, we could compare the count of the car class. However, what if you want to compare cty of each classes? Even you draw histogram of numerical data (cty) for each categorial data (car class), comparing multiple graphs side to side is not easy. In this case, boxplot becomes handy.

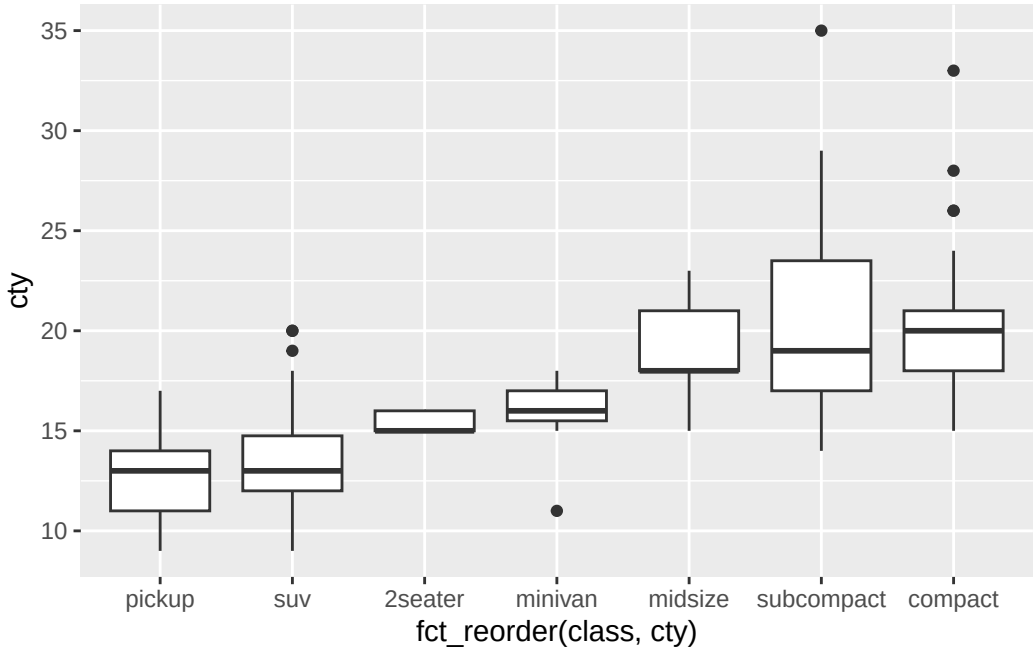
```
ggplot(mpg) +
  geom_boxplot(aes(y = cty, x = class))
```



Now, you can easily compare median values of cty of each class and IQR range to show 'which class is more efficient' easily.

*Small Note box plot above makes comparison easy, but readability is low as median are in random order. We can use fct_reorder() (Review concept on factors!) to order classes based on cty values.

```
ggplot(data = mpg) +
  geom_boxplot(mapping = aes(x = fct_reorder(class, cty), y = cty))
```



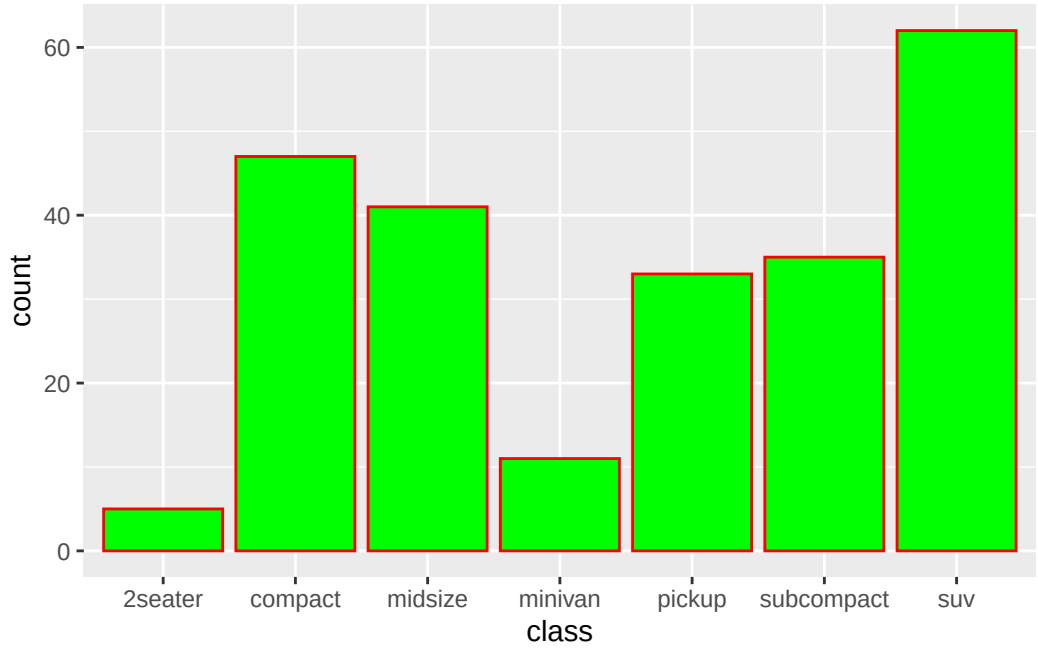

```
# in fct_reorder, We're ordering class using median cty value.
# fct_reorder(A, B) A: Data you want to order, B: Function you want to use for ordering (defa
```

8.5.1.3 Visual of bar graphs

As we saw earlier, we can edit color of bar graphs. First, let's try changing color of all bars. Rememer when to put argument in and out of aes()? If you want to change all colors regardless of features (classes, drv, etc), where should you put the argument?

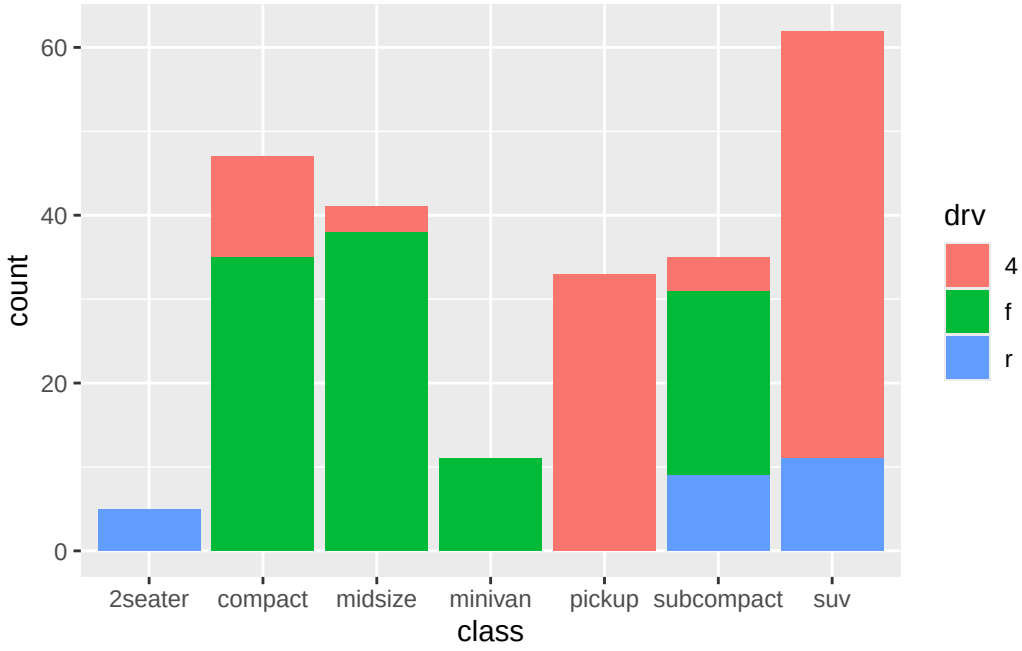
In bar graph, there are two separate colors. Color for the border of bars uses 'color' argument, while color of bars themselves uses 'fill' argument.

```
ggplot(mpg) +
  geom_bar(aes(x = class), stat = 'count', color = "red", fill = 'green')
```



Now, what if you want to see count of drv per class? In this case, we should use multiple colors per bar to show different drv types within the bar (feature dependent). In this case, should color/fill argument go in or outside of aes()?

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv))
```



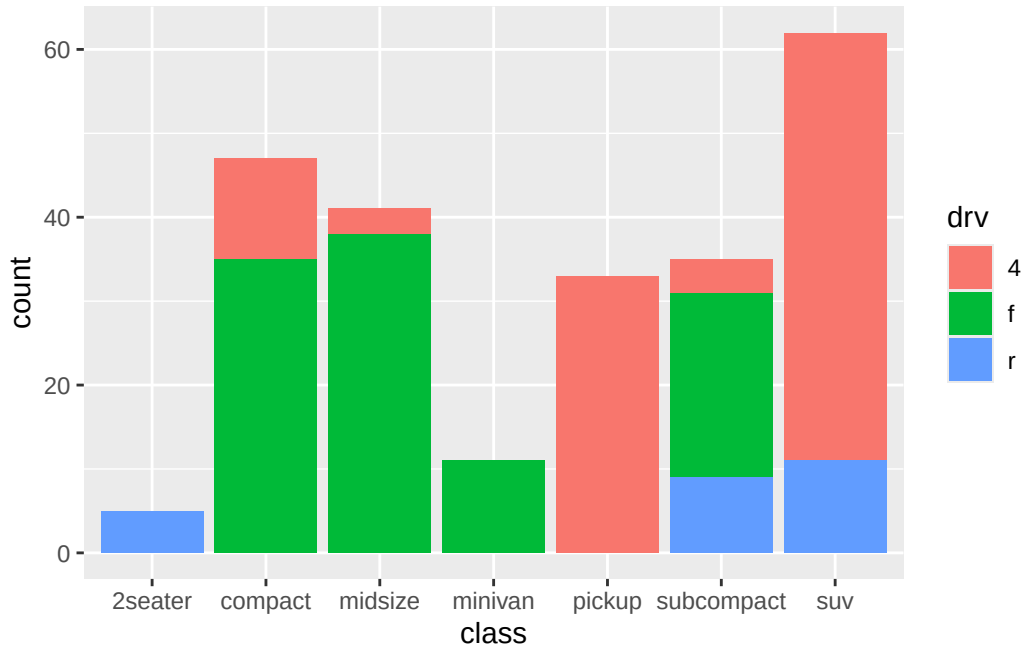

```
#small note: default stat() of geom_bar is count. So you can omit stat = 'count'
```

Pause, one question. Some might say "Stacking all drv types into single column makes count difficult to read. I want to be able to read each drv counts easily." Valid challenge accepted. Since we asked ggplot() to create bar graph based on class and count, each bars only shows how many class X exist - no matter how many forward, or rear wheel cars are within that column. So, for people who want to see those minor details, we can separate column by details using 'position' argument.

Here's option for position argument:

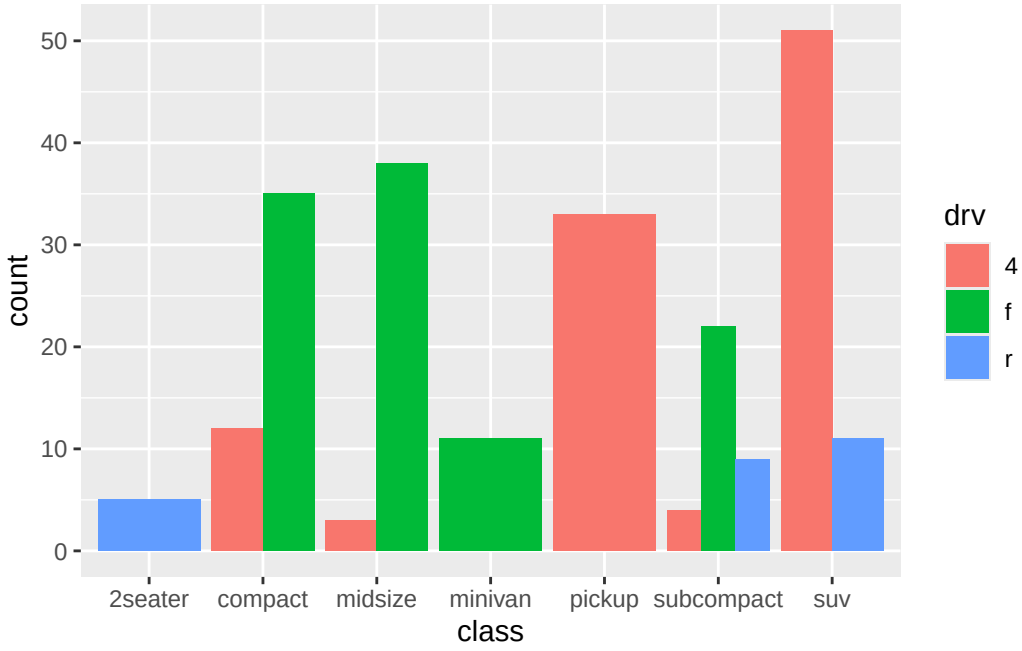
stack: This is the default of position argument. It "stacks" one on the top of another.

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv), position = 'stack')
```



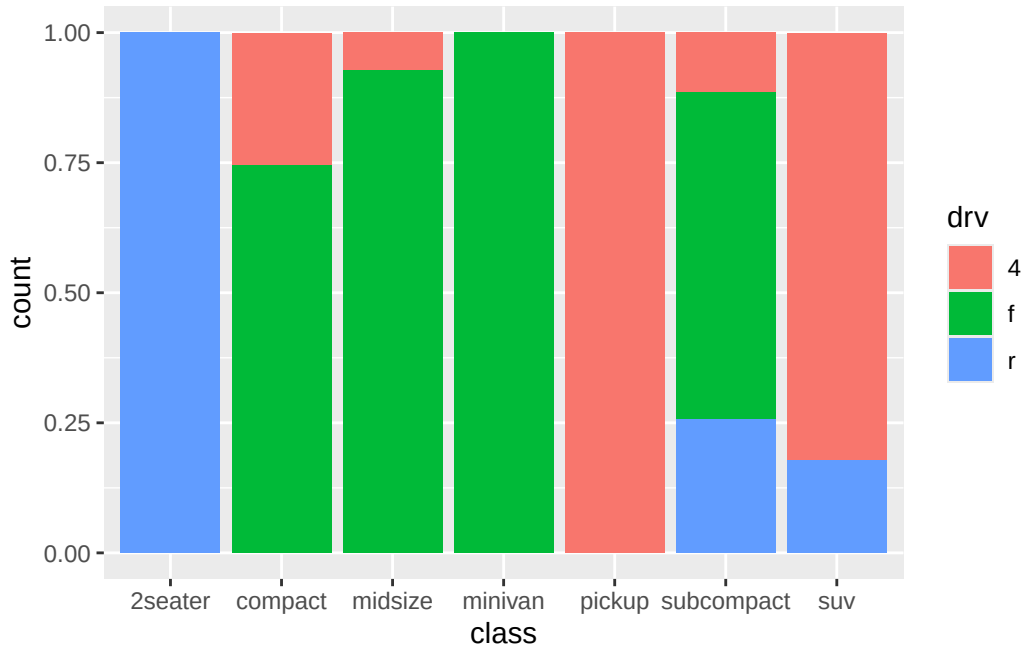
dodge: Place different sub-types (drv in this case, where fill color is differentiated) next to each other.

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv), position = 'dodge')
```



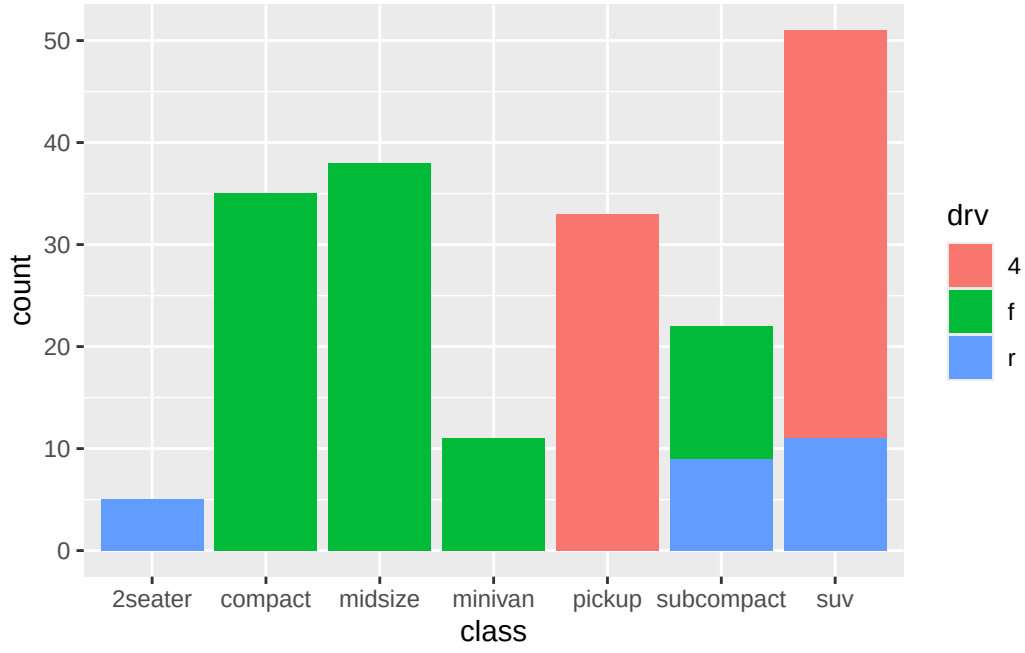
fill: Show percentage of each sub-type within the column. Which means, all column will be fixed at 1.0 (since combining all percentage of sub-types within the column will be 100%)

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv), position = 'fill')
```



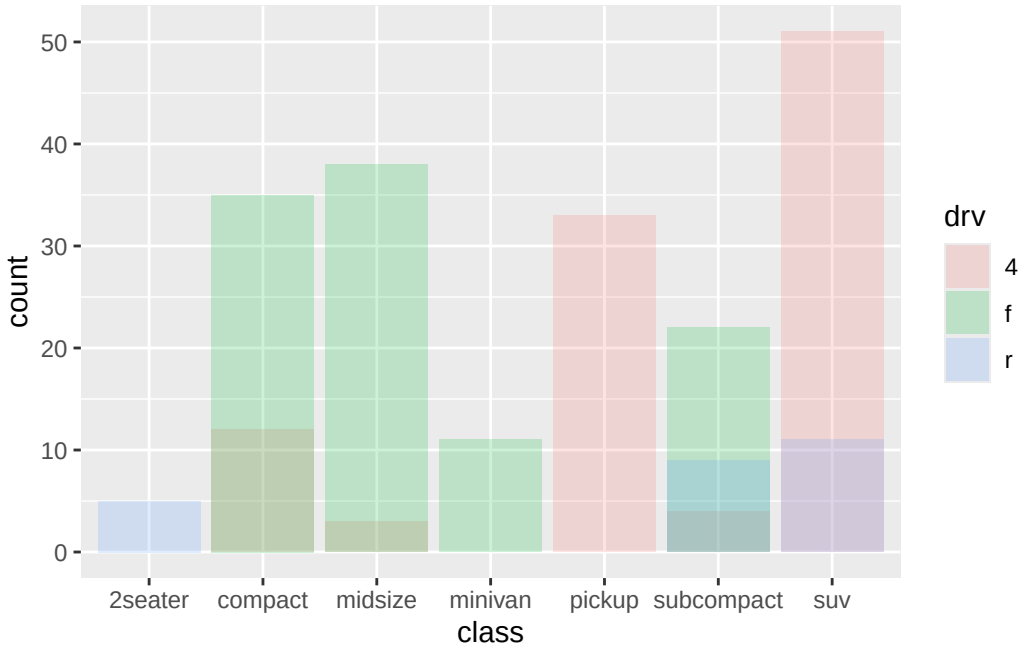
identity: Put all sub-types starting from y = 0.

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv), position = 'identity')
```



For the graph above, do you see how four wheel drive (red) cars are missing from compact and midsize? This is because identity stack all sub types starting form y = 0, so smaller sub-types will be hidden by bigger ones. So, you must mix with alpha so that you can see everything.

```
ggplot(mpg) +
  geom_bar(aes(x = class, fill = drv), position = 'identity', alpha = 0.2)
```




```
# Again, remember that we're setting all bars with alpha = 0.2, not assigning different alpha
```

8.5.2 More geom() types

There are a lot of geom(). These are list of frequently used geom().

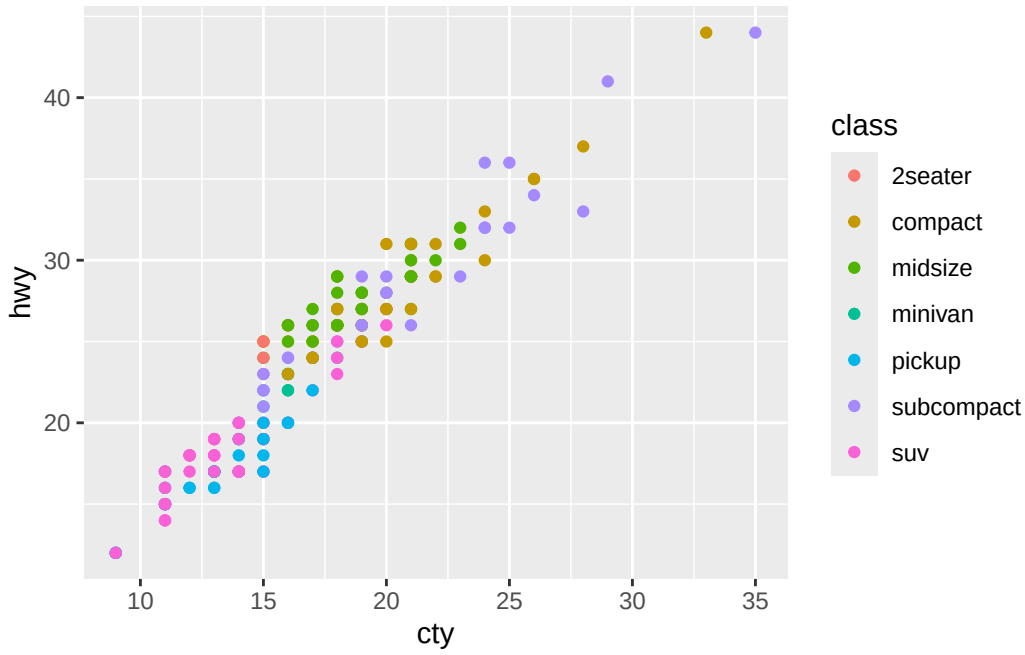
geom Function	Plot Type	Properties
`geom_point()`	Scatterplot	color, alpha, shape, size
`geom_line()`	Line graph	color, alpha, linetype, size
`geom_bar()`	Bar chart	color, fill, alpha
`geom_histogram()`	Histogram	color, fill, alpha, linetype, binwidth
`geom_boxplot()`	Box plot	color, fill, alpha, notch, width
`geom_hline()`	Horizontal lines	color, alpha, linetype, size
`geom_vline()`	Vertical lines	color, alpha, linetype, size
`geom_jitter()`	Jittered points	color, size, alpha, shape
`geom_rug()`	Rug plot	color, side
`geom_smooth()`	Fitted line	method, formula, color, fill, linetype, size
`geom_text()`	Text annotations	Too many, google them.

Let's review them one by one.

8.5.2.1 Drawing Straight Line

Let's revisit one of our first example

```
p <- ggplot(mpg) +
  geom_point(aes(x = cty, y = hwy, color = class))

p
```



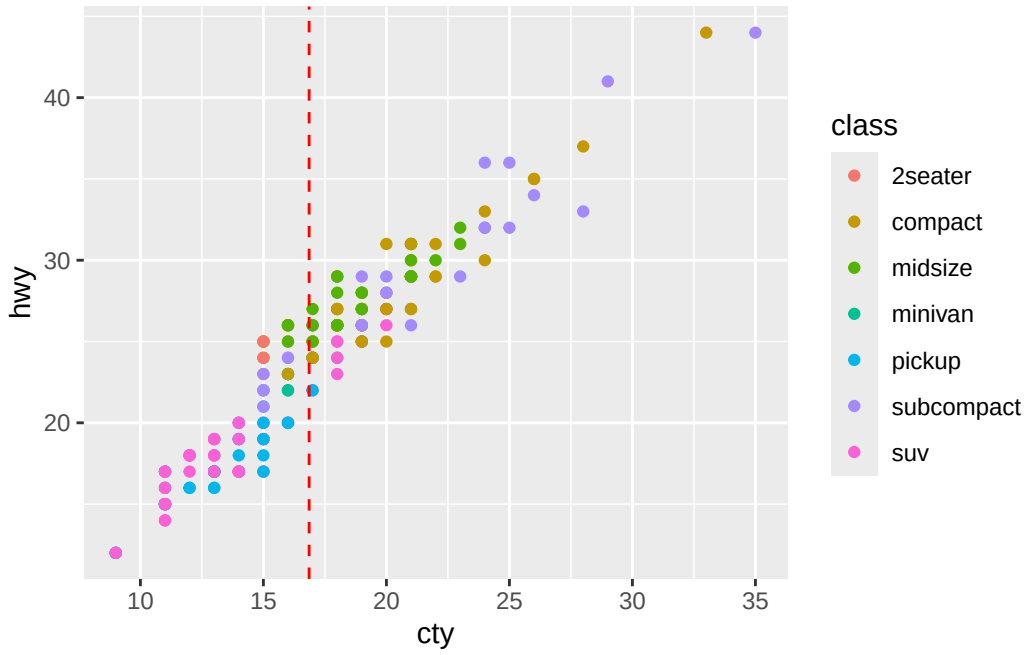
When we first see the graph, we want to question: What's the average performance? What's average x and y? (Just for this mpg data), do car perform better on highway or city?

To address these statistical question visually, we should draw a vertical line and horizontal line for average values.

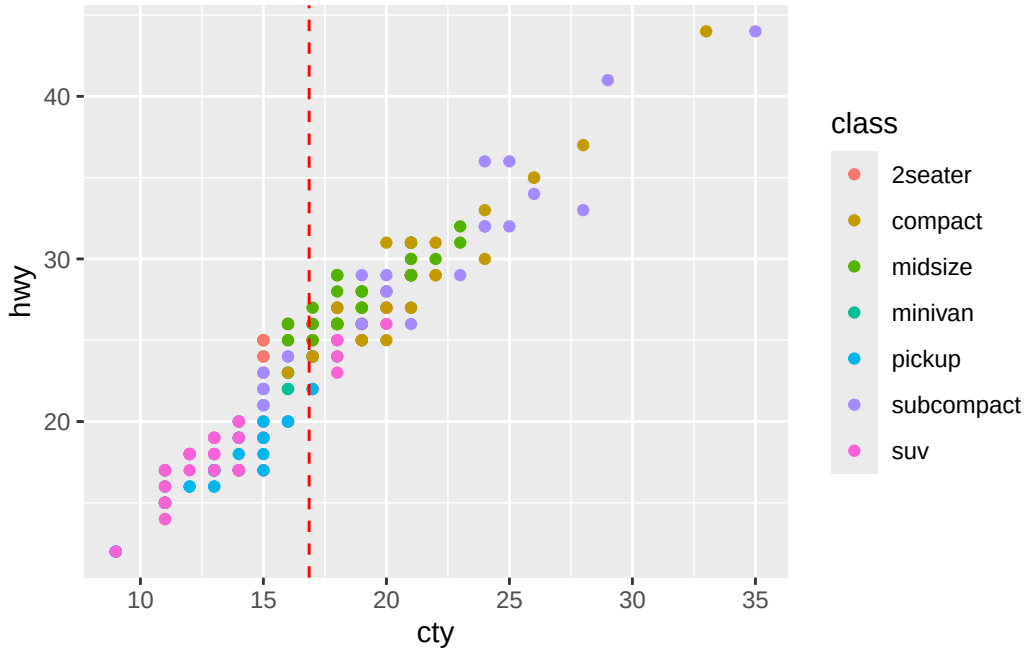
Drawing straight line has straightforward function name: vline for vertical, hline for horizontal line. Both functions, we need to define x and y interceptors.

For vline, we want x coordinate to be average of cty value.

```
p + geom_vline(aes(xintercept = mean(cty)), linetype = 2, color = "red")
```




```
# Change numbers and try out multiple linetype

p + geom_vline(xintercept = mean(mpg$cty), linetype = 2, color = "red")
```



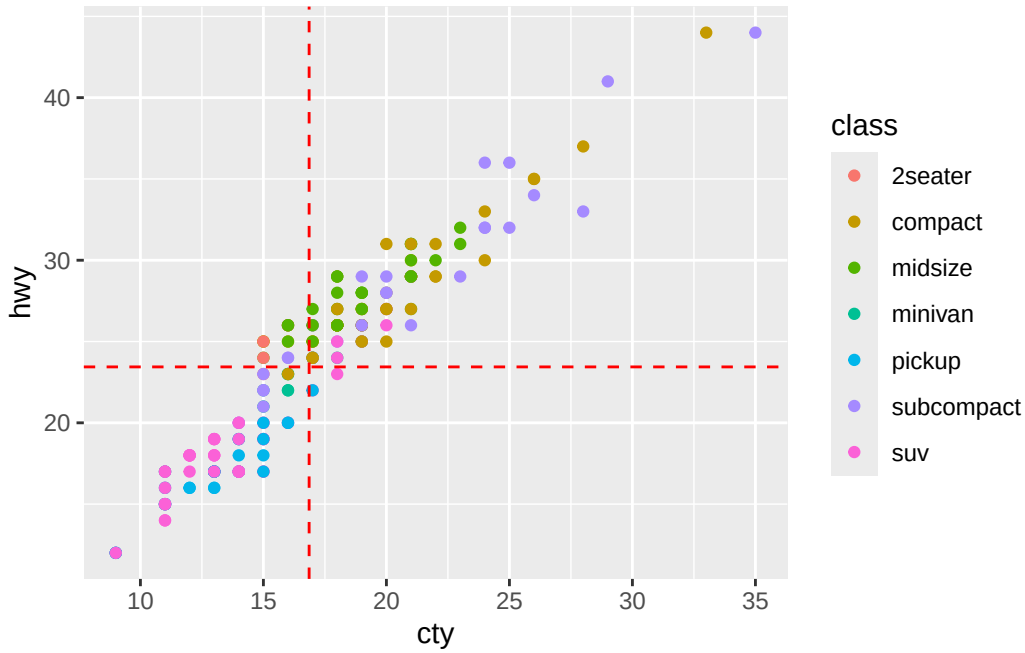
Few points you should remember. First is difference between first and second code. While output looks the same, they act different. Do you remember how aes() works? It defines the visualization of each data point. This means that anything inside aes() is checked every single point. So when it looks at x-intercept (x coordinate when y = 0, so x coordinate of vertical line) of first data point, it draws red line. When it looks at x-intercept of second data point, it draws red line again. And every time data point is plotted, vline will draw over and over. However, in the second code, x-intercept is defining x coordinate for geom_vline(), not aes(). So it's once again about arguments inside and outside of aes(). Very important.

Second, aes() was layer of ggplot (remember components in ggplot()?). So when you state you're looking at cty column in the aes(), it knows that you're talking about mpg data from the 'data =' layer. However, in the second version of code, your trying to use cty column outside of the aes. Because geom_vline() is just a function, not a layer, it doesn't know what data it supposed to use. So if there is no saved global variable called cty in the enviornment tab, it'll return error. As result, you must call mpg$cty.

Long story short, for time and resource efficiency, you should use second version of code for drawing vline and don't forget to index data set you're using.

Similary, let's draw hline to visualize average of hwy values.

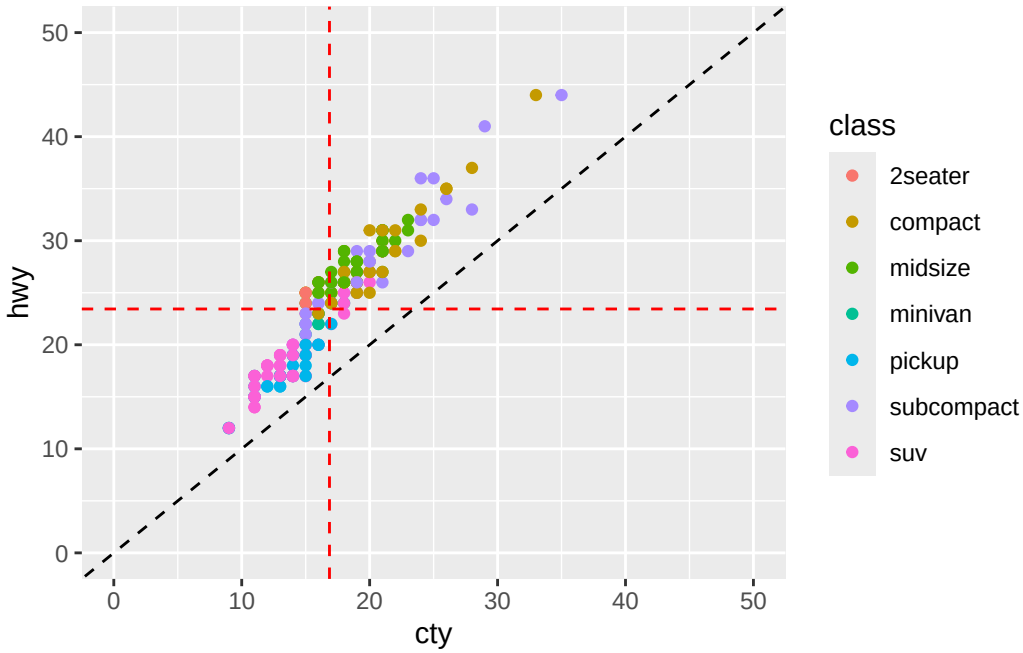
```
p1 <- p +
  geom_vline(xintercept = mean(mpg$cty), linetype = 2, color = "red") +
  geom_hline(yintercept = mean(mpg$hwy), linetype = 2, color = "red")

p1
```



Now, last question: does car perform better at city or hwy? Answer would be city if cty (city fuel efficiency) is higher than hwy (highway fuel efficiency), and other way around in the opposite case. Then, we could draw hwy = cty line and see if data point is above or below the line.

To draw diagonal line, we use abline(). parameter are slope and intercept. Also, if you want to cahnge the window/range of the plot, you can use scale_(axis you want to revise)_continuous() function to change the ends of the axis.

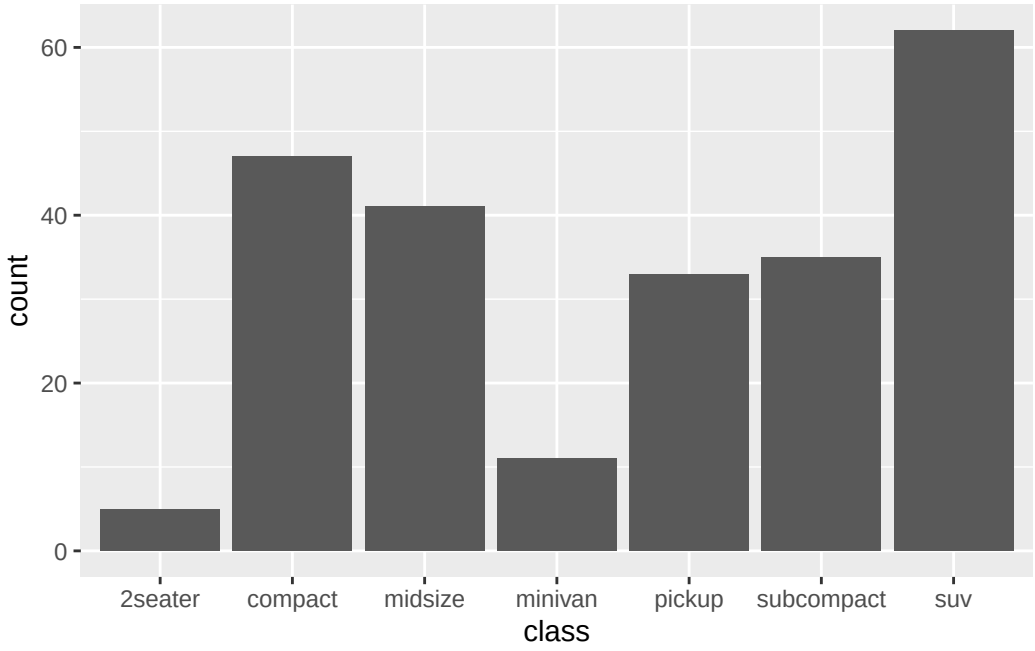
```
p1 +
  geom_abline(intercept = 0, slope = 1, linetype = 2, color = "black") +
  scale_x_continuous(limits = c(0, 50)) + #limits = c(0,50) as limit starts at zero and ends
  scale_y_continuous(limits = c(0, 50))
```



Looking at the plot, we can tell all cars have higher hwy than cty!

8.5.2.2 Ordering Bar Graph

```
ggplot(mpg) +
  geom_bar(aes(class))
```



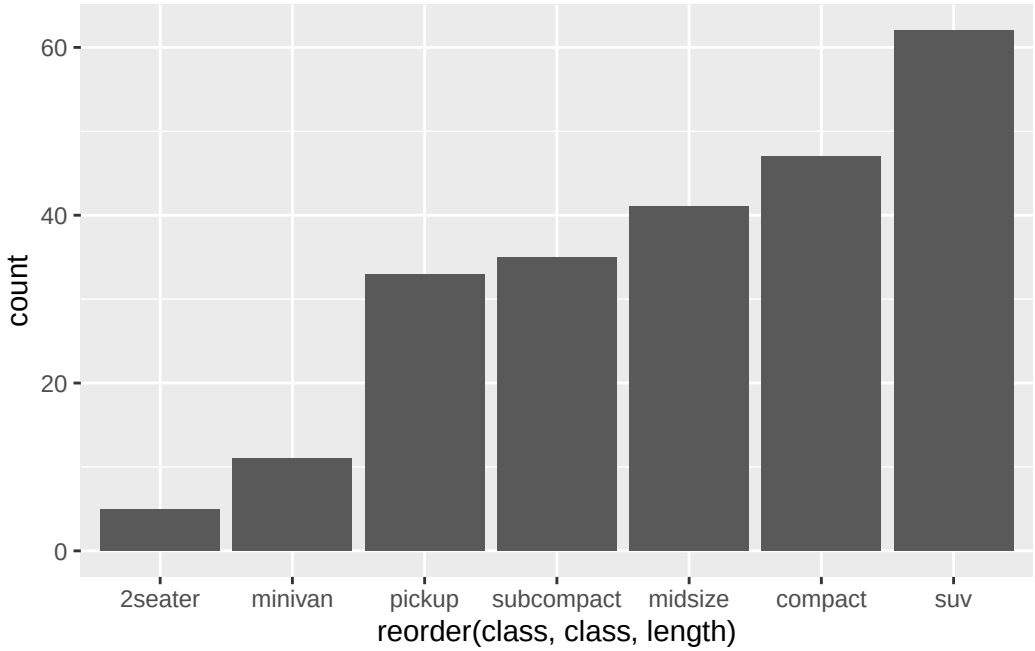
That was bar graph we made earlier. Again, categories are not ordered based on Y values and it's difficult to understand. So, let's order them. But they're categorical variable, and we can't compare text to text and say who's bigger. We can't even use fct_reorder() because Y axis are just count rather than values stored in the column. In this case, there are two ways to do it.

First way: Using basic R function, reorder(): Do you remember reorder() we used in Chapter 6 while talking about factor and levels? Each factors (data frame is still a list, so mpg and class still have levels and factors) have legnths (if there's 50 suv, length of suv is 50). So, let's try using reorder(). Here, there were three parameters.

```
reorder(Target, Data, Function)

Target: What category are we looking at?
Data: What data contains data we need to get numbers for function?
Function: What standard are we using for ordering?
```

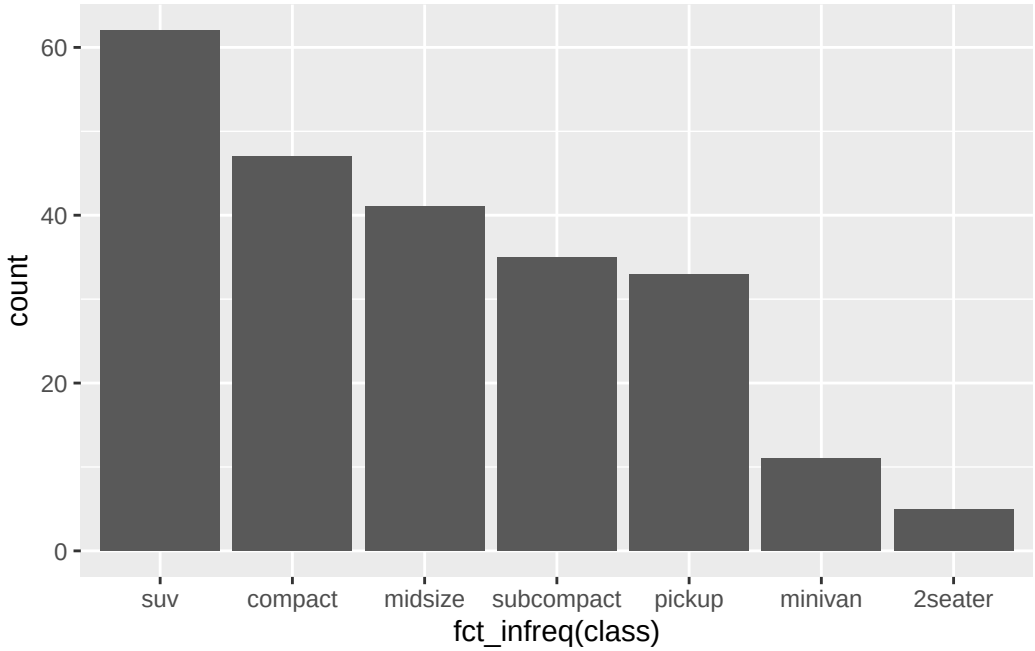
In this case, we are trying to order based on the count (frequency) of each car class. So, target will be "class." And we need a count for each class of cars (since that's the number that decides ordering order), which we can find from "class" (by counting how many 'suv' are in class, for exmaple). Last, Function will be "length" (how many times class X was counted).

```
ggplot(mpg) +
  geom_bar(aes(x = reorder(class, class, length)))
```



Second way: using tidyverse function. Like fct_reorder() we mentioned earlier, we have a function for counting the number of frequency, fct_infreq().

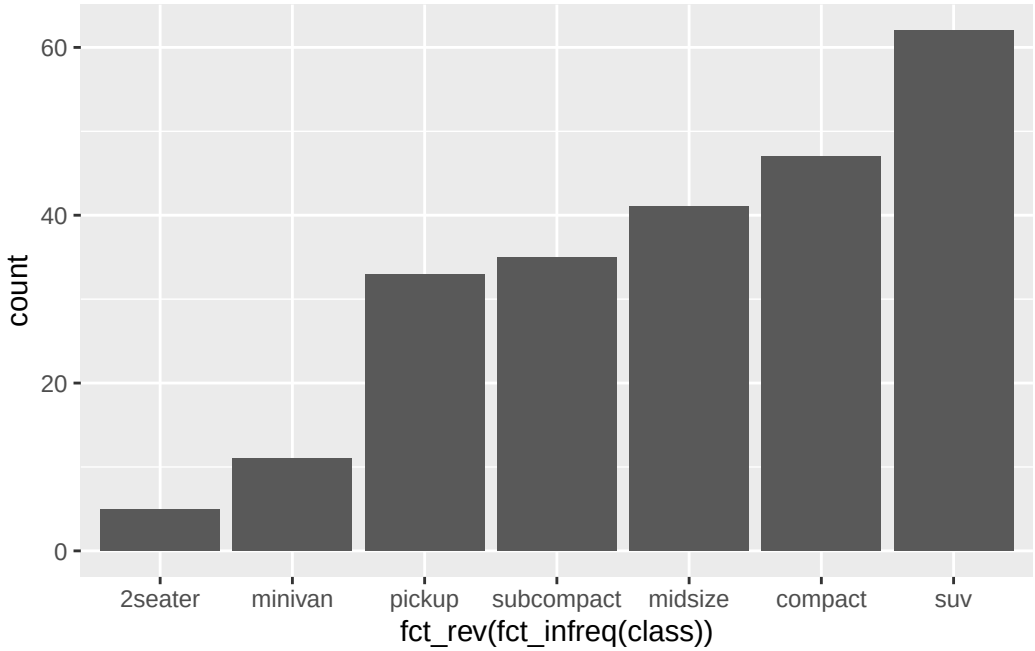
```
ggplot(mpg) +
  geom_bar(aes(x = fct_infreq(class)))
```




```
#fct_infreq(A) orders categorial data A based on the index (how many of them exist)
```

Either case, if you want to flip the order of presenting the data, you can use fct_rev().

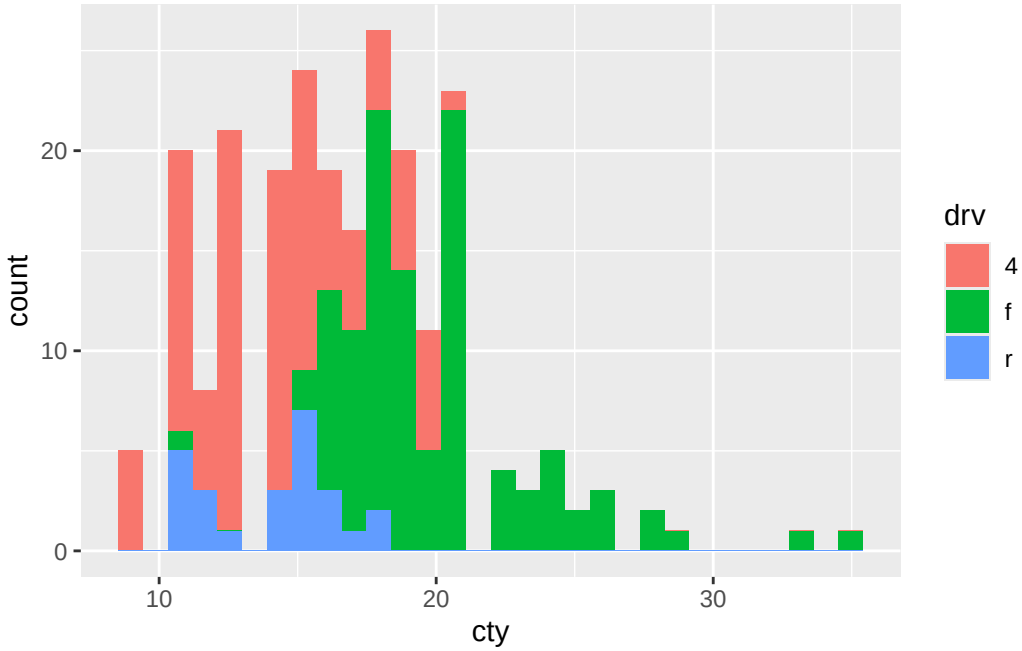
```
ggplot(mpg) +
  geom_bar(aes(x = fct_rev(fct_infreq(class))))
```



Sometimes, ordering bar graph based on the length isn't enough. Let's look at the example below.

```
ggplot(mpg) +
  geom_histogram(aes(cty, fill = drv))
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

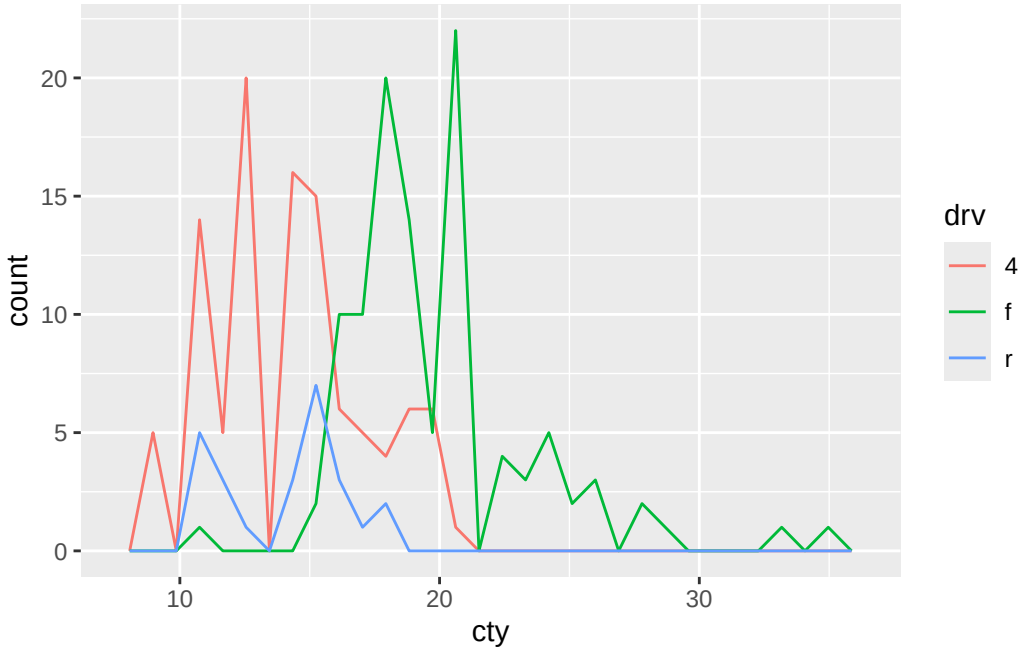


Even you want to use fct_infreq(), you cannot do that. It is because cty is a continuous numerical data, not a categorical data with distinct 'buckets' like 'class.' Also, you cannot see the relative size of each drv sub-types per bar because they are stacked to each other. This is where 'position = identity' becomes useful because everything starts from y = 0, making size comparison easier. However, there was a readability issue.

To address such disadvantage, we have new type of geom() called geom_freqpoly() and geom_density(). These draws line graphs instead of bars, which keeps identity mode while helping us reading the relative size for continuous data.

```
ggplot(mpg) +
  geom_freqpoly(aes(cty, color = drv))
```

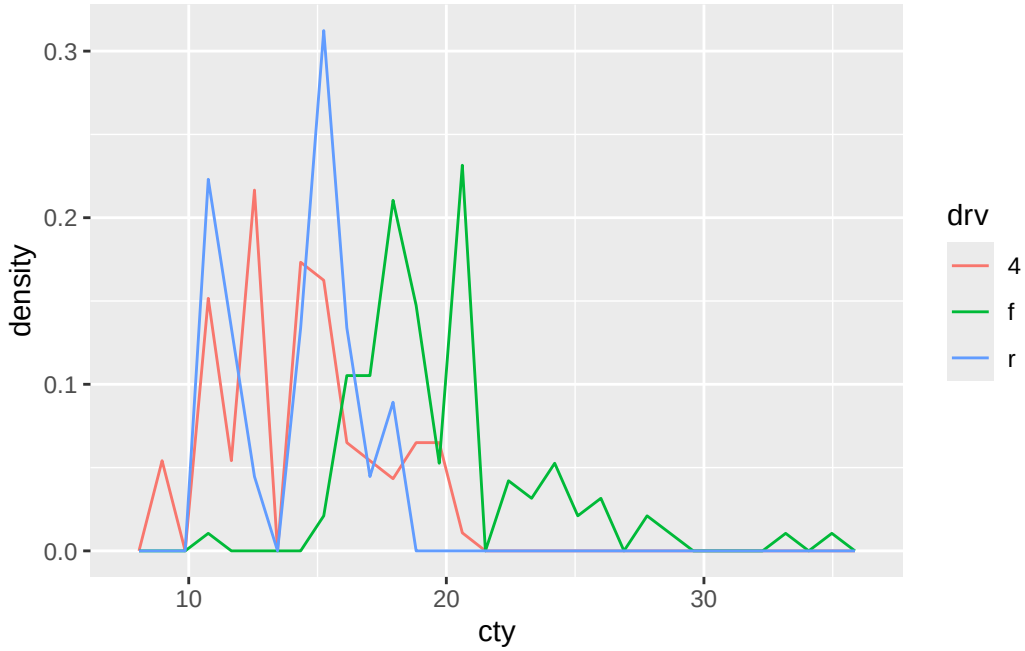
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



While we can compare count per cty bins, it is still difficult to tell "f is larger than r by how much" because they're in absolute count. Again, we can use after_stat(density) in the y-axis to compare relative size of each.

```
ggplot(mpg) +
  geom_freqpoly(aes(cty, after_stat(density), color = drv))
```

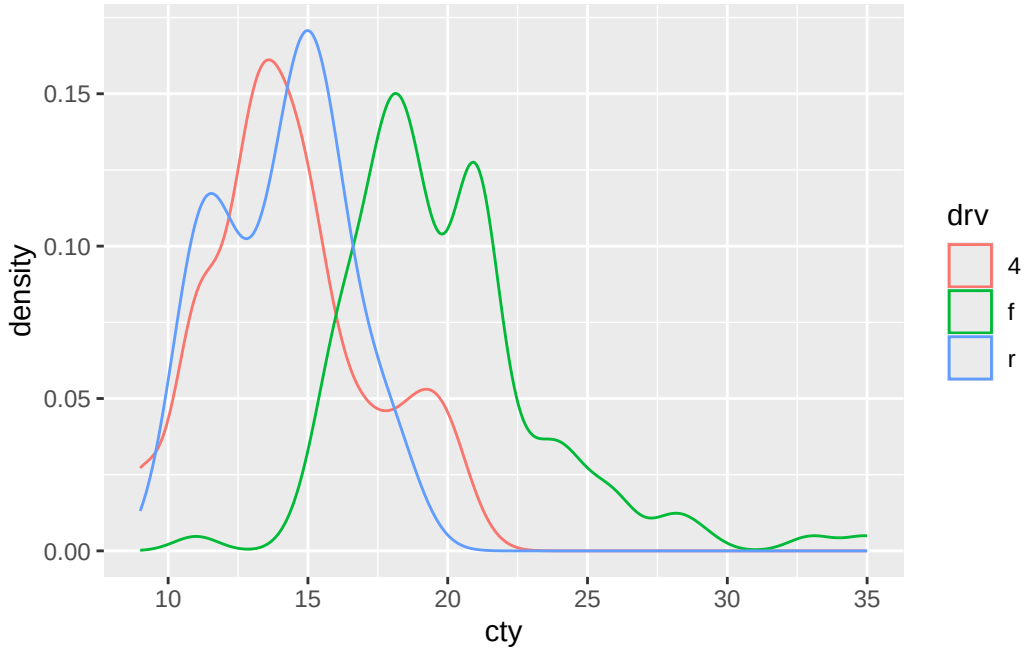
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



But there is still one consideration remaining: histogram and fre_poly basis their counting on "how wide each bins are." If each bin is 100000 wide, there will be only 1 bar showing. If it's 0.000001, data will be really detailed.

So, geom_density() shows us a probability density (it's like estimating density at every single values of x-axis) to remove this binwidth issue.

```
ggplot(mpg) +
  geom_density(aes(cty, color = drv))
```

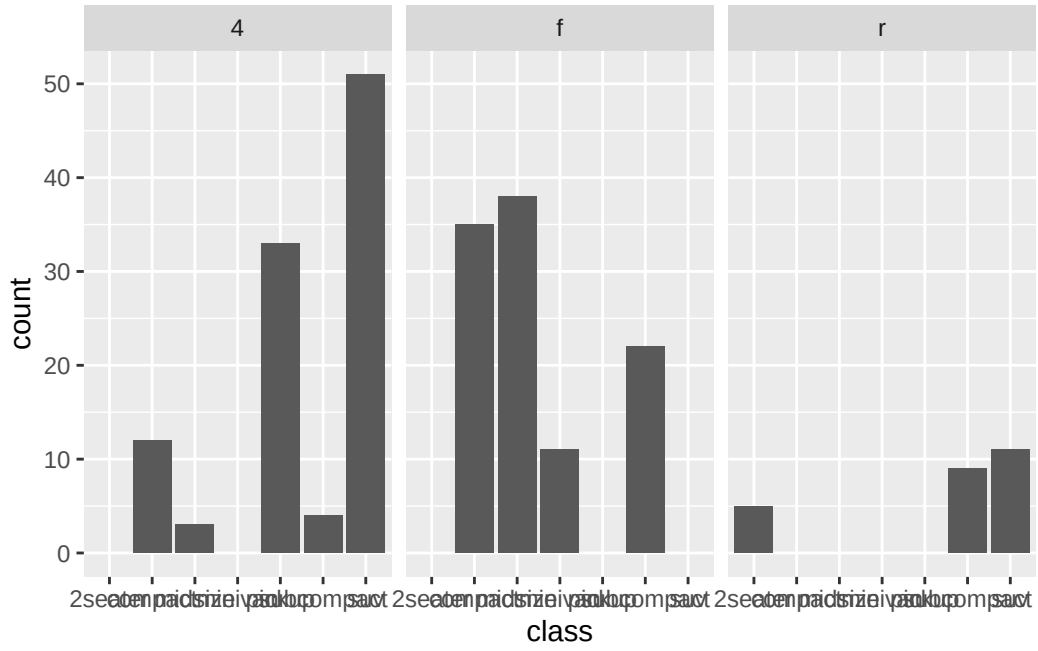


8.5.2.3 geom_jitter()

Sometimes, you want to explain the relationship of two different variables. For example, "How many suv cars are forward wheel drive? how about four wheel drive?" To answer this, there are two ways to do it.

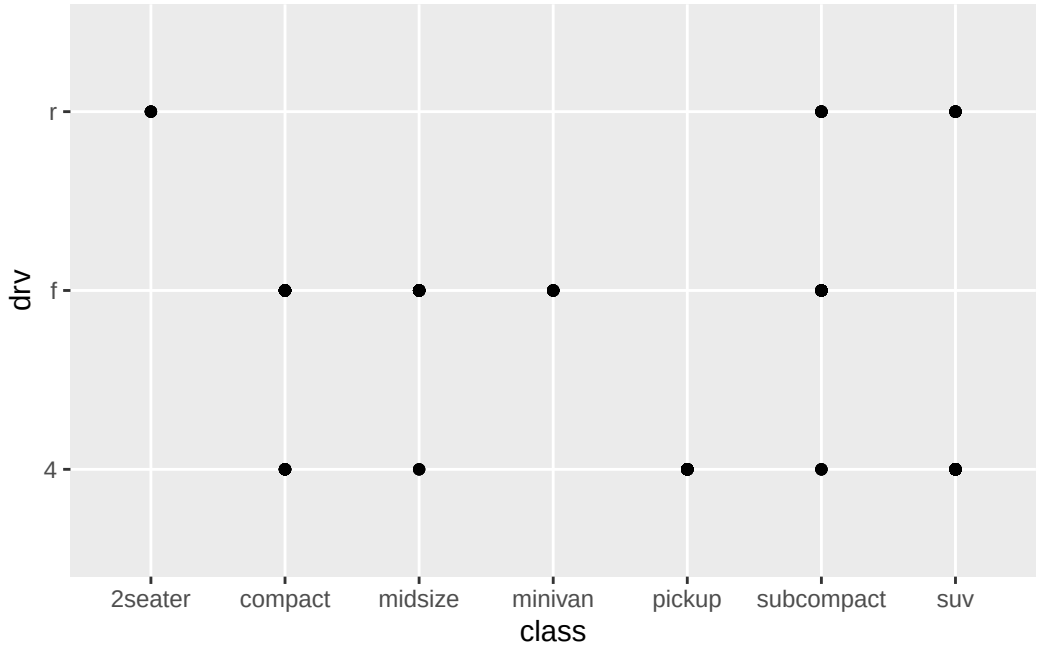
First, is to use facets. We can draw bar graph of count vs. class per drv.

```
ggplot(mpg, aes(class)) +
  geom_bar() +
  facet_wrap(~drv)
```



Second, is to draw a scatter plot with drv as y axis and class as x axis.

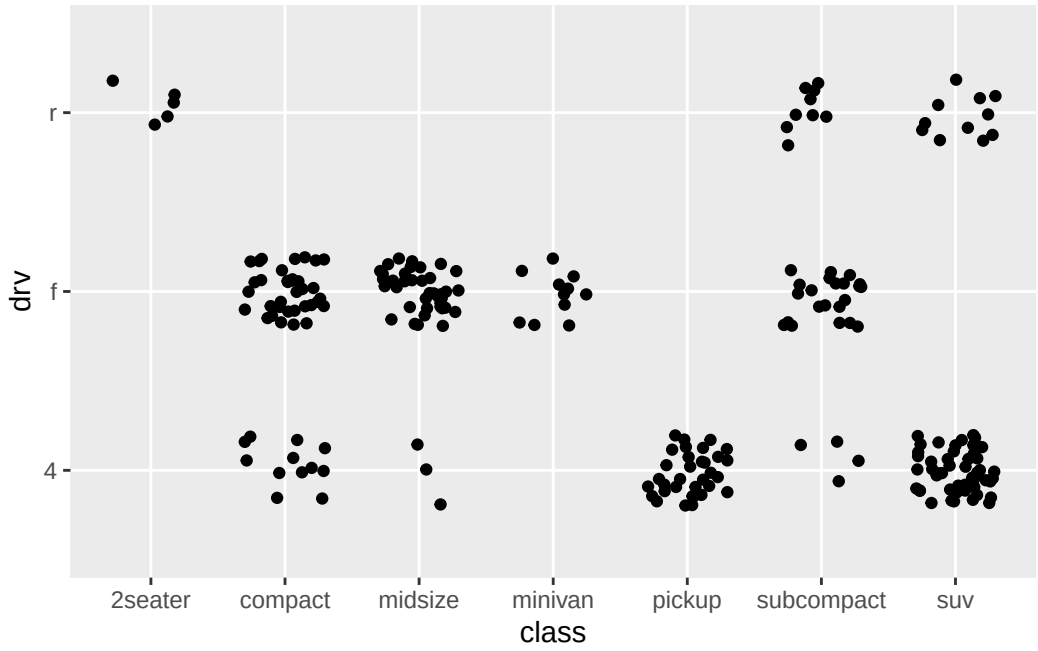
```
ggplot(mpg, aes(class, drv)) +
  geom_point()
```



Here, we can see the co-existence of two features. Minivan only has forward drv, while 2seaters only have rear drv. However, there is one big limitation: Because we are drawing scatter plot of two categorial variables, options are limited: All forward drv minivan will exactly overlap onto each other. This hurts readability as we cannot tell how many cars actually belongs to each dots.

To resolve this issue, we have geom_jitter(), which spreads data points randomly with given distance.

```
ggplot(mpg, aes(class, drv)) +
  geom_jitter(width = 0.3, height = 0.2) # you need to define spreadness of both direction
```

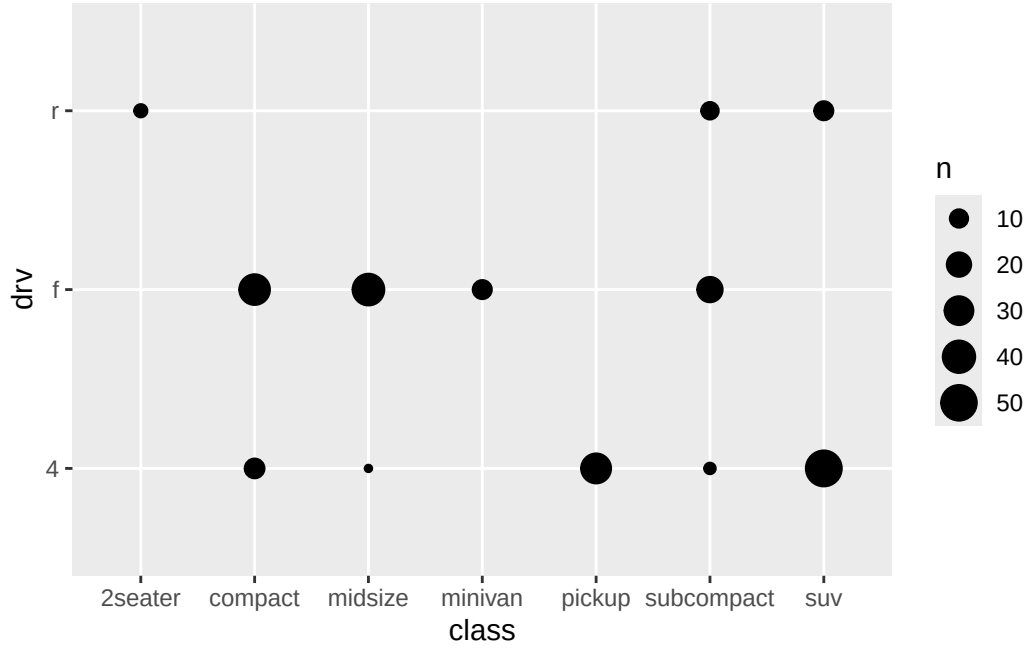


Now, you can see how many data points were overlapping in same spot! But be careful, points will start intruding another data point if you set jitter to wide. For example, if you set width = 10, you might find dot that supposed be on (compact, 4) on (midsize, 4).

8.5.2.4 geom_count()

If your goal is just to see relative numbers, jitter() might be difficult as you need to count how many points there are. In this case, we can use geom_count(), which scales with number of overlapping points.

```
ggplot(mpg, aes(class, drv)) +
  geom_count()
```

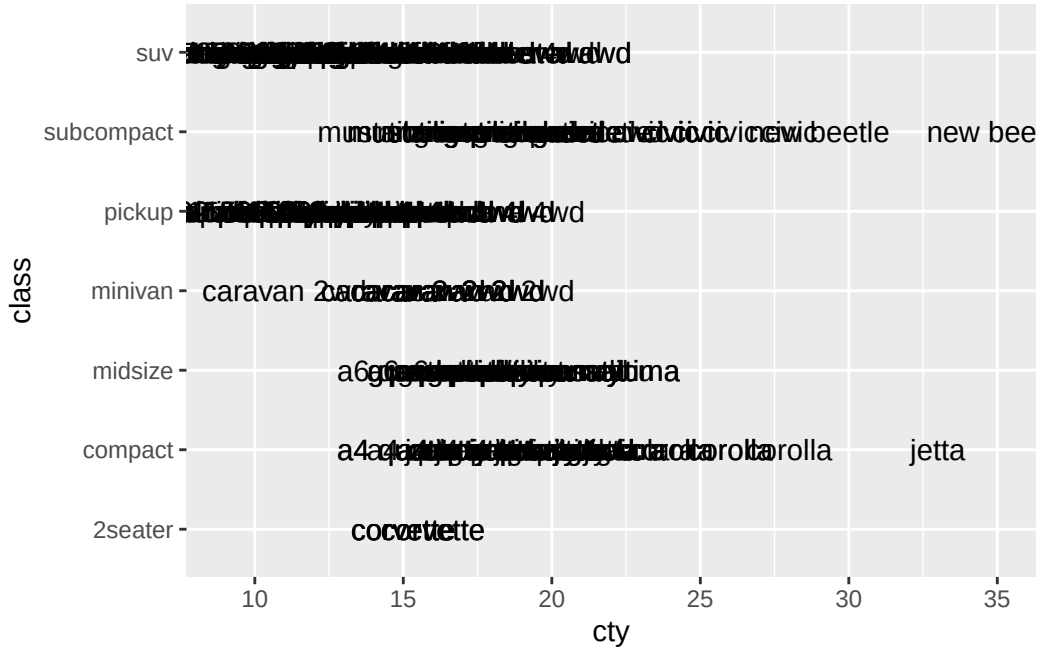


8.5.2.5 geom_text()

While we look at the color-based differentiation, have you thought "it's difficult to tell by color. I want to see the car class"?

You can display text using geom_text()!

```
ggplot() +
  geom_text(mpg, mapping = aes(cty, class, label = model))
```

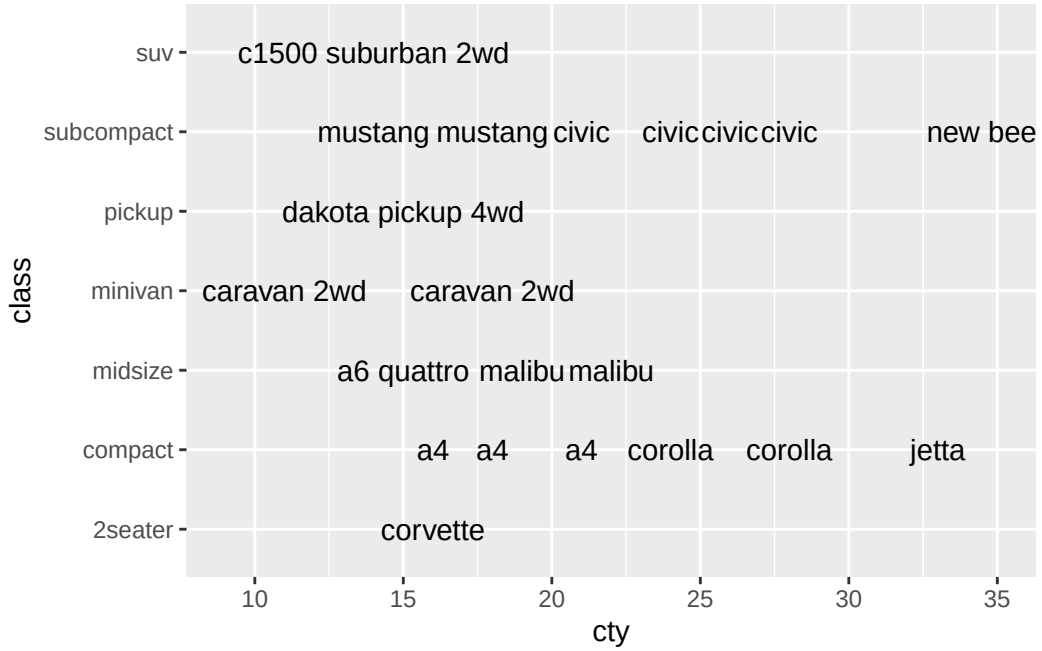


In geom_text, you must state what 'text' you want to display using 'label =' argument within aes() (besides defining what x and y axis of the plot will be!)

However, do you see how wordy the plot is? It's because so many points are overlapping that test becomes blurry. To resolve this, there are two ways:

First, you can use 'position = position_jitter(A,B)' argument in geom_text() to spread out. Second, in case there are so many overlapping points that jitter() cannot help you, you can use check_overlap() argument. In this case, if one text appears in a location, same text will not appear again in the similar spot so that you don't get much overlaps.

```
ggplot() +
  geom_text(mpg, mapping = aes(cty, class, label = model), check_overlap = T)
```

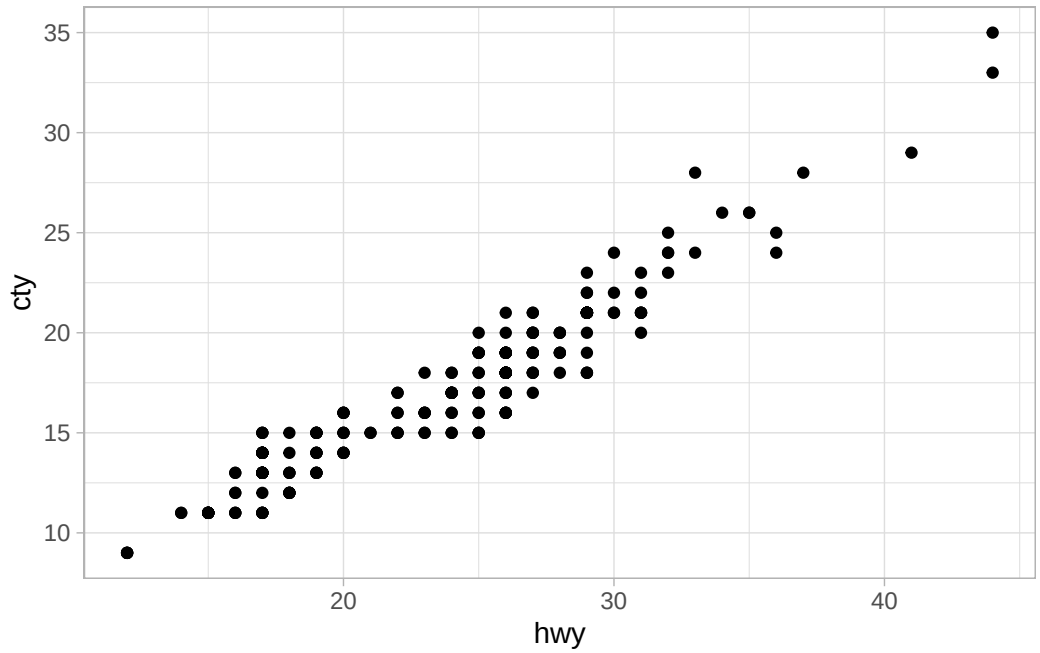


8.6 Theme

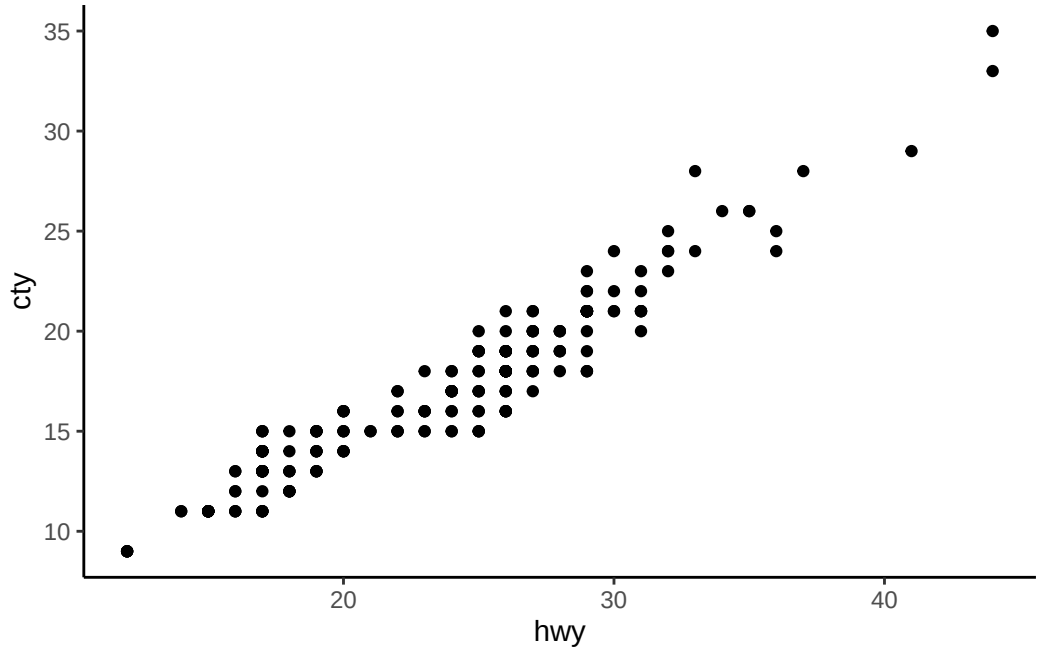
So far you might got bored with boring looking basic plot setting. theme() layer allows you to use different visuals of the plot! There are many of them, so google them and see if you like anything.

Here is example of how you can use it. Default is theme_gray() if you don't add theme() layer.

```
p <- ggplot(mpg) +
  geom_point(aes(hwy, cty))

p + theme_light()
```




```
p + theme_classic()
```

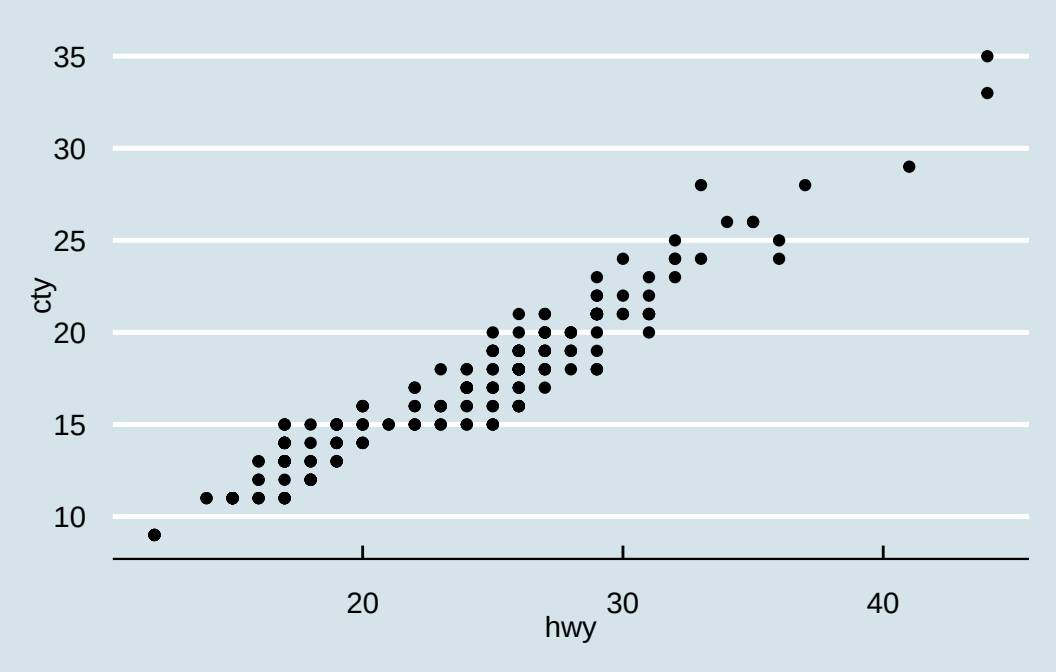


While ggplot() itself provide many themes, you can look for custom themes by installing package.

```
install.packages("ggthemes") #run it on your console
```

```
library(ggthemes)
```

```
p + theme_economist()
```



8.7 Coordinates()

Last layer of the ggplot()! In coordinates(), we set features for axis.

If the data is continuous, we use

```
scale_x_continous() # or y_continuous
```

If the data is categorical, we use

```
scale_x_discrete() # or y_discrete()
```

```
<ggproto object: Class ScaleDiscretePosition, ScaleDiscrete, Scale, gg>
    aesthetics: x xmin xmax xend
    axis_order: function
    break_info: function
    break_positions: function
    breaks: waiver
    call: call
    clone: function
    dimension: function
    drop: TRUE
    expand: waiver
    get_breaks: function
    get_breaks_minor: function
    get_labels: function
    get_limits: function
    get_transformation: function
    guide: waiver
    is_discrete: function
    is_empty: function
    labels: waiver
    limits: NULL
    make_sec_title: function
    make_title: function
    map: function
    map_df: function
    n.breaks.cache: NULL
    na.translate: TRUE
    na.value: NA
    name: waiver
    palette: function
    palette.cache: NULL
    position: bottom
    range: environment
    range_c: environment
    rescale: function
    reset: function
    train: function
    train_df: function
    transform: function
    transform_df: function
    super:  <ggproto object: Class ScaleDiscretePosition, ScaleDiscrete, Scale, gg>
```

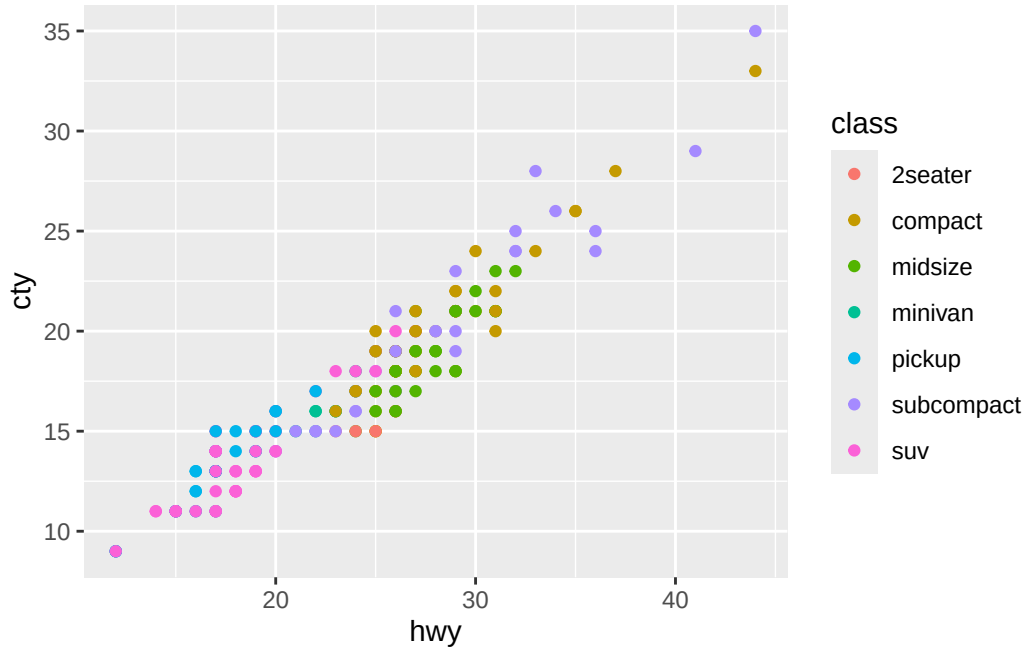
Arguments for coordinates() are following:

- name: Title of axis
- breaks: Location of major grid lines
- minor_breaks: Location of minor grid lines
- labels: Labels on the grid
- limits: Window/Range of axis
- trans: Applying transformations (log, exponential, etc) to axis
- position: Location of axis display
- coord(): Type of coordinate system

Let's review each of them.

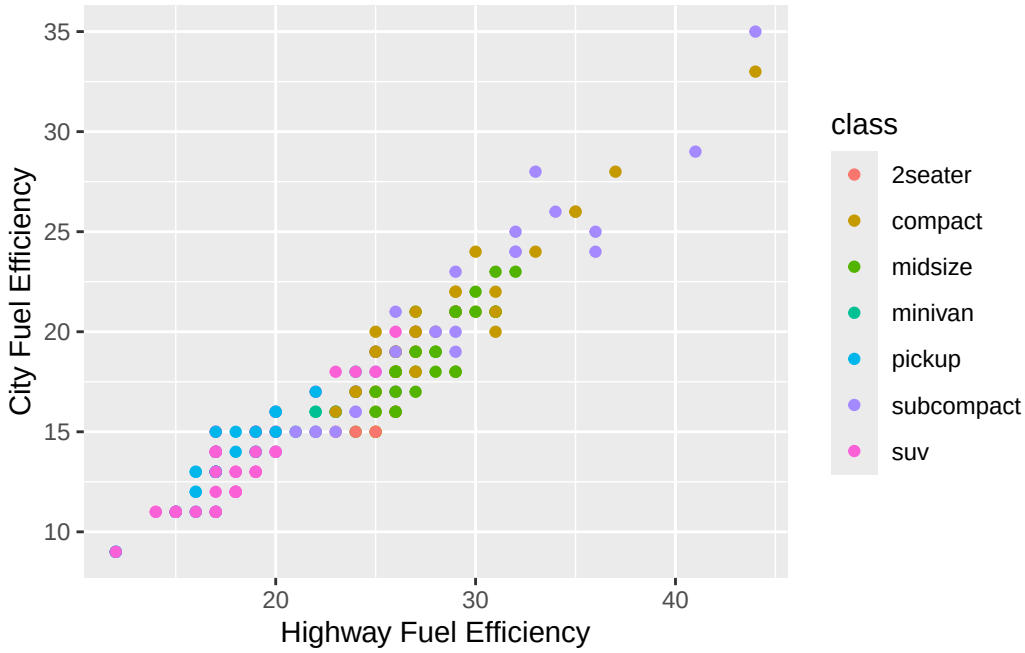
8.7.1 name()

```
p5 <- ggplot() +
  geom_point(mpg, mapping = aes(x = hwy, y = cty, color = class))

p5
```



For readers, axis titles are not clear. So, let's rename them.

```
p6 <- p5 +
  scale_x_continuous(name = "Highway Fuel Efficiency") +
  scale_y_continuous(name = "City Fuel Efficiency")

p6
```



8.7.2 breaks(), minor_breaks()

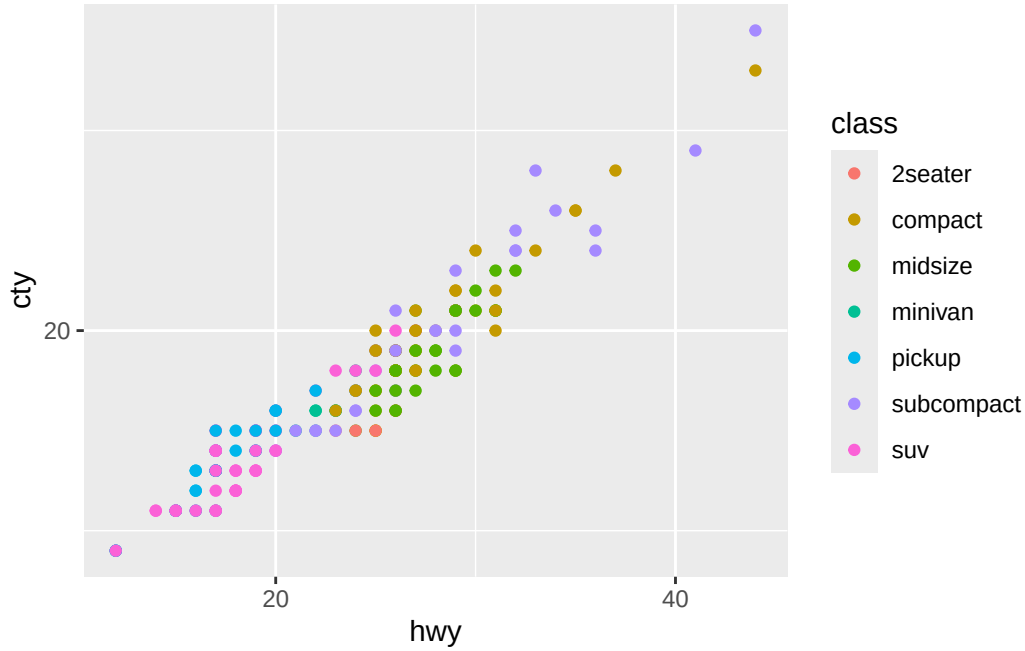
breaks() is distance of major grid lines.

```
p7 <- p6 +
  scale_x_continuous(breaks = seq(0, 40, by = 20),
                     minor_breaks = seq(0, 40, by = 10)
                     ) + # If you don't remember seq(), review Chapter 2 or 3!
  scale_y_continuous(breaks = seq(0, 40, by = 20),
                     minor_breaks = seq(0, 40, by = 10)
                     )
```

```
Scale for x is already present.
```

```
Adding another scale for x, which will replace the existing scale.
Scale for y is already present.
Adding another scale for y, which will replace the existing scale.
```

```
p7
```



Do you see how now major breaks (grid line with labels) are every 20, while minor breaks (gridline without label) are by 10?

8.7.3 trans

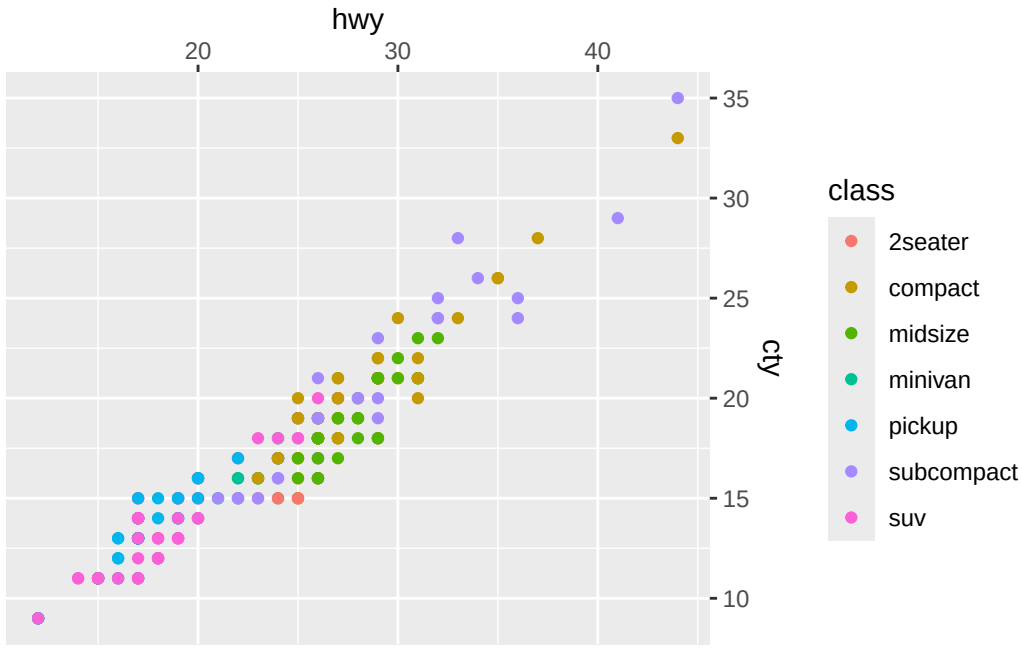
Let's skip limits since we already talked about it. For trans, it's operations that changes axis. Right now they're plain numbers. However, you can use (trans = "log10") to change break's label in logmathic scale. This becomes useful when all data points are so skewed (e.g., most of data are $< 10\hat{}6$ while some are $> 10\hat{}6$) that all data looks clustered.

8.7.4 position

Position changes where labels are displayed.

```
p7 + scale_x_continuous(position="top") +
  scale_y_continuous(position="right")
```

```
Scale for x is already present.
Adding another scale for x, which will replace the existing scale.
Scale for y is already present.
Adding another scale for y, which will replace the existing scale.
```

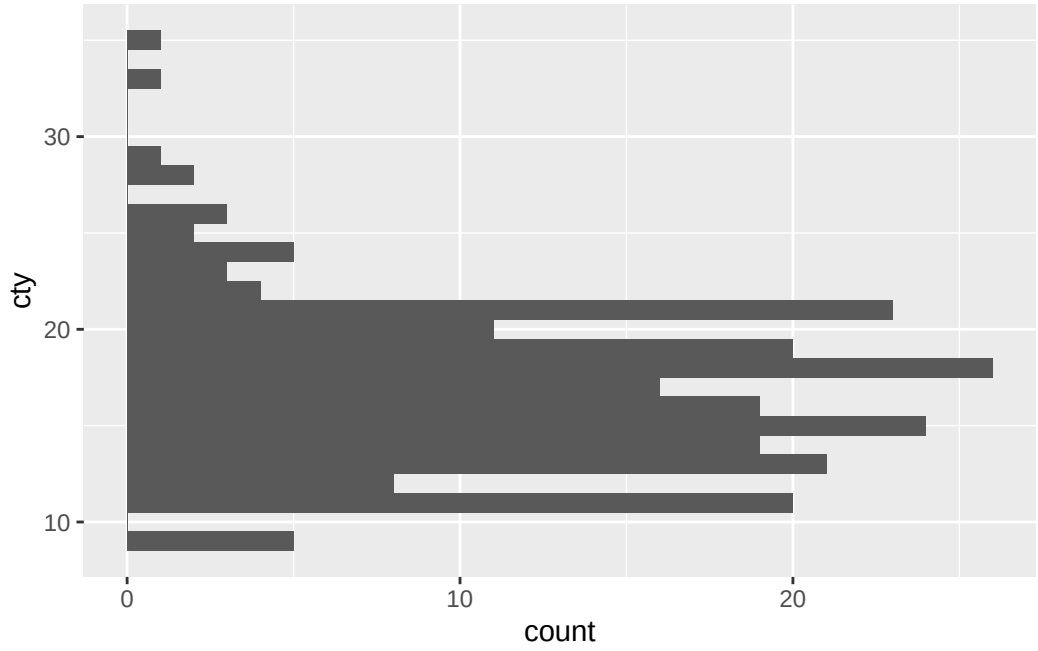


Now, scales are displayed at top right!

8.7.5 coord()

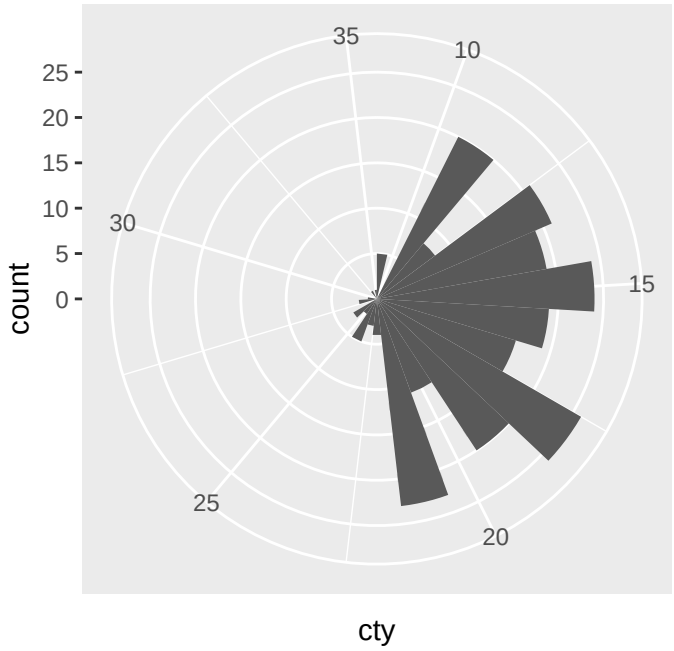
Sometimes, you want to flip x and y axis. Specifically in histograms for readability. In that case, we can use coord_flip()

```
p8 <- ggplot(mpg, aes(cty)) +
  geom_histogram(binwidth = 1)

p8 + coord_flip()
```



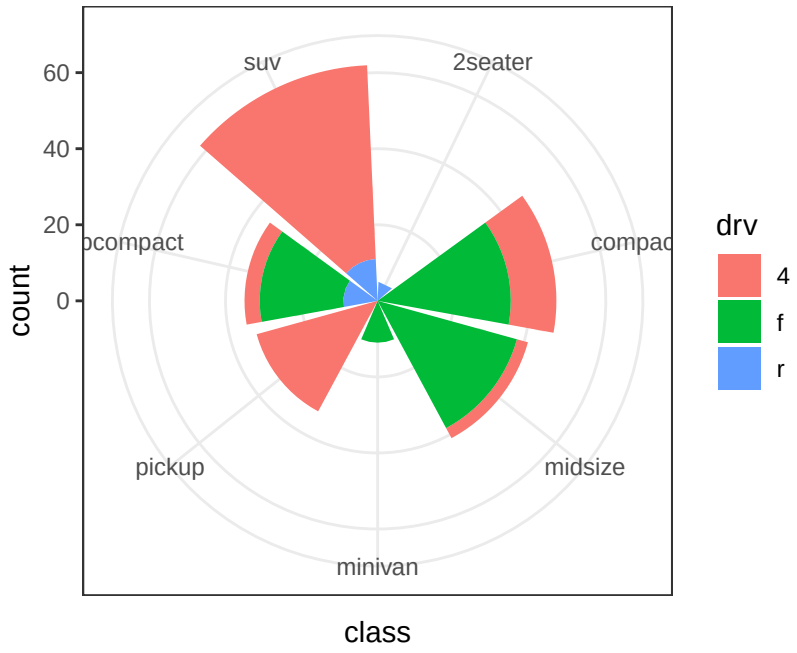
Or, you can change regular coordinates into polar coordinate.

```
p8 + coord_polar()
```



Polar coordinates are handy when you're trying to create pie chart!

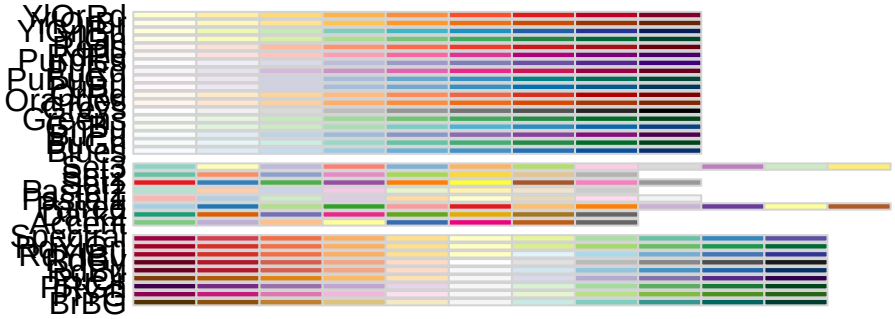
```
ggplot(mpg) +
  aes(class, fill = drv) +
  geom_bar() +
  coord_polar() +
  theme_bw()
```



8.8 Colors

Last, in case you wonder how to change scale of colors within aes(). We can manually set colors in case we don't like default setting of ggplot() using RColorBrewer prackage.

```
RColorBrewer::display.brewer.all()
```



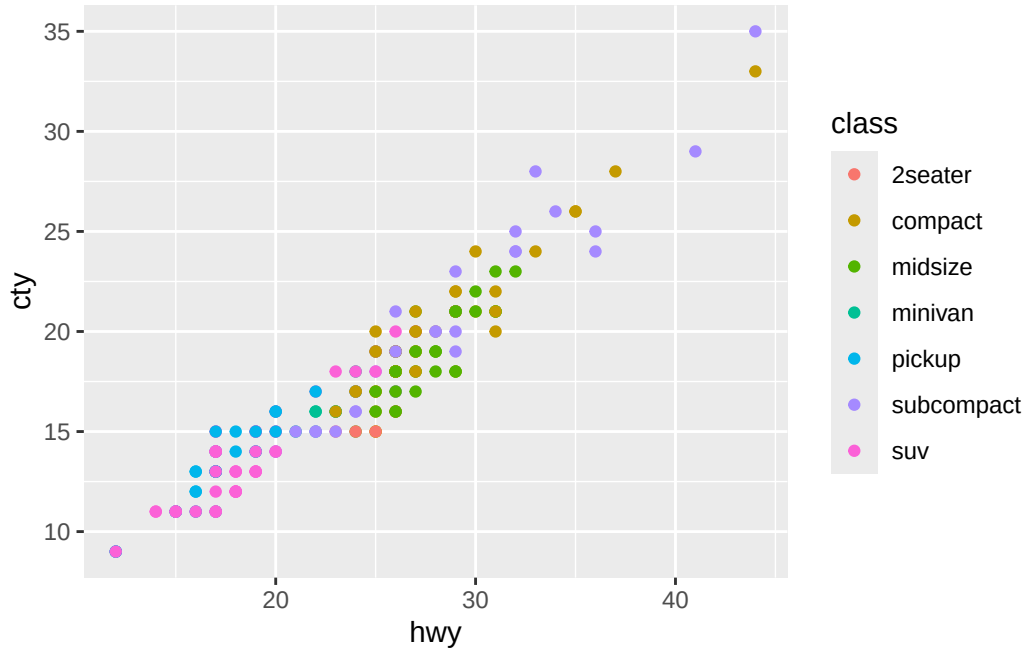
```
RColorBrewer::brewer.pal.info
```

```
         maxcolors category colorblind
BrBG            11      div       TRUE
PiYG            11      div       TRUE
PRGn            11      div       TRUE
PuOr            11      div       TRUE
RdBu            11      div       TRUE
RdGy            11      div      FALSE
RdYlBu          11      div       TRUE
RdYlGn          11      div      FALSE
Spectral        11      div      FALSE
Accent           8     qual      FALSE
Dark2            8     qual       TRUE
Paired          12     qual       TRUE
Pastel1          9     qual      FALSE
Pastel2          8     qual      FALSE
Set1             9     qual      FALSE
Set2             8     qual       TRUE
Set3            12     qual      FALSE
Blues            9      seq       TRUE
BuGn             9      seq       TRUE
```

```
BuPu            9       seq         TRUE
GnBu            9       seq         TRUE
Greens          9       seq         TRUE
Greys           9       seq         TRUE
Oranges         9       seq         TRUE
OrRd            9       seq         TRUE
PuBu            9       seq         TRUE
PuBuGn          9       seq         TRUE
PuRd            9       seq         TRUE
Purples         9       seq         TRUE
RdPu            9       seq         TRUE
Reds            9       seq         TRUE
YlGn            9       seq         TRUE
YlGnBu          9       seq         TRUE
YlOrBr          9       seq         TRUE
YlOrRd          9       seq         TRUE
```

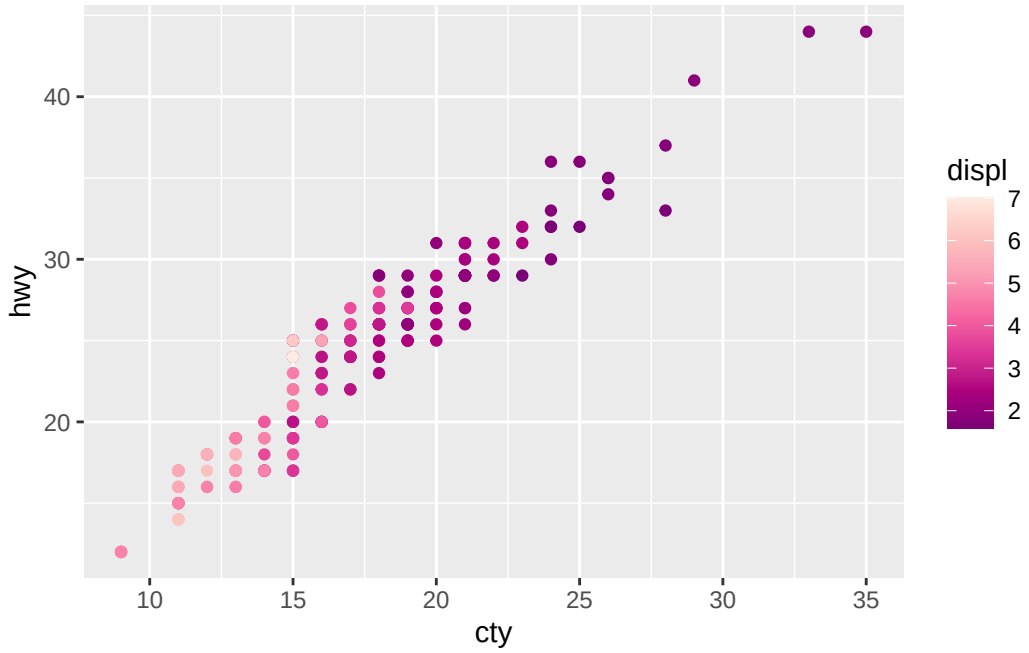
If you want to use one of them, you can set it using scale_color_brewer() layer (continuous) or scale_fill_brewer() (categorical). Be careful, they're case sensitive.

```
p5 + scale_fill_brewer(palette = "Set1")
```



However, you still see each classes are assigned with complete different color. If you want them to be natural gradient, you can use scale_color_distiller() (continuous) or scale_fill_distiller() (categorical).

```
ggplot(mpg, aes(cty, hwy, color = displ)) +
  geom_point() +
  scale_color_distiller(palette = "RdPu")
```



8.9 Summary

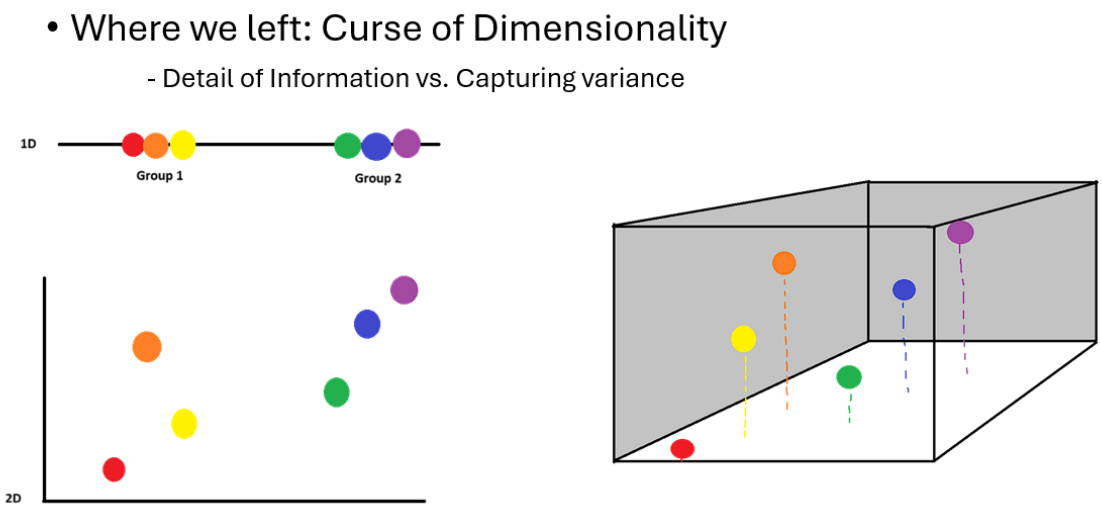
Today, we reviewed a lot on how to visualize the data into figures. If you want to save your figure, you can run the cell, right click, and hit "save as~"

Good job on following through Chapter 1 ~ 8. From next chapter, we will start applying conceptual data analysis methods we talked about in the meetings. Once you're done with Chapter 8, please email me so that I can upload Chapter 9 and afterwards.

9 Dimension Reduction

9.1 Why do we need it?

In the meeting, we talked about curse of dimensional.



Let's load example data frame to recap what it was. Download it using this link: Link

Just like we did last chapter, let's load tidyverse and read csv.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.2.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
vitals <- read.csv("Vitals.csv")
head(vitals)
```

```
  X Patient_ID Age Diabetes_Status Temp_C Heart_Rate Pulse_Machine_BPM
1 1          1  60             Yes   37.6        101               104
2 2          2  33             Yes   37.5         81                85
3 3          3  33              No   36.5         82                80
4 4          4  44             Yes   37.9         84                82
5 5          5  81              No   36.7         70                68
6 6          6  82              No   37.1         87                88
  Systolic_BP Diastolic_BP   MAP Resp_Rate SpO2 Shock_Index
1         109           67  81.0        10   96        0.93
2         117           90  99.0        17   94        0.69
3         117           74  88.3        20   98        0.70
4         132           86 101.3        22   94        0.64
5         116           81  92.7        12   96        0.60
6         112           77  88.7        13   96        0.78
```

```
vitals <- vitals[, -1] #What does this do? Why do we do this? If you forgot, revisit chapter
head(vitals)
```

```
  Patient_ID Age Diabetes_Status Temp_C Heart_Rate Pulse_Machine_BPM
1          1  60             Yes   37.6        101               104
2          2  33             Yes   37.5         81                85
3          3  33              No   36.5         82                80
4          4  44             Yes   37.9         84                82
5          5  81              No   36.7         70                68
6          6  82              No   37.1         87                88
  Systolic_BP Diastolic_BP   MAP Resp_Rate SpO2 Shock_Index
1         109           67  81.0        10   96        0.93
2         117           90  99.0        17   94        0.69
3         117           74  88.3        20   98        0.70
4         132           86 101.3        22   94        0.64
5         116           81  92.7        12   96        0.60
6         112           77  88.7        13   96        0.78
```

Here, you see lots of features. Two things:

- MAP (Mean Arterial Pressure): (SBP + 2 * DBP) / 3. Indicates organ perfusion.

- Shock index: HR / SBP. Assess hemodynamic stability.

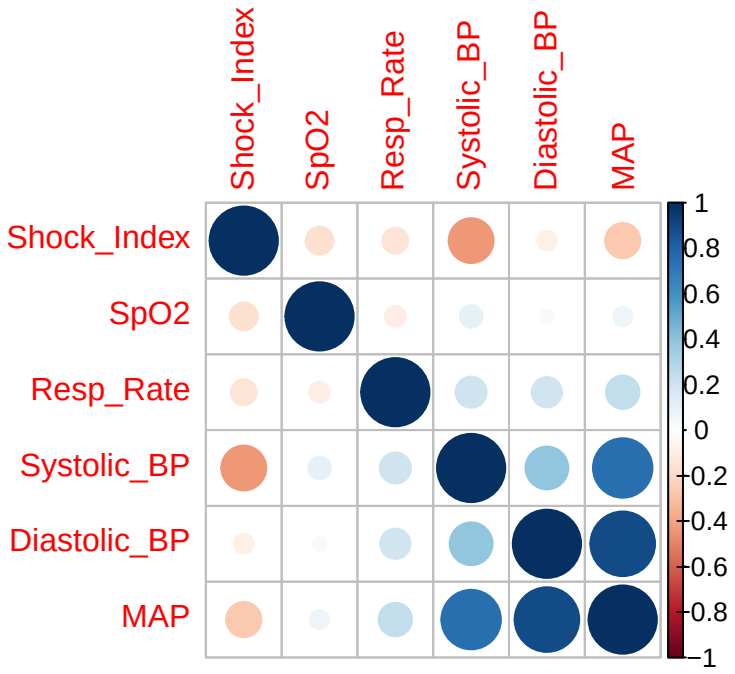
These variables, which are calculated from existing variables, are called derived variables. In data analysis, we often use derived variables as they can be meaningful while reducing number of variables. For example, height or weight itself cannot tell whether individual is obese. You might think, "isn't patient obese if they are 200kg?" What if he is 2m tall and that his weight is within normal range for his height? However, if we combine height and weight into BMI, that can indicate whether someone is obese or not.

But then, is having multiple variables good thing since they have significance? Let's figure it out. First, let's check correlation among variables. For sake of readability, let's only use last 5 variables.

```
library(corrplot) #This package helpes us calculating and plotting correlations
```

```
corrplot 0.95 loaded
```

```
cor(vitals[, 7:12]) %>% #calculate corrlation (cor) and pass the result to (%>%)
  corrplot(order = "hclust") #plot the correlation
```



Correlation score closer to 1 (darker blue, bigger circle) means two variables are positively and linearly related, and score closer to -1 means they are negatively and linearly related.

If we try to score patient's health using those 5 variables, what would happen? Even thought all blood pressure , oxygen saturation, and respiratory rate are important, score will be dictated by blood pressure because more than half of variables are from identical source.

That said, what were some consequence of high dimensional data? There are three major downsides:

1. Waste of resource: Because there are more data to process and calculate, model suffers from longer calculation time. As result, more human and time efforts are wasted for meangingless data
2. Overlapping variables: Like example we saw above, the more variables there are, the higher probability of having multiple features with high-correlation or even sharing identical source. As result, you artifically hide actually meaningful features reflected in the analysis.
3. Overfitting: As you include more features, there is probability that you might include features that are irrelevant to the analysis topic. For example, let's say your analyzing gene of cancer patient. There are limited numbers of genes that cause cancer. However, if you analyze against entire human gene, something weird can happen. Because everyone contains gene that suppress cancer, your analysis will conclude that 'cancer suppress gene is causing cancer,' even though supressor genes are out of list of 'cancer-causing genes.'

That said, condensing data set with really meaningful and non-repetitive features becomes important. And that's, why we need dimension reduction.

9.2 PCA (Principal Components Analysis)

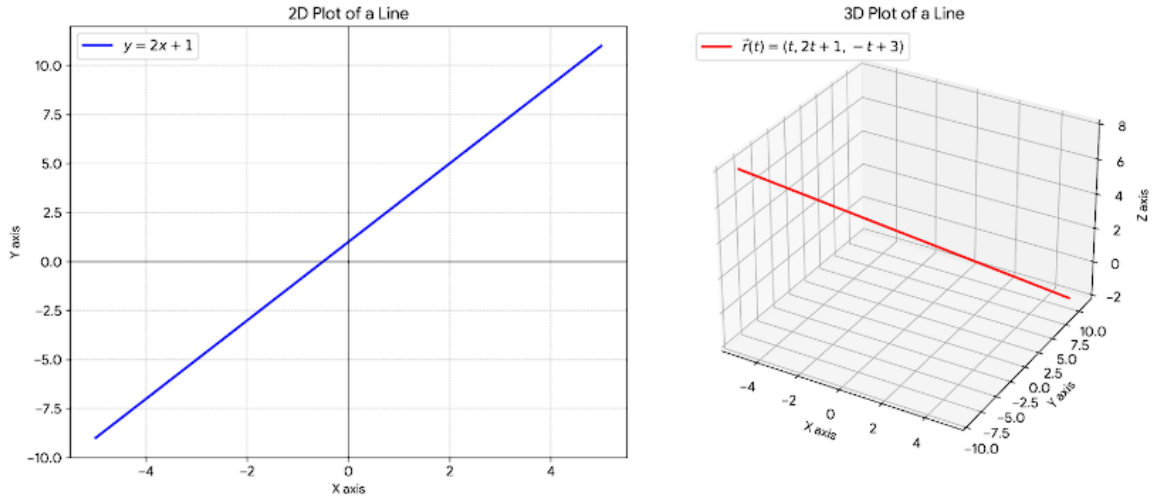
9.2.1 Concept of PCA

During the meeting, we said PCA is like finding a perfect shadow (line that captures most variance). Since we reviewed visualized approach, let's talk little bit about the concept in mathematical aspect. First, let's look at what those 'lines' actually means, and how we maximize variance of data points.

For each features, there will be one axis that explains the feature. Think like axis on the plot.

1 feature: line/one axis (e.g., you can plot severity of pain on a line with scale of 0 to 10)
2 features: plane, which has two axis (e.g., you can plot relationship between SBP and DBP on 2d plot)
3 features: 3d cube, which has three axis
and more…

Then, all lines on a space will be combination of each axis, like figure below.



* If you haven't learn vector equation, r(t) is something like this:

As demension increases, you cannot draw a line using "y = ~" form. For example, draw z = 2x + y on the desmos. It's a plane! So to draw a line, we need to define exact (x,y,z,…) coordinate for all points existing on the line. For example, in y = 2x + 1, we can say all possible combination of (x,y) on that line is (t, 2t + 1), where t is random number. So, instead of y(x) = 2x + 1, we can express it using r(t) = <t, 2t + 1>.

When we draw a line using combination of existing axis, we call that 'linear combination.' For example, line ax + by + cz.

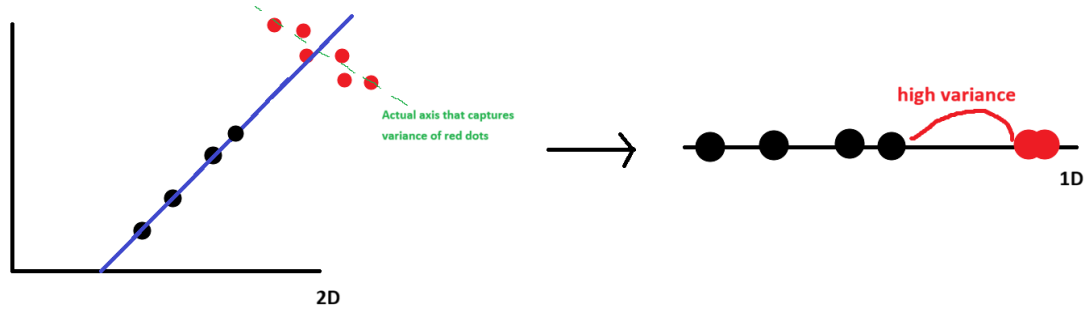
Now, let's look at equation of principal component with q number of features.

$$Z_k = a_{1k}(X_1 - \bar{X}_1) + a_{2k}(X_2 - \bar{X}_2) + \cdots + a_{pk}(X_p - \bar{X}_p)$$

- Zk is a principal component (PC) made from linear combination of features, where k = 1, 2, 3,… q. Why k can't be larger than q? It's because we only have q features we can use to combine! You can't come up with combination using 3 features in 2d plot.
- ajk is a loading/weight of each feature. For example, in y = ax, y will be horizontal line if a = 0, diagonal is a = 1, and vertical if a becomes infinitely large. So ajk means "How well kth PC aligns with jth feature?" If a1k = 1.0, it means first PC perfectly aligns with first feature/axis, and if a1k = 0.0, it means first PC doesn't align with first feature at all.
- Last part, X1 is feature 1, and $\bar{X}_1$ is average of feature 1 (e.g., in the corr plot we saw above, it would be all average values of MAP values). You might got confused: we just said linear combination is 'a(feature1) + b(feature2)…' But why suddenly $(X_1 - \bar{X}_1)$ instead of feature itself (X1)?

PC is a axis/line that captures most 'variance.' Then, how do we calculate the spread of points within the feature? We can't compare distance of all point pairs in the feature. Instead, we look for individual point's distance from their average. So, we pick a line that goes through the mean of feature1, and find a slope to tune the spread - we have PC that works for feature 1. But, there are multiple features in the data set (that's why we're performing PCA, right?)! This means that not only feature 1, but also rest of the feature's variance should be maximized through PC.

However, here is the trap: is variance of feature 2/3/… shown when they are plotted onto PC actually reflects the spread of the data points (distance from feature 2/3/…'s mean) or is it because of distance from points to PC (distance from PC)? Figure below visualize this issue:

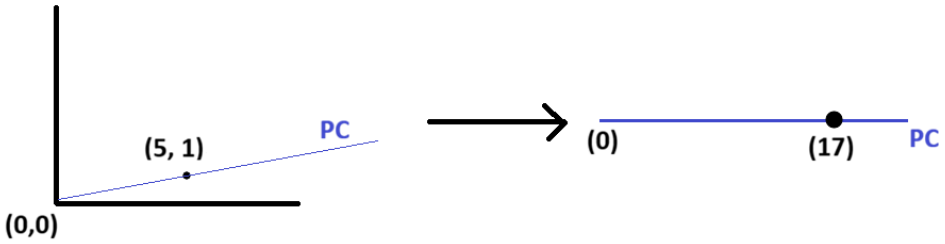


Here, PC (blue line) captures variance of patient group 1 (black dots) reasonably well. However, it completely fails to capture the variance of the patient group 2 (red dots). However, PCA can't tell that. On the 1D plot, do you see there's a massive distance between two patient groups? Because mean of two groups are so far away, it looks like PC is capturing variance of all points well based on "high variance" region (fake variance) even though it fails separating group 2.

So, to reflect variance of all points accurately, we need to standardize the location of means across all features. How? by $(X_1 - \bar{X}_1)$! If average height is 1.67m, we can move every point by 1.67m to the left so that average seats on the origin. Now, we have a stable and accurate way to check PC's ability to capture variance, as PC goes through (0,0) where center/average of all data points are!

Now, we know how PC line Zk is created. Then what? If we select random weights for the equation, we can calculate new coordinate of a point along the Zk.

For example, in "Zk= 2x + 3y," 2 and 3 are weight, where as x and y are features. If raw 2D plot using x and y as axis have a first point (5, 1), Z1 = 17.



If we calculate Zk coordinate for all data points, we can calculate variance of points on the PC, which is (sum of square of Zk values) / (N - 1). Algorithm test all possible combinations of weight until it finds the highest variance value. Now, we know how PCA reduce dimension!

9.2.2 Running PCA

Executing PCA is done by function called prcomp() in stat package. You need to select

```
prcomp(analyzing data)
```

```
result <- prcomp(vitals[, c("Systolic_BP", "Diastolic_BP")]) # Running PCA using last 2 vital

result
```

```
Standard deviations (1, .., p=2):
[1] 14.462526  8.564503

Rotation (n x k) = (2 x 2):
                   PC1        PC2
Systolic_BP  0.9014644  0.4328533
Diastolic_BP 0.4328533 -0.9014644
```

In the result, you see two tabs. Standard deviation is squared root of (sum of variance squared), where it shows how well each data points are separated when you plot all data into each PC. For example, if you plot all points on PC1, standard deviation is 2.14, while using PC2 gives std of 0.106.

Rotation is where it shows loading of each features. Here, we see that PC1 seperates SBP well, while PC2 separates DBP well.

But our interest is "How much variance does each PC capture," not if points are sepereated. prcomp() is consist of multiple lists. Let's look at some other components in prcomp().

```
summary(result)
```

```
Importance of components:
                          PC1    PC2
Standard deviation     14.4625 8.5645
Proportion of Variance  0.7404 0.2596
Cumulative Proportion   0.7404 1.0000
```

Proportion of variance is how much each PC explain the variance. As you can see, in PCx, number x is labeled in a order of explained variance (PC that captures the most becomes first PC). Cumulative proportion is total variance explained using All PCs starting from PC1 to xth. Since we started with 2 axis, PC2 will give us 100% explained variance.

Besides rotation/std/explained variance/cumulative variance, we have one more data in prcomp(), which is center. Center is just avereage value of each features.

```
result$center
```

```
 Systolic_BP Diastolic_BP
      119.67        77.53
```

Let's revisit the equation we talked about earlier and see if result aligns with our explanation (PC1 capturing variance well).

$$Z_k = a_{1k}(X_1 - \bar{X}_1) + a_{2k}(X_2 - \bar{X}_2) + \cdots + a_{pk}(X_p - \bar{X}_p)$$

Then, our PC1 will be

$$Z_1 = (Loading\ of\ SBP)(SBP - Avg\ SBP) + (Loading\ of\ DBP)(DBP - Avg\ DBP)$$

Let's vizualize PC1 with our data.

Note1: I'm saving each loading into separate variable for readability of the code, but you can just use result$rotation and center directly

Note2: Let's talk about why slope and intercept equation looks like that.

Slope - loading was 'how much of feature is used in PC.' If 100% of feature 1 is used, loading of feature 1 would be 1.0. But what does that really mean? If point move 1 unit along feature

PC, it will also move 1 unit on feature 1 axis. If loading of feature 1 was 0.4, point would move 0.4 unit along feature 1 axis everytime it moves 1 unit on the PC.

Slope is (change of y) / (change of x). If x axis is feature 1 and y axis is feature 2, what would happen when point moves by 1 unit along PC? point will move (loading of feature 1) on y direction, and (loading of feature 2) on x direction. Now we have slope! (Loading1) / (Loading 2).

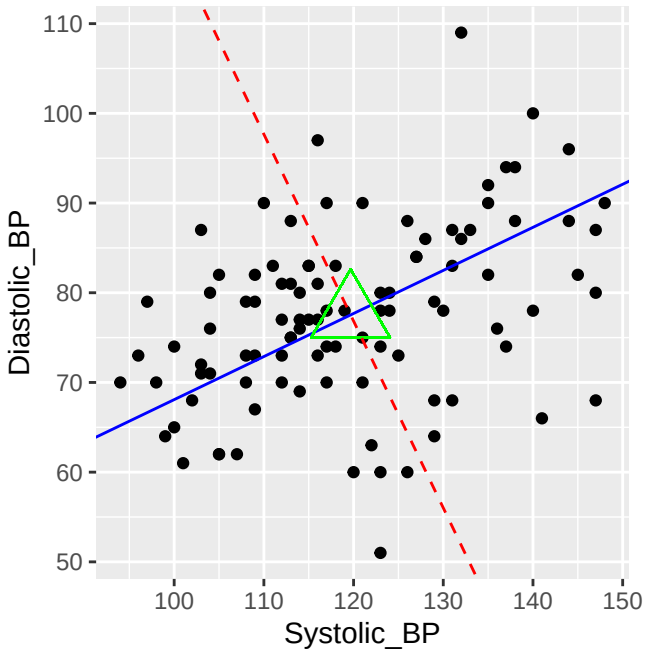
Intercept - b = y - ax. Since we know slope, we need x and y values that's on the PC. How would you find (x,y) along PC when all data points are scattered around and we don't know equation of PC? Recall how PC is calculated: PC goes through the average of all features because of mean-centering (moving averages to (0,0))! That said, we can use center values stored in the result.

```
Load_SBP <- result$rotation[1,1]
Load_DBP <- result$rotation[2,1]
Avg_SBP <- result$center[1]
Avg_DBP <- result$center[2]


p <- ggplot(vitals, aes(Systolic_BP, Diastolic_BP)) +
  geom_point() + #plotting data points
  geom_abline(slope = Load_DBP / Load_SBP,
              intercept = Avg_DBP - Avg_SBP * (Load_DBP / Load_SBP), color ="blue") + #plotting l
  geom_point(x = result$center[1], y = result$center[2], color="green", size=10, shape=2) #pl
```

Does shape of PC1 make sense? if you try to spin it around the center, can you find a line where points are spread throughout PC further? No.

Similarly, let's add PC2.

```
# 1. Grab the loading for Principal Component 2 (Column 2)
Load2_SBP <- result$rotation[1,2]
Load2_DBP <- result$rotation[2,2]

# 2. Add the PC2 line to existing plot 'p'
p +
  geom_abline(slope = Load2_DBP / Load2_SBP,
              intercept = Avg_DBP - Avg_SBP * (Load2_DBP / Load2_SBP),
              color = "red",
              linetype = "dashed") +
  coord_fixed(ratio = 1) #This makes visualized x and y range identical
```



Look at relationship between PC1 and PC2. Do you notice anything? PC1 and PC2 are orthogonal (perpendicular)! Let's revisit the result

```
summary(result)
```

```
Importance of components:
                          PC1    PC2
Standard deviation     14.4625 8.5645
Proportion of Variance  0.7404 0.2596
Cumulative Proportion   0.7404 1.0000
```

It says PC1 explains about 74% of variance. Then, where are 26% of missing spread? Look at our visualized PCs in the figure above. If you compress all points onto PC1 (blue line), think about what happens to points that are stacked in (perpendicular to PC1) direction: They will be on the exact same spot on the PC1. This means that if we use only PC1, we cannot explain any variance that are spread perpendicular to PC1!

Losing information (variance) in this manner also means that if we introduce PC2, which will explain information missed in PC1, it will be perpendicular direction!

As result, every time we add new PC, it will be orthogonal to all other PCs.

Now, let's perform PCA using all values in the data frame.

```r
vital_pcr <- vitals %>%
  select(-Patient_ID, -Diabetes_Status) %>% #We perform PCA on numbers, not letters or ID (no
  prcomp()

summary(vital_pcr)
```

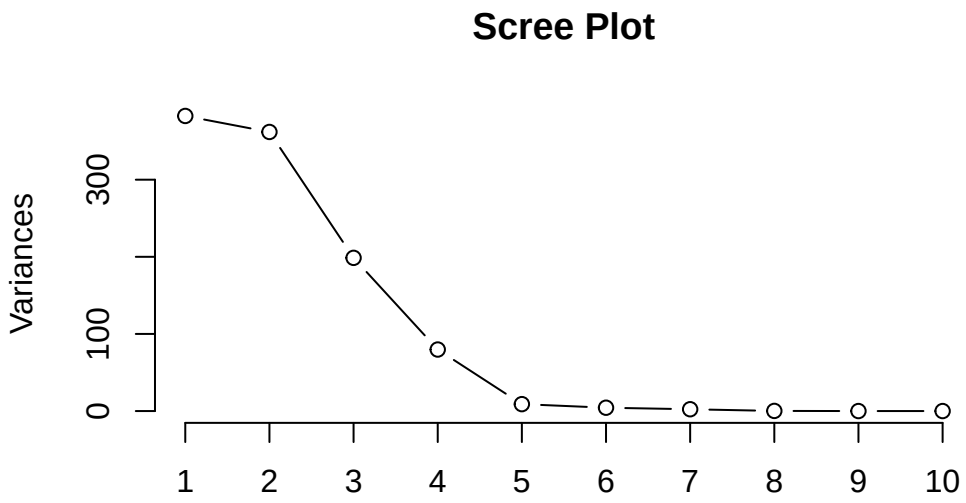
```
Importance of components:
                          PC1     PC2     PC3    PC4     PC5     PC6     PC7
Standard deviation     19.5610 19.0226 14.0945 8.9321 2.99623 2.08373 1.53297
Proportion of Variance  0.3683  0.3483  0.1912 0.0768 0.00864 0.00418 0.00226
Cumulative Proportion   0.3683  0.7167  0.9079 0.9847 0.99337 0.99755 0.99982
                           PC8     PC9    PC10
Standard deviation     0.43630 0.02134 0.01314
Proportion of Variance 0.00018 0.00000 0.00000
Cumulative Proportion  1.00000 1.00000 1.00000
```

9.2.3 Scree Plot

Now, we have 10 PCs because we have 10 features! When we perform further data analysis, should we use all 10PCs? No, the purpose of PCA was to reduce dimension to resolve 3 issues we discussed earlier. Then how many PCs should we use? Threshold percent of cumulative explained variance, we talked about it during the meeting.

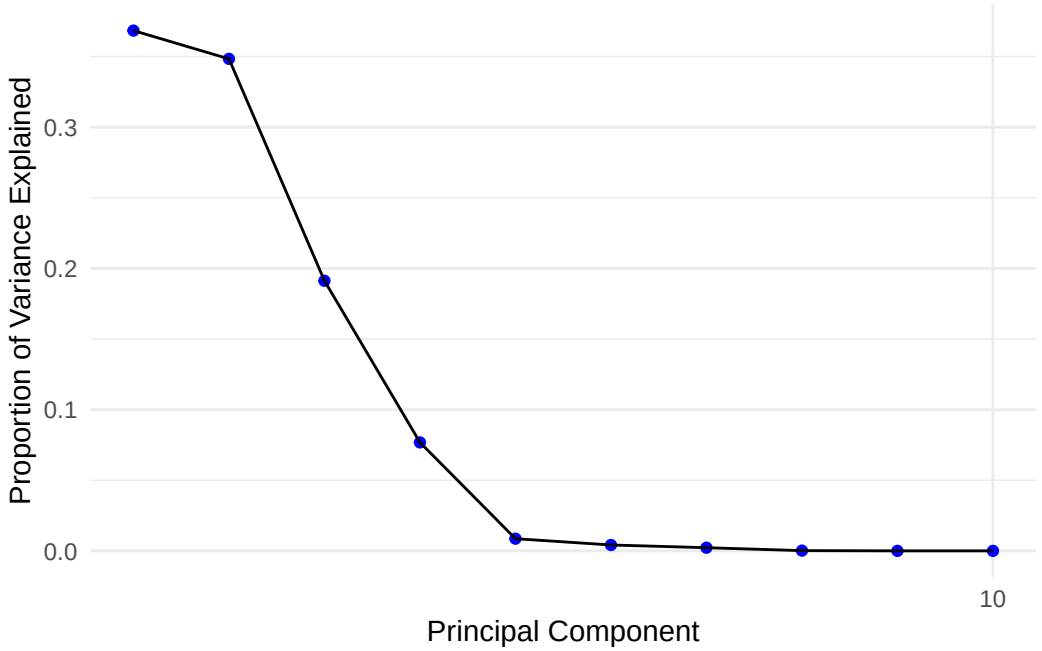
Here, let's talk about another method of determining how many PCs we should use: Elbow method through scree plot. Scree plot is a figure where you plot proportion of variance per PC (x axis - PCs, y axis - explained variance). To draw a plot, we use screeplot().

```r
screeplot(vital_pcr, type = "lines", main = "Scree Plot") #If you leave type empty, default
```



Or, we can save percent variances manually and use ggplot().

```r
#1. Square std to find variance, turning it into percent variance
var_prop <- vital_pcr$sdev^2 / sum(vital_pcr$sdev^2)

#2. Create var_prop data frame for ggplot()
scree_data <- data.frame(PC = 1:length(var_prop), Explained_Variance = var_prop)

ggplot(scree_data, aes(PC, Explained_Variance)) +
  geom_point(color = "blue") +
  geom_line(color = "black") +
  scale_x_continuous(breaks = length(scree_data$PC)) +
  labs(x = "Principal Component", y = "Proportion of Variance Explained") +
  theme_minimal()
```



While we want to reduce the dimension, we don't want to lose too much data. So, we want to keep the number of PCs where "adding another PC does not add any meaningful information" There are many ways to choose this number, but we'll review only few common methods, from simplist (crude analysis) to complex (rigorous analysis):

1. Using cumulative explained varariance (typically over 70~90% depends on the purpose). Limitation: This barely differentiate what's the boundary of PC that captures meaningful and negligible variance.
2. Elbow method, where you see variance of PC decrease most significantly or starts to flat out. Since elbow is where information start to become meaningless, we use number of PCs before the eblow. Limitation: standard of "elbow" and "flatness" is subjective, potentially hurting credibility of analysis.
3. Using Kaiser Criterion (rule of 1), where only PCs with raw variance higher than 1 are used. Limitation: The Kaiser Criterion is easily fooled by redundant data. The Rule of 1 assumes that every column in our dataset represents a unique, valuable piece of information. However, clinical data often contains redundant features (e.g., exporting "Manual Pulse," "Machine Heart Rate," and "ECG Heart Rate" for the same patient). Because these three columns perfectly correlate, PCA will group them together into a single Principal Component. That PC will capture a massive amount of variance higher than 1.0. The Kaiser Criterion will tell us this is the most "meaningful" trend in our clinic, but in reality, it is just the exact same vital sign repeated three times. It artificially inflates the variance without adding any real clinical value.

IMPORTANT: To use this method, you must standardize the data set, which we will discuss in later.

4. Using Parallel Analysis, where computer creates data set with random numbers of same features and rows. Because values in the data set are 'random,' (as they're spread equally), there is no pattern or 'which PC are significant.' as result, it will show linear-like line on the scree plot. We compare our scree plot and random-value based scree plot, we conclude which PCs contains lesser information than 'random.'

To perform parallel analysis, we use package called psych. Install it in your console.

```
install.packages("psych", repos = "http://cran.rstudio.com/")
```

Then, you can generate scree plot from performing PCA of randomly-generated values that has same number of features/entries with our vital data frame.

```
fa.parallel() is the function we use.

fa.parallel(our actual data frame, mode, number of random entries we need, main (if you want
```

Before we run the code, don't forget to load the package!

```
library(psych)

#fa = "PC" tells code that we're doing parallel analysis for PCA
#we need to feed raw data set that we run pcr, not the pcr result. Don't but vital_pcr!

vital_data <- vitals %>%
  select(-Patient_ID, -Diabetes_Status)

pa_result <- fa.parallel(vital_data, fa = "PC", main = "Parallel Analysis")
```

If you ran the code, you probably got following error:

```
Warning in fac(r = r, nfactors = nfactors, n.obs = n.obs, rotate = rotate,  :
  An ultra-Heywood case was detected.  Examine the results carefully
```

Heywood case means two or more features have 100% or higher multicollinearity. This make sense! If you revisit features, we have derived features like MAP, which are basically same thing as regular SBP/DBP since the source of features are identical. So, code is telling us "your data set is not processed well enough for PCA!"

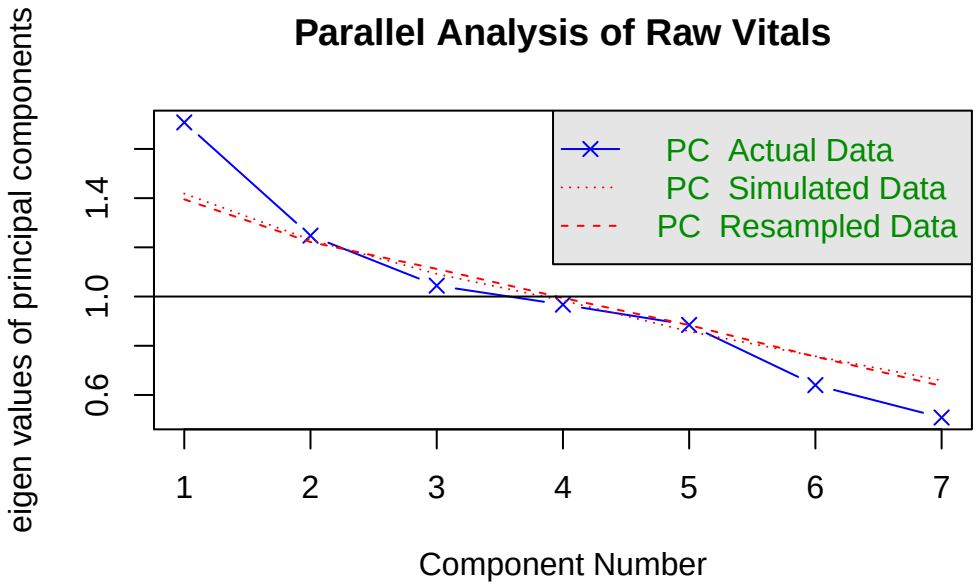
So, let's remove those overlapping features and try again.

```
library(psych)
```

```
        : 'psych'
```

```
The following objects are masked from 'package:ggplot2':

    %+%, alpha
```

```
clean_vitals <- vitals[, c("Age", "Temp_C", "Heart_Rate", "Systolic_BP", "Diastolic_BP", "Res
pa_result <- fa.parallel(clean_vitals, fa = "pc", main = "Parallel Analysis of Raw Vitals")
```




```
Parallel analysis suggests that the number of factors =  NA  and the number of components =
```

- PC Actual data: scree plot of PCA using original data set
- PC Simulated data: scree plot of PCA using randomly generated values PC
- PC Resampled data: scree plot of PCA using values from original data set but randomly mixed

Both PC simulated/resampled data establish baseline noise (zero statistical significance) level. Any PC that has meaningful separation of data points will have higher explained variance than variance explained by baseline noise. In the plot above, code concluded that PC1 alone is good enough dimension reduction for further analysis (number of components = 1).

Limitation: Sometimes, these random baseline noise underestimate of real components when data set contains overwhelmingly dominant pattern. This imbalanced data cause smaller (but clinically important) signals to be discarded as noise.

For example, if clinic overflows with fever patients, you will see high variance in a lot of vital signs, like body temperature.This huge variance makes small change in SpO2, which is still clinically significant, appears PCs with small variance captured. As result, important information looks like a random noise.

5. Resampling PCA

Resampling in here is different to resampled data you saw in #4. Resampled data is mixing data randomly (e.g., exchanging values in [row1, column1] with [row1, column3]). Resampling data means you're extracting subset/portion of data to perform analysis. For example, executing PCA with 60% raw data and remaining 40% on validating the precision/accuracy of the result.

So we understand that resampling split data to run PCA. But, how do we test the result with reamining data? This process is called reconstruction. Think this like derivation and integral! (where all result ends with "+C")

1) When we reduce dimension we will get a PC score per PC axis. For example, original (1,2) -> PC +1.3 (like taking derivative of a function). This part will be done by using only portion of the data.

2) We calculate PC scores using remaining data which weren't used for PC weight/loadings calculation. (Testing phase)

3) Then, we try to backtrack/guess what the original location of testing points using the PC data. For example, because (1,2) become PC score of 1.3, when we see one of the testing point's PC score is 1.3, we guess that that point was originally at (1,2)!

4) But because reduced dimension always lose some information, guessing original data will have error (like +C in integral). For example, we guess calculated PC score of 1.3 is originally located at (1,2). But, it was actually at (1,3) checking at origianl coordinate of testing set!

5) Based on how 'off' the guessed location is from original data, we can calculate magnitude of error. Now we repeat the step 1~4 using 0 (complete random guess) ~ feature of original data set (no dimension reduced), calculting error for each number of PCs used. The lower the error is, more accurate and stable the model is.

Let's look at how it's performed using missMDA package, which helps us with resampling testings.

```
#1. Install package that helps us dividing data sets and running cross validation (resamplin
# install.packages("missMDA")
library(missMDA)


#2. Run the Cross-Validation Stress Test
# ncp.max = 6: Tells the computer to test models using anywhere from 0 to 6 components
# method.cv = "Kfold": Tells the computer to use the "hide 20% of the patients" method

cv_result <- estim_ncpPCA(clean_vitals, ncp.max = 6, method.cv = "Kfold")
```

```
  |
  |                                                                      |   0%
  |
  |=                                                                     |   1%
  |
  |=                                                                     |   2%
  |
  |==                                                                    |   3%
  |
  |===                                                                   |   4%
  |
  |====                                                                  |   5%
  |
  |====                                                                  |   6%
  |
  |=====                                                                 |   7%
  |
  |======                                                                |   8%
  |
  |======                                                                |   9%
  |
  |=======                                                               |  10%
  |
  |========                                                              |  11%
  |
  |========                                                              |  12%
  |
```

```
|=========                                                             |  13%
|
|==========                                                            |  14%
|
|===========                                                           |  15%
|
|===========                                                           |  16%
|
|============                                                          |  17%
|
|=============                                                         |  18%
|
|=============                                                         |  19%
|
|==============                                                        |  20%
|
|===============                                                       |  21%
|
|================                                                      |  22%
|
|================                                                      |  23%
|
|=================                                                     |  24%
|
|==================                                                    |  25%
|
|==================                                                    |  26%
|
|====================                                                  |  27%
|
|====================                                                  |  28%
|
|=====================                                                 |  29%
|
|=====================                                                 |  30%
|
|======================                                                |  31%
|
|=======================                                               |  32%
|
|=======================                                               |  33%
|
|========================                                              |  34%
```

```
|
|=========================                                              |  35%
|
|=========================                                              |  36%
|
|==========================                                             |  37%
|
|===========================                                            |  38%
|
|============================                                           |  39%
|
|============================                                           |  40%
|
|=============================                                          |  41%
|
|=============================                                          |  42%
|
|==============================                                         |  43%
|
|==============================                                         |  44%
|
|===============================                                        |  45%
|
|===============================                                        |  46%
|
|================================                                       |  47%
|
|================================                                       |  48%
|
|=================================                                      |  49%
|
|==================================                                     |  51%
|
|==================================                                     |  52%
|
|===================================                                    |  53%
|
|===================================                                    |  54%
|
|====================================                                   |  55%
|
|====================================                                   |  56%
|
```

```
|=======================================                               |  57%
|
|=======================================                               |  58%
|
|========================================                              |  59%
|
|========================================                              |  60%
|
|=========================================                             |  61%
|
|=========================================                             |  62%
|
|==========================================                            |  63%
|
|==========================================                            |  64%
|
|===========================================                           |  65%
|
|===========================================                           |  66%
|
|============================================                          |  67%
|
|============================================                          |  68%
|
|=============================================                         |  69%
|
|=============================================                         |  70%
|
|==============================================                        |  71%
|
|==============================================                        |  72%
|
|===============================================                       |  73%
|
|===============================================                       |  74%
|
|================================================                      |  75%
|
|================================================                      |  76%
|
|=================================================                     |  77%
|
|=================================================                     |  78%
```

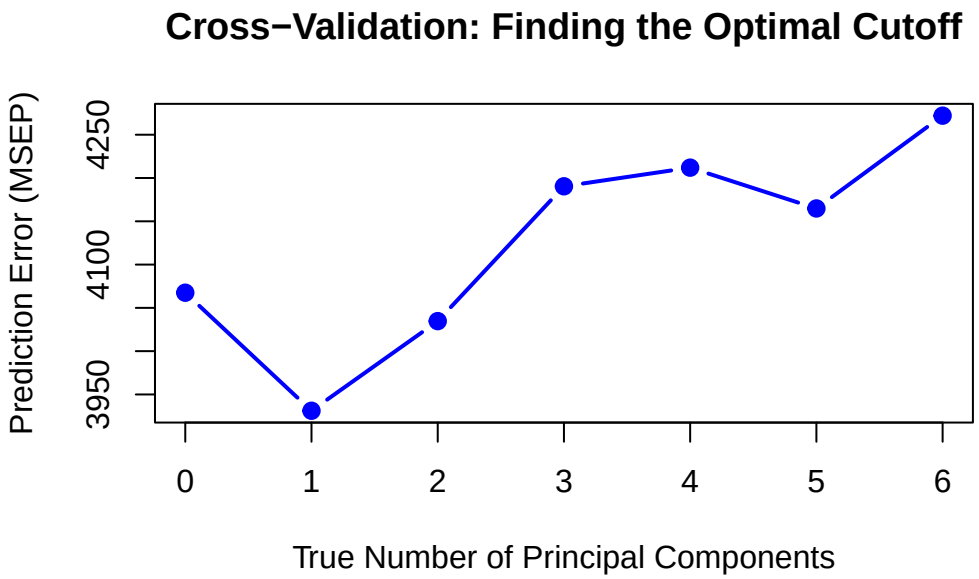
```
|
|=============================================================          |  79%
|
|==============================================================         |  80%
|
|===============================================================        |  81%
|
|================================================================       |  82%
|
|=================================================================      |  83%
|
|==================================================================     |  84%
|
|===================================================================    |  85%
|
|====================================================================   |  86%
|
|=====================================================================  |  87%
|
|======================================================================= |  88%
|
|======================================================================== |  89%
|
|=========================================================================|  90%
|
|=========================================================================|  91%
|
|=========================================================================|  92%
|
|=========================================================================|  93%
|
|=========================================================================|  94%
|
|=========================================================================|  95%
|
|=========================================================================|  96%
|
|=========================================================================|  97%
|
|=========================================================================|  98%
|
|=========================================================================|  99%
|
```

```
  |======================================================================| 100%
```

```
cv_result$ncp
```

```
[1] 1
```

```
# Create the correct X-axis numbers (starting at 0)
# length(cv_result$criterion) is 7, so this makes a sequence: 0 ~ 6
x_values <- 0:(length(cv_result$criterion) -  1)

# Draw the plot with the explicitly defined X and Y axes
plot(x = x_values,
     y = cv_result$criterion,
     type = "b",
     pch = 19,
     col = "blue",
     lwd = 2,
     xlab = "True Number of Principal Components",
     ylab = "Prediction Error (MSEP)",
     main = "Cross-Validation: Finding the Optimal Cutoff")
```



Now, you see that using 2PC is best dimension reduction that reflects nature of original data structure.

Question to think: using 6 PCs, where no dimension reduction is used, error is high. You might think if there is no dimension reduction, there is no information loss, so that error should be low. Why error is high? Contact me if you cannot think of why!

9.2.4 Loading Plot

We discussed what loading is. Then, can you guess what loading plot is? Yes, it's a plot that shows loadings of each feature. We do draw loading plot because looking and comparing 'which feature influence seperation on the xth PC' becomes difficult as number of features increase (the more numbers there are, the longer/difficult it becomes!)

There are few ways to do this.

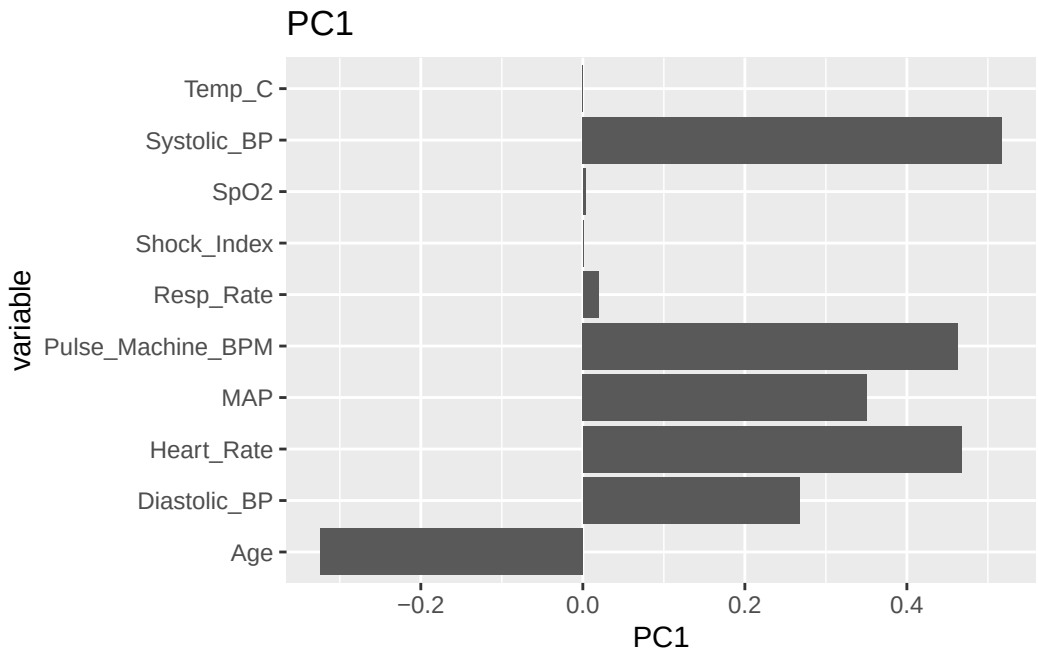
1. First is a long ggplot() way, by extracting and plotting like we did in chapter8 (try it yourself)

```
#1. Let's seperate loading values.

loading <- data.frame(vital_pcr$rotation[,1:6]) %>%
            rownames_to_column(var="variable")
loading
```

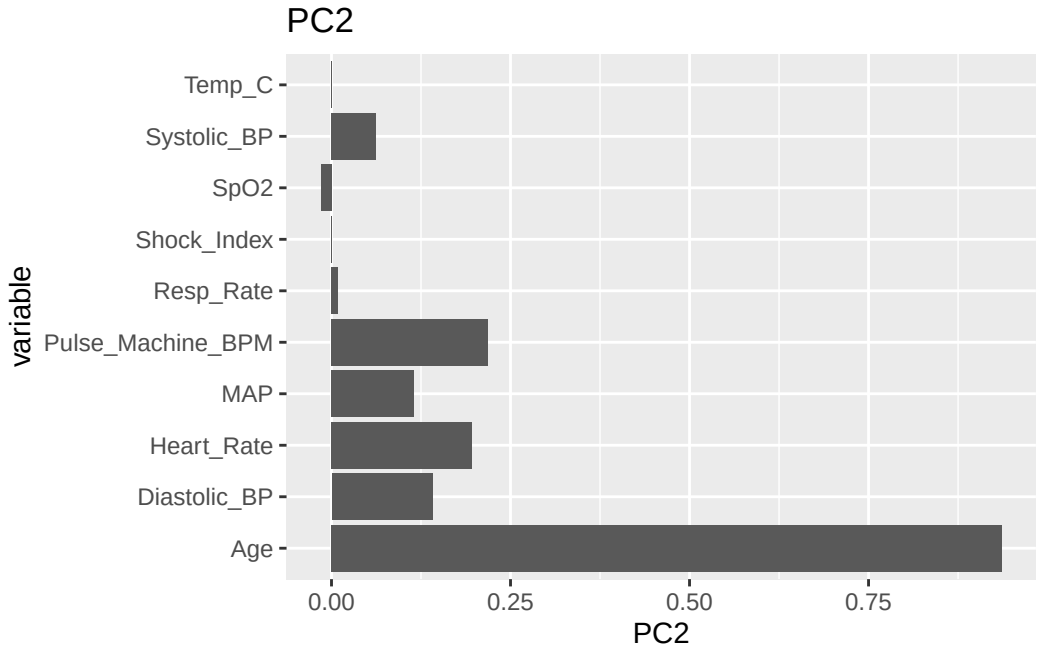
```
            variable           PC1           PC2          PC3           PC4
1                Age -0.3236863010  0.9361901311 -0.084249562 -0.105983485
2             Temp_C -0.0003356208  0.0009366694 -0.001975027 -0.005652870
3         Heart_Rate  0.4670868614  0.1961732193  0.477627572  0.033767990
4  Pulse_Machine_BPM  0.4630652068  0.2188213971  0.501068703  0.017514856
5        Systolic_BP  0.5172484705  0.0618204320 -0.482165023 -0.650585843
6       Diastolic_BP  0.2675689068  0.1410245424 -0.350258589  0.706542812
7                MAP  0.3508646118  0.1146822356 -0.393838345  0.254382136
8          Resp_Rate  0.0192148295  0.0092466844 -0.052545900  0.002680935
9               SpO2  0.0035140865 -0.0152134667 -0.024913779 -0.009569748
10       Shock_Index  0.0009537038  0.0013311862  0.006750500  0.003843825
             PC5          PC6
1  -0.005762431 -0.015056545
2   0.009600738  0.029395799
3   0.132753631 -0.197155830
4  -0.105180313  0.161923270
5  -0.030369325  0.022269051
6  -0.028145509  0.006687559
7  -0.028488688  0.012973984
8   0.970417772 -0.116795998
```

```
9  -0.164162830 -0.958886957
10  0.001427475 -0.002271849
```

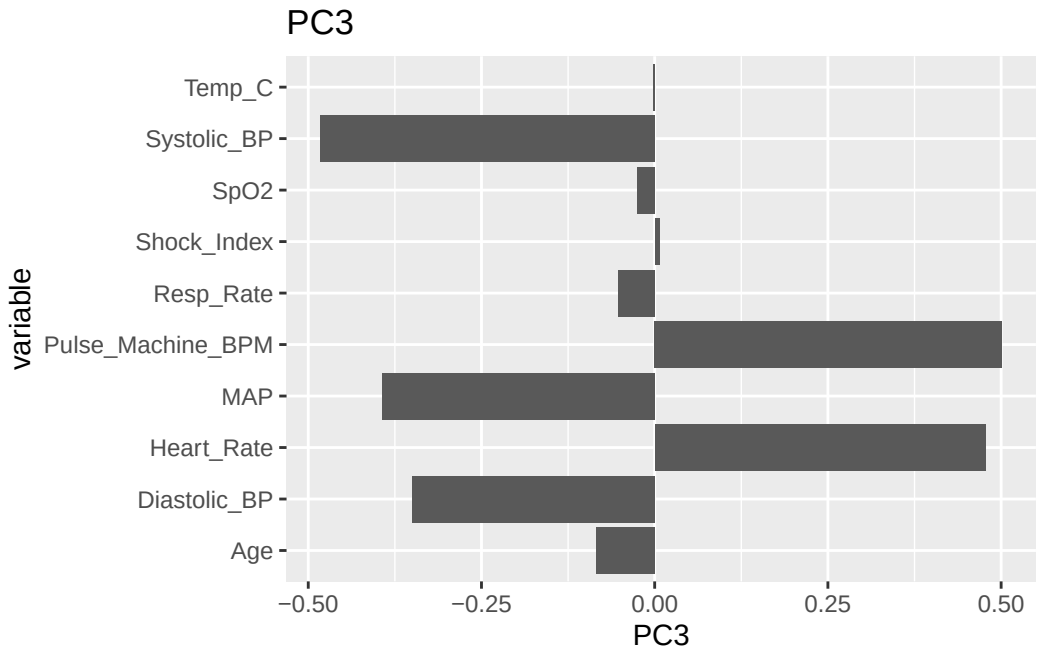
```r
#2. Let's plot them. Here, we only did up to PC3, but remember that we actually had 6 feature

# 1. Install and load patchwork (the best plot-stitching tool in R)
library(ggplot2)

# 2. Build three individual plots using your original wide columns
p1 <- ggplot(loading, aes(x = variable, y = PC1)) +
  geom_col() +
  coord_flip() +
  ggtitle("PC1")

p2 <- ggplot(loading, aes(x = variable, y = PC2)) +
  geom_col() +
  coord_flip() +
  ggtitle("PC2")

p3 <- ggplot(loading, aes(x = variable, y = PC3)) +
  geom_col() +
  coord_flip() +
  ggtitle("PC3")

p1
```


PC1
Temp_C
Systolic_BP
SpO2
Shock_Index
Resp_Rate
Pulse_Machine_BPM
MAP
Heart_Rate
Diastolic_BP
Age
variable
−0.2
0.0
0.2
0.4
PC1

```
p2
```


PC2
Temp_C
Systolic_BP
SpO2
Shock_Index
Resp_Rate
Pulse_Machine_BPM
MAP
Heart_Rate
Diastolic_BP
Age
variable
0.00
0.25
0.50
0.75
PC2

```
p3
```



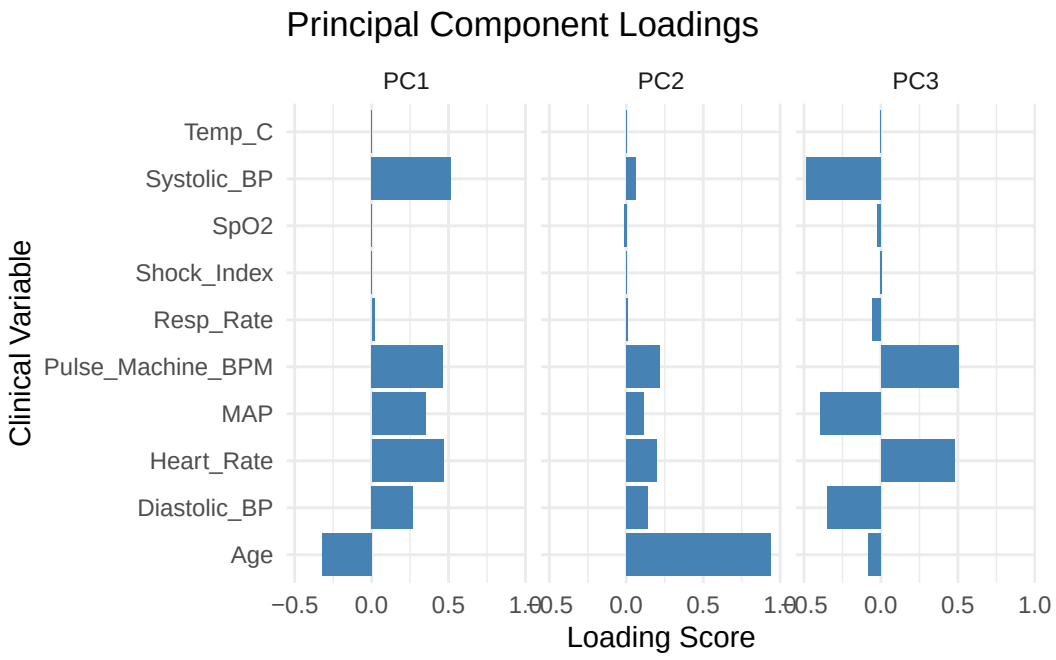
But, repeating same code over and over takes time and tiring. This is because you need to pick one column per axis, which force you to write ggplot() per column. To resolve this issue, we can do something called

`pivot_longer()`

This function tells ggplot to combine data across different column so that you can assign multiple column for one axis. Then, we could use facet_wrap() because each column becomes subtype you can seperate!

```
loading %>% #Pass loading data pivot_loner()
  pivot_longer(cols = c(PC1, PC2, PC3),
               names_to = "component",
               values_to = "loading_score") %>% #Pass combined data to ggplot() without varia
  ggplot(aes(x = variable, y = loading_score)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  facet_wrap(~ component) +
  labs(title = "Principal Component Loadings",
       x = "Clinical Variable",
```

```
       y = "Loading Score") +
  theme_minimal()
```

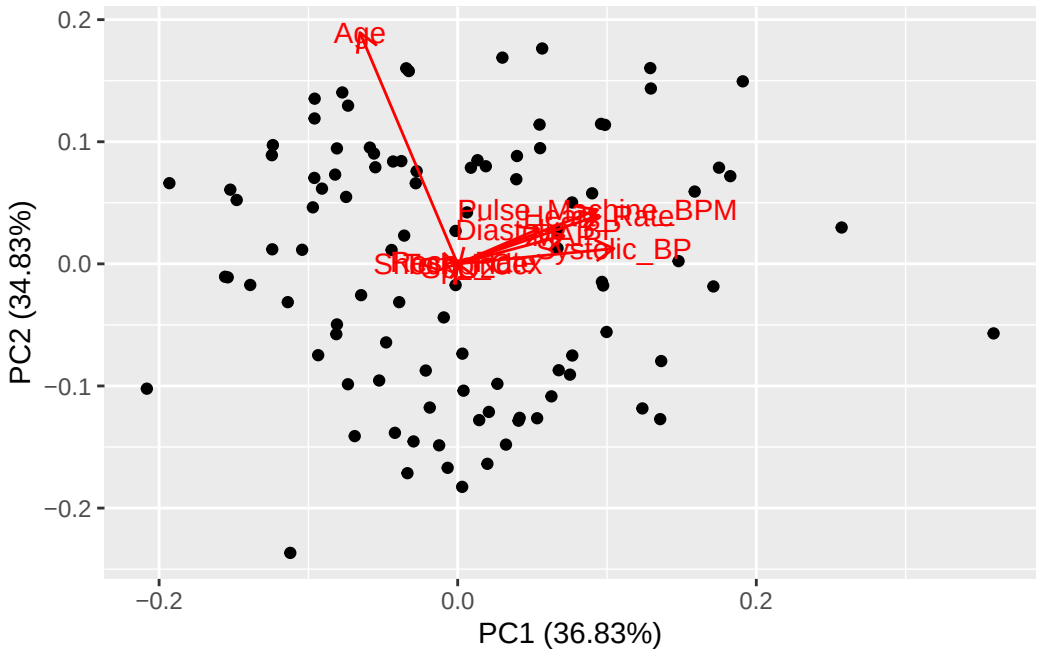


9.2.4.1 loading on PCA plot

While checking loadings per PC helps us with understanding how each feature influence each PC, it is difficult to understand "So what's feature's influence on overall/full PCA plot" There are few ways to do it.

1. First way is ggplot(). There is a separate package called ggfortify, which allow us to build ggplot() for PCA result. We use autoplot() to visualize the result, and change what varaible we want to plot by using the arguments. Google for plotting different features!

```
library(ggfortify)

autoplot(vital_pcr, loadings = TRUE, loadings.label = TRUE, loadings.label.size = 4)
```




```
# loadings = true (plot loading on PCA plot)
# loadings.label = true (label which arrow is which feature)
```

Now, instead of reading multiple loading plots separately and say "higher age pushes points to the left side of PC1, and upper side of PC2," we can look at PCA plot and recognize how each features change point's location on PCA directly.

9.3 Conclusion

Today, we looked at how we reduce dimension through PCA. Remember, 1) Standarize data set to remove population skewness 2) Decide how many PCs do you need to use 3) Run PCA.

Next chapter, we will look at how to cluster them!